Embodied AI

From Perception-Action Loops to Foundation Models

Nurdun Mijit

May 26, 2026

Contents

Preface **vi**

I Foundations **1**

1 What is Embodied AI **2**
1.1 Motivation . . . 2
1.2 The Perception-Action Loop . . . 2
1.3 POMDP Formulation . . . 2
1.4 Key Distinctions from Disembodied AI . . . 3
1.4.1 Sequential Decision-Making under Uncertainty . . . 3
1.4.2 Active Perception . . . 3
1.4.3 Physical Constraints . . . 4
1.4.4 Long Horizons . . . 4
1.5 The Embodiment Spectrum . . . 4
1.6 Intellectual Foundations . . . 4
1.6.1 Brooks' Subsumption Architecture . . . 4
1.6.2 Gibson's Affordances . . . 5
1.7 Modern Framing . . . 5
1.8 Key Benchmarks . . . 5
1.9 The Perception-Action Loop as Information Processing . . . 5
1.10 Open Problems . . . 6

2 Simulation Environments and Sim-to-Real Transfer **7**
2.1 Motivation . . . 7
2.2 Simulation as a Computational Abstraction . . . 7
2.3 Habitat: High-Performance Visual Simulation . . . 7
2.3.1 Habitat-Sim . . . 7
2.3.2 Habitat-Lab . . . 8
2.3.3 Scanned 3D Datasets . . . 8
2.4 AI2-THOR: Interactive Object Manipulation . . . 8
2.5 iGibson and Full Physics . . . 9
2.5.1 Object States . . . 9
2.5.2 BEHAVIOR Benchmark . . . 9
2.6 NVIDIA Isaac Sim . . . 9
2.7 Other Simulation Platforms . . . 9
2.7.1 MuJoCo . . . 9
2.8 The Sim-to-Real Gap . . . 10
2.9 Domain Randomization . . . 10
2.9.1 Canonical Example: OpenAI Rubik's Cube Hand . . . 11
2.10 Domain Adaptation . . . 11
2.11 System Identification . . . 11
2.12 Actuator Networks for Legged Locomotion . . . 11
2.13 Connections . . . 12
2.14 Open Problems . . . 12

II Navigation **13**

3 Visual Navigation: PointGoal and ObjectGoal **14**
3.1 Motivation . . . 14
3.2 Problem Formulation . . . 14
3.2.1 Observation Space . . . 14
3.2.2 Action Space . . . 14
3.3 PointGoal Navigation . . . 15
3.3.1 Classical Approach: Map + Plan + Execute . . . 15
3.3.2 End-to-End RL for PointNav . . . 15
3.3.3 DD-PPO and the Scaling Hypothesis . . . 15
3.3.4 Why GPS+Compass Matters . . . 16
3.4 ObjectGoal Navigation (ObjectNav) . . . 16
3.4.1 Problem Definition . . . 16
3.4.2 Semantic Mapping + Frontier Exploration . . . 16
3.4.3 SemExp: Learned Semantic Priors . . . 17
3.4.4 ZER: Zero-Shot ObjectNav via CLIP . . . 17
3.4.5 PONI: Potential Functions for ObjectNav . . . 17
3.5 Evaluation Metrics . . . 18
3.5.1 Success Rate . . . 18
3.5.2 SPL (Success Weighted by Path Length) . . . 18
3.5.3 SoftSPL and Progress Metrics . . . 18
3.6 Modular vs. End-to-End . . . 18
3.7 Connections . . . 19
3.8 Open Problems . . . 19

4 Vision-and-Language Navigation (VLN) **20**
4.1 Motivation . . . 20
4.2 Problem Formulation . . . 20
4.3 Datasets . . . 20
4.3.1 R2R: Room-to-Room . . . 20
4.3.2 R4R and RxR . . . 21
4.4 Evaluation Metrics . . . 21
4.4.1 Success Rate and Path Alignment . . . 21
4.4.2 Normalized Dynamic Time Warping (nDTW) . . . 21
4.5 Sequence-to-Sequence Baseline . . . 21
4.5.1 Instruction Encoding . . . 21
4.5.2 Visual Encoding . . . 22
4.5.3 Action Decoding . . . 22
4.5.4 Limitations . . . 22
4.6 Speaker-Follower: Data Augmentation and Pragmatic Reasoning . . . 22
4.6.1 Speaker Model . . . 22
4.6.2 Follower Model . . . 23
4.6.3 Pragmatic Inference . . . 23
4.6.4 Environmental Dropout . . . 23
4.7 EnvDrop: Robustness via Visual Perturbation . . . 23
4.8 PREVALENT: Pre-training for VLN . . . 24
4.8.1 Pre-training Tasks . . . 24
4.9 HAMT: History-Aware Multimodal Transformer . . . 24
4.9.1 Architecture . . . 24
4.9.2 Long-Term Memory via History Attention . . . 25
4.10 Cross-Modal Alignment as a Mathematical Challenge . . . 25

4.11 Connections . . . 25
4.12 Open Problems . . . 26

5 SLAM and Spatial Understanding 27
5.1 Motivation . . . 27
5.2 Core Concepts and Key Papers . . . 27
5.2.1 Probabilistic Formulation of SLAM . . . 27
5.2.2 EKF-SLAM: The Classical Filtering Approach . . . 28
5.2.3 FastSLAM: Rao-Blackwellized Particle Filtering . . . 29
5.2.4 Pose Graph SLAM: Optimization on Lie Groups . . . 29
5.2.5 Learnable SLAM and Beyond . . . 30
5.2.6 From Classical to Learned SLAM Architectures . . . 30
5.3 Key Techniques . . . 31
5.3.1 Loop Closure Detection . . . 31
5.3.2 Learned SLAM: Neural SLAM . . . 31
5.3.3 Implicit Representations for Dense SLAM . . . 32
5.3.4 Beyond Geometry: Semantic and Task-Driven Mapping . . . 33
5.4 Connections . . . 34
5.5 Open Problems . . . 34

III Manipulation & Interaction 36

6 Robotic Grasping 37
6.1 Motivation . . . 37
6.2 Core Concepts and Key Papers . . . 37
6.2.1 The Grasp Synthesis Problem . . . 37
6.2.2 Force Closure: The Foundation of Grasp Stability . . . 37
6.2.3 Grasp Quality Metrics . . . 38
6.2.4 Dex-Net: Data-Driven Grasp Quality at Scale . . . 39
6.2.5 GG-CNN: Generative Grasping . . . 40
6.3 Key Techniques . . . 40
6.3.1 6-DoF Grasp Detection from Point Clouds . . . 40
6.3.2 Contact-GraspNet and Implicit Grasp Representations . . . 41
6.3.3 Task-Oriented and Semantic Grasping . . . 41
6.3.4 The Complete Grasping Pipeline . . . 42
6.4 Connections . . . 42
6.5 Open Problems . . . 43

7 Language-Conditioned Manipulation 44
7.1 Motivation . . . 44
7.2 Core Concepts and Key Papers . . . 44
7.2.1 CLIPort: Language-Grounded Transporter Networks . . . 44
7.2.2 SayCan: Grounding Language in Affordances . . . 45
7.2.3 RT-1: Robotics Transformer . . . 46
7.2.4 InstructRL: LLM-Generated Reward Functions . . . 47
7.2.5 RT-2: Vision-Language-Action Models . . . 47
7.3 Key Techniques . . . 48
7.3.1 Language Grounding via Multimodal Fusion . . . 48
7.3.2 Compositional Generalization . . . 48
7.3.3 Behavioral Cloning and the Compounding Error Problem . . . 49
7.4 Connections . . . 49

7.5 Open Problems . . . 50

8 Long-Horizon Task Completion 51
8.1 Motivation . . . 51
8.2 Core Concepts and Key Papers . . . 51
8.2.1 The ALFRED Benchmark and the Anatomy of Difficulty . . . 51
8.2.2 Task and Motion Planning: The Classical Synthesis . . . 52
8.2.3 FILM: LLM-Guided Modular Execution . . . 53
8.2.4 Episodic Transformer: End-to-End Memory-Based Policy . . . 54
8.3 Key Techniques . . . 55
8.3.1 Skill Chaining and the Options Framework . . . 55
8.3.2 Progress Monitoring and Recovery . . . 56
8.3.3 Hierarchical Reinforcement Learning . . . 56
8.3.4 Plan Verification and Monitoring via Language Models . . . 57
8.4 Connections . . . 57
8.5 Open Problems . . . 58

IV Learning Paradigms 60

9 Imitation Learning for Robotics 61
9.1 Motivation . . . 61
9.2 Behavioral Cloning and Covariate Shift . . . 61
9.2.1 Formulation . . . 61
9.2.2 The Covariate Shift Problem . . . 61
9.2.3 Error Bound Analysis . . . 62
9.3 DAgger: Dataset Aggregation . . . 62
9.3.1 The Core Algorithm . . . 62
9.3.2 Theoretical Guarantee . . . 63
9.4 Inverse Reinforcement Learning . . . 63
9.4.1 The IRL Framework . . . 63
9.4.2 Maximum Entropy IRL . . . 63
9.4.3 Generative Adversarial Imitation Learning . . . 64
9.5 Learning from Play . . . 65
9.6 Action Chunking with Temporal Ensembles . . . 66
9.7 Key Techniques and Practical Considerations . . . 67
9.8 Connections to Other Fields . . . 67
9.9 Open Problems . . . 68
9.10 Summary . . . 69

10 Embodied Reinforcement Learning 70
10.1 Motivation . . . 70
10.2 Core Concepts and Key Papers . . . 70
10.2.1 The Embodied Markov Decision Process . . . 70
10.2.2 Sim-to-Real Transfer: The Central Problem . . . 71
10.2.3 Asymmetric Actor-Critic . . . 72
10.2.4 Recurrent RL for Partial Observability . . . 73
10.3 Key Techniques . . . 73
10.3.1 Exploration in Sparse Reward Settings . . . 73
10.3.2 Multi-Task Embodied RL . . . 73
10.4 Connections . . . 74
10.5 Open Problems . . . 74

10.6 Summary 75

V Foundation Models & Frontiers 76

11 Foundation Models for Robotics 77
11.1 Motivation 77
11.2 Core Concepts and Key Papers 77
11.2.1 PaLM-E: Embodied Multimodal Language Model 77
11.2.2 RT-2: Vision-Language-Action Model 78
11.2.3 Octo: An Open-Source Generalist Policy 79
11.2.4 OpenVLA: An Open Vision-Language-Action Model 80
11.2.5 π_0: Flow Matching for Continuous Actions 80
11.2.6 The Common Architecture 81
11.3 Key Techniques 81
11.4 Connections 81
11.5 Open Problems 82
11.6 Summary 82

12 Frontiers and Future Directions 83
12.1 The State of Embodied AI 83
12.2 Capability Gaps 83
12.2.1 Dexterous Manipulation 83
12.2.2 Mobile Manipulation 83
12.2.3 Whole-Body Control 84
12.2.4 Long-Horizon Autonomy 84
12.2.5 Natural Interaction 85
12.3 Data: The Defining Bottleneck 85
12.4 The Road to General-Purpose Robots 86
12.5 Ethical Considerations 86
12.6 Recommended Reading Trajectory 87
12.7 Summary 87

Preface

Embodied AI studies artificial agents that perceive, reason, and act in physical or simulated 3D environments. Unlike "disembodied" AI—which operates on static datasets of images, text, or tabular data—embodied agents must close the perception-action loop: every action changes what they see next, and every observation informs their next decision. This coupling makes embodied problems fundamentally sequential, interactive, and rich.

This tutorial covers the key ideas, seminal papers, and foundational techniques of Embodied AI, organized by capability: navigation, mapping, manipulation, task completion, and the learning paradigms that power them. The narrative traces a continuous arc from classical geometric approaches through deep reinforcement learning to the emerging era of vision-language-action foundation models.

Each chapter is built around the critical papers that defined a problem or solved it. We derive the core mathematics, explain the architectures, and most importantly—explain *why* each idea matters and how it connects to the next. No pseudocode. Only equations, reasoning, and intuition.

Nurdun Mijit

Part I

Foundations

Chapter 1

What is Embodied AI

1.1 Motivation

For decades, the dominant paradigm in artificial intelligence treated cognition as symbol manipulation on pre-collected data. ImageNet classifiers, GPT-style language models, and AlphaFold all operate on static snapshots of the world: a curated dataset is assembled, a model is trained, and performance is evaluated on a held-out test split. This *internet AI* paradigm has produced spectacular results, but it sidesteps a fundamental property of natural intelligence—the fact that agents exist *inside* the world they perceive, and their actions change what they observe next.

The **Embodiment Hypothesis** states that intelligence requires a physical body interacting with a physical world [**?**]. Perception is not passive reception of data; it is *active* and *purpose-driven*. An agent must decide where to look, where to move, what to touch, and how to manipulate—and each decision alters the stream of future observations. This chapter formalizes embodiment as a mathematical problem, contrasts it with disembodied AI, and establishes the intellectual foundations upon which the rest of this book is built.

1.2 The Perception-Action Loop

At its core, an embodied agent executes a closed-loop interaction with its environment:

$$\cdots \rightarrow o_t \rightarrow \text{reason} \rightarrow a_t \rightarrow o_{t+1} \rightarrow \cdots \tag{1.1}$$

At every discrete time step t, the agent receives an observation o_t, selects an action a_t, and the environment transitions to a new state, yielding observation o_{t+1}. This loop continues until the task is completed or a horizon T is reached. Critically, o_{t+1} depends on a_t: the agent shapes its own data distribution. This makes embodied learning fundamentally different from supervised learning, where the data distribution is fixed and independent of the model's parameters.

The observation-action history up to time t is denoted:

$$\tau_t = (o_0, a_0, o_1, a_1, \ldots, o_t). \tag{1.2}$$

The agent's behavior is defined by a **policy** $\pi(a_t \mid \tau_t)$, which maps trajectories to a distribution over actions. The task is specified through a reward function $R(o_t, a_t, o_{t+1})$ or, more commonly in embodied settings, a success condition that evaluates to $\{0, 1\}$ at the terminal timestep. The agent's objective is to maximize cumulative expected reward:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{T-1} \gamma^t R(o_t, a_t, o_{t+1}) \right], \tag{1.3}$$

where $\gamma \in (0, 1]$ is the discount factor and the expectation is taken over the stochastic dynamics of the environment and the agent's own policy.

1.3 POMDP Formulation

Embodied problems are naturally modeled as **Partially Observable Markov Decision Processes** (POMDPs). A POMDP is a tuple $\langle S, A, O, T, R, \Omega, \gamma \rangle$ where:

$$S \quad \text{— state space (the true physical configuration of the world)} \tag{1.4}$$
$$A \quad \text{— action space (locomotion, manipulation, head rotation, etc.)} \tag{1.5}$$
$$O \quad \text{— observation space (egocentric RGB, depth, proprioception)} \tag{1.6}$$
$$T(s' \mid s, a) \quad \text{— transition dynamics} \tag{1.7}$$
$$R(s, a) \quad \text{— reward function} \tag{1.8}$$
$$\Omega(o \mid s) \quad \text{— observation model} \tag{1.9}$$
$$\gamma \quad \text{— discount factor.} \tag{1.10}$$

The defining property of a POMDP is that the agent never observes the true state s_t directly. Instead, it receives a partial observation $o_t \sim \Omega(\cdot \mid s_t)$ that provides incomplete and possibly noisy information. This **partial observability** is not an engineering nuisance—it is the central intellectual challenge of Embodied AI.

To make decisions under partial observability, the agent must maintain a **belief state** $b_t(s) = P(s_t = s \mid \tau_t)$, a probability distribution over the true state given the history. The belief update is performed via Bayesian filtering:

$$b_{t+1}(s') = \eta \cdot \Omega(o_{t+1} \mid s') \sum_{s \in S} T(s' \mid s, a_t)\, b_t(s), \tag{1.11}$$

where η is a normalization constant. In continuous, high-dimensional state spaces (the norm in Embodied AI), exact belief tracking is intractable, which motivates the use of learned representations—the subject of later chapters.

1.4 Key Distinctions from Disembodied AI

Four structural properties separate embodied intelligence from static-pattern recognition:

1.4.1 Sequential Decision-Making under Uncertainty

In classification, each example is independent. In embodiment, every decision is conditioned on a history of previous decisions and observations. The consequence of choosing a_t is not merely an immediate reward but a shift in the distribution of future states. This **temporal credit assignment** problem—which action in a chain of 500 steps was responsible for failure?—is exponentially harder than per-sample loss minimization.

1.4.2 Active Perception

Where you look determines what you see. A disembodied model receives the entire image and must classify it. An embodied agent must decide *what visual information to acquire next*: rotate the head, walk around an occluder, crouch to see under a table. This is formalized as the **Next-Best-View** problem: given a current belief b_t, which physical camera pose a_t maximizes the expected information gain?

$$a_t^* = \arg\max_{a \in A} \; \mathbb{E}_{o \sim \Omega(\cdot \mid s, a)} \left[\mathrm{KL}\left(b_{t+1}(\cdot \mid o, a) \parallel b_t \right) \right]. \tag{1.12}$$

Active perception makes vision fundamentally *sequential* and *goal-directed*.

1.4.3 Physical Constraints

Embodied agents operate in $\mathbb{R}^3$ Euclidean space. They cannot walk through walls, must respect joint limits, cannot exceed motor torques, and cannot grasp objects outside their kinematic reach. These constraints induce a notion of **reachability**: given initial pose $\mathbf{p}_0$ and configuration $\mathbf{q}_0$, the set of states reachable within k timesteps is:

$$\mathcal{R}_k(\mathbf{p}_0, \mathbf{q}_0) = \{s \in S \mid \exists(a_0, \ldots, a_{k-1}) \text{ s.t. } P(s_k = s \mid s_0) > 0\}\,. \tag{1.13}$$

Physical constraints mean that *not all states are accessible*, and the topology of the accessible region is determined by geometry and dynamics.

1.4.4 Long Horizons

ImageNet classification requires one forward pass. Embodied tasks often require hundreds or thousands of sequential decisions. The probability of compounding errors grows exponentially with horizon length. If each action has independent failure probability ϵ, then the probability of a successful episode of length T is at most $(1-\epsilon)^T \approx e^{-\epsilon T}$.

1.5 The Embodiment Spectrum

Embodied AI is not binary—it exists on a spectrum from passive perception to full interactive autonomy:

Regime	Agent's Role	Example
Passive perception	Fixed observer	ImageNet, COCO
Interactive perception	Moving observer with intent	Ego4D, EPIC-KITCHENS
Embodied navigation	Moves through environment	Habitat PointNav, ObjectNav
Embodied manipulation	Physically interacts with objects	AI2-THOR, iGibson
Full task completion	Plans, navigates, manipulates	ALFRED, BEHAVIOR

This spectrum captures the progression from *seeing* to *moving* to *interacting* to *accomplishing*. Each step introduces additional sources of complexity and partial observability.

1.6 Intellectual Foundations

1.6.1 Brooks' Subsumption Architecture

Rodney Brooks' 1991 paper *Intelligence without Representation* [**?**] argued that intelligent behavior can emerge from layers of sensorimotor couplings without explicit symbolic representations of the world. The **subsumption architecture** decomposes behavior into layers, each an augmented finite state machine that reads sensors and drives actuators:

$$\text{Layer 0:} \quad \text{avoid obstacles} \tag{1.14}$$
$$\text{Layer 1:} \quad \text{wander} \tag{1.15}$$
$$\text{Layer 2:} \quad \text{explore} \tag{1.16}$$
$$\vdots \tag{1.17}$$

Higher layers can *subsume* (override) lower layers by suppressing outputs or injecting new inputs. There is no central world model, no planner, no symbolic reasoning. Behavior emerges from the interaction of simple reflexive modules. This idea—that robust behavior does not require a detailed internal map—influenced reactive control and later, model-free RL for navigation.

1.6.2 Gibson's Affordances

James J. Gibson's theory of **affordances** holds that perception is directly of *action possibilities*. A chair "affords" sitting; a handle "affords" grasping; a doorway "affords" passage. The affordance depends on the agent's body: a staircase affords climbing to a human but not to a wheeled robot. Mathematically, an affordance can be modeled as a relation between environmental features and agent capabilities:

$$\mathcal{A}(o, e) = \{(a, c) \mid \text{capability } c \text{ enables action } a \text{ given observation } o\}\,. \tag{1.18}$$

Affordances bridge perception and action without explicit object classification. In modern Embodied AI, this idea resurfaces in **affordance prediction** models that directly map visual input to interaction heatmaps.

1.7 Modern Framing

In its contemporary form, Embodied AI integrates four pillars:

$$\text{Embodied AI} = \{\text{visual perception}\} \oplus \{\text{spatial reasoning}\} \oplus \{\text{physical interaction}\} \oplus \{\text{language grounding}\}\,. \tag{1.19}$$

Visual perception provides the raw sensory stream. **Spatial reasoning** maps this to 3D geometry and topology. **Physical interaction** covers locomotion, grasping, and manipulation. **Language grounding** enables agents to follow instructions, ask for help, and report findings—this is the subject of Vision-and-Language Navigation (Chapter 4) and language-guided manipulation (Chapter 7).

1.8 Key Benchmarks

The field coalesced around standardized benchmarks that provide reproducible evaluation:

- **Habitat** [**?**, **?**]: PointNav and ObjectNav tasks in photorealistic 3D scans (Matterport3D, HM3D). High-throughput simulation (10,000 FPS) enables large-scale RL training.
- **AI2-THOR** [**?**]: Interactive household environments with articulated objects (cabinets, microwaves, drawers). Supports both navigation and manipulation.
- **iGibson** [**?**, **?**]: Full physics simulation with PyBullet, supports grasping, pushing, and object state transitions (temperature, wetness). The BEHAVIOR benchmark defines 100 household tasks.
- **ALFRED** [**?**]: Language-guided task completion in AI2-THOR. Unifies instruction following, navigation, and object manipulation.

These benchmarks collectively span the embodiment spectrum and define the empirical landscape studied in this book.

1.9 The Perception-Action Loop as Information Processing

Let us formalize the loop in information-theoretic terms. The agent's goal can be viewed as minimizing the **epistemic uncertainty** about how to achieve the task. At time t, the agent possesses knowledge encoded in the belief b_t. An action a_t generates new observation o_{t+1}, yielding information gain:

$$\Delta I_t = \mathrm{KL}\left(b_{t+1}(\cdot \mid o_{t+1}, a_t) \parallel b_t(\cdot)\right). \tag{1.20}$$

An optimal exploration strategy selects actions that maximize cumulative information gain while respecting task constraints. This links embodied exploration to optimal experimental design and active learning.

1.10 Open Problems

1. **Grounding the POMDP**: How should we learn T, Ω, and R from interaction data without an accurate simulator? Model-based RL offers a path, but learned models compound errors over long horizons.
2. **Universal embodiment**: Human intelligence transfers across bodies (I can drive a car, ride a bicycle, and use chopsticks with the same brain). Can we train a single policy that adapts to different morphologies?
3. **Continual learning in the loop**: In the real world, the environment changes (furniture rearranged, seasons change). How should an agent update its policy online without catastrophic forgetting?
4. **The semantics of affordances**: How do we learn affordances that generalize to novel object instances, shapes, and physical properties never seen during training?
5. **When is a world model necessary?**: Brooks argued that representation is unnecessary. Modern systems depend heavily on internal maps. Resolving this tension—when should an agent maintain explicit spatial memory, and when should it act reactively?—remains open.

Summary

Embodied AI is the study of agents that close the perception-action loop. The POMDP is its formal foundation, and partial observability is its defining challenge. The next chapters build the tools—simulation, navigation, manipulation, language grounding, and learning paradigms—that let agents succeed in this challenging regime.

Chapter 2

Simulation Environments and Sim-to-Real Transfer

2.1 Motivation

Training embodied agents in the physical world is expensive, slow, and dangerous. A robot arm takes seconds per grasp attempt; a legged robot requires safety gantries and human supervision; a navigation agent cannot be reset to arbitrary initial poses without human intervention. To learn behaviors that require millions of environment interactions—the scale at which deep RL operates—simulation is not optional: it is *essential*.

Simulation offers four virtues that the real world cannot match: (1) **Cost**—1000 GPUs running Habitat can simulate billions of steps per day at negligible marginal cost. (2) **Safety**—a simulated robot can fall, collide, and break without consequence. (3) **Scalability**—episodes can be parallelized across thousands of environments, each with different layouts, object configurations, and lighting conditions. (4) **Ground truth**—perfect depth maps, exact object poses, and collision-free path verifications are available for free.

This chapter surveys the major simulation platforms used in Embodied AI research, characterizes the **sim-to-real gap** that arises when policies trained in simulation are deployed in the physical world, and presents the mathematical techniques that bridge this gap.

2.2 Simulation as a Computational Abstraction

Formally, a simulator is a function that implements the POMDP dynamics. Given a state $s_t \in S$ and action $a_t \in A$, the simulator returns:

$$s_{t+1} = f_{\text{phys}}(s_t, a_t) \tag{2.1}$$

$$o_{t+1} = f_{\text{render}}(s_{t+1}) \tag{2.2}$$

$$r_{t+1} = f_{\text{reward}}(s_t, a_t, s_{t+1}), \tag{2.3}$$

where f_{phys} integrates equations of motion (rigid body dynamics, joint constraints, contact forces), f_{render} generates synthetic sensor observations (RGB, depth, semantic segmentation), and f_{reward} computes the task-specific reward signal.

The key design tension in simulators is between **fidelity** (how closely the simulation matches physical reality) and **speed** (how many frames per second can be rendered). Different simulators occupy different points on this Pareto frontier.

2.3 Habitat: High-Performance Visual Simulation

Habitat [**?**, **?**] is the dominant platform for visual navigation research. Its core architectural insight is the decoupling of the *simulation engine* from the *dataset*.

2.3.1 Habitat-Sim

Habitat-Sim is a high-performance rendering engine built on the Magnum graphics library. It loads textured 3D meshes—from datasets such as Matterport3D, Gibson, HM3D, and Replica—

and renders them with hardware-accelerated OpenGL. Key design decisions:

- **GPU-accelerated rendering**: Multiple observations (from different agents or camera poses) are batched and rendered in parallel, achieving 10,000+ FPS on a single GPU with 128 parallel agents.
- **Lightweight physics**: Collision detection using the bounding volume hierarchy of the scene mesh. No full dynamics simulation—objects are static (the scene is a "scann") and the agent moves as a cylinder.
- **Modular sensors**: RGB, depth, semantic segmentation, and GPS+Compass sensors can be attached to the agent with configurable resolution, field of view, and noise parameters.

2.3.2 Habitat-Lab

Habitat-Lab provides the RL training interface. It wraps Habitat-Sim in a Gym-style API and implements task definitions, metrics, and baseline algorithms. A typical PointNav training loop processes observations in batches of size $B \times N$ where B is the number of parallel environments and N is the rollout length for each environment.

2.3.3 Scanned 3D Datasets

The scenes rendered by Habitat are obtained from real-world 3D scans:

$$\text{Matterport3D} : \text{90 buildings, 10,800 m}^2 \text{ of floor area} \tag{2.4}$$
$$\text{Gibson} : \text{572 full buildings, hand-curated for navigability} \tag{2.5}$$
$$\text{HM3D} : \text{1,000 buildings, 1,121 rooms, larger scale} \tag{2.6}$$
$$\text{Replica} : \text{18 highly detailed indoor scenes with dense annotations.} \tag{2.7}$$

These static scans provide photorealistic visual input but no object interactivity—furniture cannot be moved, drawers cannot be opened.

2.4 AI2-THOR: Interactive Object Manipulation

AI2-THOR [?] addresses Habitat's key limitation by introducing **interactive objects**. Built on the Unity game engine, AI2-THOR models 120+ room types (iTHOR) and 2,000+ interactive objects that respond to actions:

$$\begin{aligned} \mathcal{A}_{\text{THOR}} = \{ &\text{MoveAhead, RotateLeft, RotateRight,} \\ &\text{LookUp, LookDown, PickupObject,} \\ &\text{PutObject, OpenObject, CloseObject,} \\ &\text{ToggleObjectOn, ToggleObjectOff, SliceObject} \}. \end{aligned} \tag{2.8}$$

Each interactive object has predefined articulation states (an open/close threshold for a drawer, an on/off state for a light switch). Object affordances are deterministic given the object type—a Microwave can be opened and toggled; a Mug can be picked up. The action space is discrete and high-level, abstracting away continuous motor control.

AI2-THOR also provides a **metadata API** that returns perfect ground truth: the 3D bounding box of every object, its interaction state, and whether it is currently visible. This enables supervised training of perception modules (object detection, affordance prediction) alongside RL.

2.5 iGibson and Full Physics

iGibson [?, ?] extends interactivity to full rigid-body dynamics via the PyBullet physics engine. In iGibson, objects are physical entities with mass, friction, and collision geometry. The agent interacts through continuous joint commands rather than predefined high-level actions.

2.5.1 Object States

iGibson introduces continuous object state variables that go beyond discrete affordances:

$$\nu_{\text{pose}} \in SE(3) \quad \text{— 6-DoF pose} \tag{2.9}$$
$$\nu_{\text{temperature}} \in \mathbb{R}_{\geq 0} \quad \text{— thermal state} \tag{2.10}$$
$$\nu_{\text{wetness}} \in [0, 1] \quad \text{— moisture level} \tag{2.11}$$
$$\nu_{\text{cleanliness}} \in [0, 1] \quad \text{— dirt/clean indicator.} \tag{2.12}$$

These states evolve under action: placing a wet sponge on a counter increases the counter's wetness; cooking on a stove increases object temperature. This enables tasks like "clean the table" or "cook the egg" that require reasoning about non-geometric object properties.

2.5.2 BEHAVIOR Benchmark

The BEHAVIOR benchmark [?] defines 100 household tasks in iGibson, spanning cleaning, cooking, rearranging, and maintenance activities. Each task requires long-horizon planning, manipulation of multiple objects, and adaptation to object state changes. The benchmark exposes the limitations of current methods: the state of the art as of 2023 achieved under 20% success rate on most BEHAVIOR tasks.

2.6 NVIDIA Isaac Sim

Isaac Sim targets industrial robotics with high-fidelity physics and tight integration with the Robot Operating System (ROS). It provides:

- **PhysX 5.0**: GPU-accelerated rigid-body, soft-body, and fluid dynamics.
- **RTX rendering**: Ray-traced lighting, shadows, and reflections for photorealistic training images.
- **Domain Randomization (DR)** tools: Programmatic variation of textures, lighting, and dynamics parameters built into the platform.
- **ROS 2 bridge**: Synchronized data streaming between the simulator and production ROS middleware.

Isaac Sim is primarily used for **sim-to-real transfer** of manipulation policies, leveraging the DR infrastructure described below.

2.7 Other Simulation Platforms

2.7.1 MuJoCo

MuJoCo [?] is the gold standard for continuous control research. Its key innovations are a soft contact model that makes dynamics differentiable and highly optimized numerical integration.

MuJoCo excels at dexterous manipulation (in-hand object reorientation, multi-finger grasping) and legged locomotion. The Gymnasium and DM Control suites wrap MuJoCo in standardized RL interfaces.

SAPIEN [**?**] extends the MuJoCo paradigm to part-based object understanding. Objects are modeled as articulated assemblies of parts connected by joints (revolute, prismatic). This enables tasks that require reasoning about object articulation—opening a laptop, pulling a drawer—at the part level rather than the object level.

2.8 The Sim-to-Real Gap

A policy $\pi_\theta(a \mid o)$ trained entirely in simulation will encounter a distribution shift when deployed in the real world. Let $\mathcal{D}_{\text{sim}}$ be the distribution over (observation, dynamics) pairs encountered during training, and $\mathcal{D}_{\text{real}}$ the distribution encountered at deployment. The sim-to-real gap is:

$$\Delta(\pi_\theta) = \left| J_{\text{real}}(\pi_\theta) - J_{\text{sim}}(\pi_\theta) \right|. \tag{2.13}$$

The gap arises from two sources:

- **Rendering gap**: Synthetic images differ from real camera images in texture detail, lighting complexity, specular reflections, and sensor noise characteristics.
- **Dynamics gap**: The physics engine's approximation of contact forces, friction, deformation, and actuator dynamics diverges from physical reality.

Formally, if the agent is trained on a randomized domain distribution $p_\Xi(\xi)$ where ξ parameterizes domain variations (textures, lighting, mass, friction), then:

$$\Delta(\pi_\theta) \leq \sup_{\xi_{\text{real}} \in \Xi_{\text{real}}} \min_{\xi_{\text{sim}} \in \Xi_{\text{sim}}} d_{\text{domain}}(\xi_{\text{real}}, \xi_{\text{sim}}), \tag{2.14}$$

where d_{domain} measures the distance in the space of domain parameters. The bound tightens when the support of p_Ξ covers the target real domain.

2.9 Domain Randomization

Domain randomization (DR) trains policies that are invariant to domain variations by sampling the domain parameters from a wide distribution during training. The objective becomes:

$$J_{\text{DR}}(\theta) = \mathbb{E}_{\xi \sim \Xi,\, \tau \sim \pi_\theta(\cdot \mid \xi)} \left[\sum_{t=0}^{T-1} \gamma^t R_\xi(s_t, a_t) \right], \tag{2.15}$$

where Ξ is a distribution over rendering parameters (texture maps, light positions, camera intrinsics, background appearance) and dynamics parameters (object masses $\in [m_{\min}, m_{\max}]$, friction coefficients $\in [\mu_{\min}, \mu_{\max}]$, joint damping, motor gains). At each training episode, a new ξ is sampled, the simulation is configured accordingly, and the agent collects a rollout.

The intuition is simple: if the agent has seen every possible variation during training, the real world becomes just one more sample. More precisely, DR minimizes the worst-case loss over the support of Ξ:

$$\pi^*_{\text{DR}} = \arg\max_\pi \min_{\xi \in \text{supp}(\Xi)} J_\xi(\pi). \tag{2.16}$$

2.9.1 Canonical Example: OpenAI Rubik's Cube Hand

The OpenAI Dactyl system [**?**] trained a five-fingered hand to solve a Rubik's cube entirely in simulation using DR, then deployed zero-shot on a physical Shadow Dexterous Hand. The domain randomization distributed over: gravity vector magnitude and orientation, joint friction and damping, cube dimensions and mass, surface friction coefficients, actuator force limits, sensor noise (position and velocity measurements), and cube color. Training involved 13,000 CPU cores running rollouts in parallel. The physical transfer succeeded without any real-world fine-tuning.

2.10 Domain Adaptation

Domain adaptation aligns the feature distributions between simulation and reality. Given a feature extractor $\phi : \mathcal{O} \to \mathbb{R}^d$, the goal is to learn ϕ such that:

$$\mathrm{KL}\left(P_{\mathrm{sim}}(\phi(o)) \parallel P_{\mathrm{real}}(\phi(o))\right) \approx 0. \tag{2.17}$$

Popular methods include:

- **Gradient Reversal Layer (GRL)**: A discriminator is trained to classify images as sim or real; the feature extractor is trained to *fool* the discriminator via a gradient sign reversal.
- **CycleGAN**: Unpaired image-to-image translation maps sim images to the style of real images (and vice versa), using a cycle-consistency loss $\|G_{\mathrm{real}\to\mathrm{sim}}(G_{\mathrm{sim}\to\mathrm{real}}(x)) - x\|_1$.
- **Randomized-to-Real (RetinaGAN)**: Extends CycleGAN with feature-matching losses for sim-to-real transfer, tuned specifically for robotic grasping.

2.11 System Identification

System identification inverts the sim-to-real mapping: given observations from the physical system, infer the simulator parameters ξ that would produce matching behavior. Formally:

$$\hat{\xi} = \arg\min_{\xi\in\Xi} \sum_{t=1}^{T} \left\| s_t^{\mathrm{real}} - f_{\mathrm{sim}}(s_{t-1}^{\mathrm{real}}, a_{t-1}; \xi) \right\|^2. \tag{2.18}$$

Once $\hat{\xi}$ is estimated, the policy can be fine-tuned in the calibrated simulator. This approach works well when the space of unknown dynamics parameters is low-dimensional (e.g., unknown link masses and joint friction in a robot arm), but scales poorly when the mismatch is high-dimensional and unstructured (e.g., visual appearance).

2.12 Actuator Networks for Legged Locomotion

For legged robots (ANYmal, Cassie), the primary sim-to-real challenge is in actuator dynamics. The simulator's ideal torque model does not capture backlash, transmission stiffness, or motor saturation. **Actuator networks** [**?**] learn a mapping from commanded position/velocity/torque to actual joint response from real-world data:

$$\tau_{\mathrm{joint}}^{\mathrm{actual}} = g_\psi\left(\tau_{\mathrm{joint}}^{\mathrm{commanded}},\ \dot{q}_{\mathrm{joint}},\ q_{\mathrm{joint}}\right), \tag{2.19}$$

where g_ψ is a small neural network trained on logged actuator data. During simulation training, the actuator network is placed *between* the policy output and the physics engine, making the simulated dynamics match reality more closely. This technique enabled zero-shot sim-to-real transfer of dynamic quadrupedal gaits.

2.13 Connections

Simulation is the scaffolding upon which all other chapters rest. Navigation policies (Chapter 3) are trained in Habitat. Manipulation policies (Chapter 6) are trained in iGibson, Isaac Sim, or MuJoCo. Language instruction data (Chapter 4, Chapter 7) is collected from simulators where perfect ground-truth action sequences are available. The sim-to-real techniques in this chapter determine whether simulation-trained policies ever leave the lab.

2.14 Open Problems

1. **Photorealism at scale**: Current simulators fall short of photorealism for complex materials (fabric, liquid, deformable objects). How do we close this gap without rendering costs that negate simulation's speed advantage?

2. **Universal domain randomization**: Current DR requires hand-specifying which parameters to randomize and over what ranges. Can we learn the randomization distribution Ξ jointly with the policy, or infer it from real-world data?

3. **Deformable and fluid simulation**: Clothing, food, and granular materials remain challenging to simulate in real time. Differentiable simulators for soft bodies are an active frontier.

4. **Sim-to-real for visual-language tasks**: Most sim-to-real work targets control (locomotion, grasping). The visual semantics gap for language grounding—objects in simulation look different from real objects, and language models are sensitive to texture—remains largely unexplored.

5. **Automatic scene generation**: Current simulators use scanned environments. Procedural generation of realistic, diverse, and semantically plausible 3D scenes could eliminate the inventory problem and enable continual training on novel environments.

Part II

Navigation

Chapter 3

Visual Navigation: PointGoal and ObjectGoal

3.1 Motivation

Navigation is the most fundamental embodied capability. Before an agent can manipulate objects, follow instructions, or complete tasks, it must first arrive at the right location. Visual navigation—navigating using only egocentric visual observations—poses the core embodied challenge in its purest form: the agent sees a stream of RGB and depth images from a first-person camera, receives a target specification, and must produce a sequence of motor commands to reach the goal.

This chapter covers the two canonical navigation formulations: **PointGoal Navigation** (PointNav), where the target is a relative coordinate, and **ObjectGoal Navigation** (ObjectNav), where the target is a semantic object category. These tasks trace the arc of the field from classical geometric approaches to end-to-end reinforcement learning and back to modular architectures that combine learned perception with geometric planning.

3.2 Problem Formulation

An episode of visual navigation is defined as follows. The agent is spawned at a random pose $p_0 = (x_0, y_0, \theta_0)$ in a previously unseen 3D environment. At each timestep t, the agent receives an observation o_t and selects an action a_t. The episode terminates when the agent issues a `STOP` action, when the maximum step budget $T_{\max}$ is exhausted, or when the agent collides with an obstacle and cannot proceed.

3.2.1 Observation Space

The observation o_t consists of:

$$o_t^{\text{RGB}} \in \mathbb{R}^{H \times W \times 3} \quad \text{— egocentric RGB image} \tag{3.1}$$

$$o_t^{\text{depth}} \in \mathbb{R}^{H \times W} \quad \text{— depth at each pixel (optional)} \tag{3.2}$$

$$o_t^{\text{GPS}} = (\Delta x_t, \Delta y_t) \quad \text{— vector from agent to goal} \tag{3.3}$$

$$o_t^{\text{compass}} = (\cos \theta_t^g, \sin \theta_t^g) \quad \text{— heading relative to goal.} \tag{3.4}$$

The GPS+Compass is a **privileged sensor**: it provides the agent with its exact position relative to the goal, which is not directly available from vision alone. Whether GPS+Compass is realistic depends on the deployment scenario (indoor localization systems, SLAM), but its availability significantly changes the problem difficulty.

3.2.2 Action Space

Two action space parameterizations are common:

$$\mathcal{A}_{\text{discrete}} = \{\text{MoveForward}(0.25\text{m}),\ \text{TurnLeft}(30^\circ),\ \text{TurnRight}(30^\circ),\ \text{STOP}\}, \tag{3.5}$$

$$\mathcal{A}_{\text{continuous}} = \{(v, \omega) \mid v \in [-V_{\max}, V_{\max}],\ \omega \in [-\Omega_{\max}, \Omega_{\max}]\}. \tag{3.6}$$

The discrete action space with fixed-forward movement and fixed-angle turns simplifies learning but introduces discretization artifacts. Continuous action spaces are more realistic but require more sophisticated RL algorithms (often PPO with Gaussian policies).

3.3 PointGoal Navigation

3.3.1 Classical Approach: Map + Plan + Execute

The traditional robotics pipeline decomposes PointNav into three stages:

1. **Mapping**: Fuse depth observations into an occupancy grid map. At each step, a new depth image D_t is unprojected to a 3D point cloud, projected onto the ground plane, and integrated into a 2D occupancy map $\mathcal{M}_t \in \{0, 1, \mathit{unknown}\}^{H \times W}$. The map update uses Bayesian log-odds:

$$L_t(c) = L_{t-1}(c) + \log \frac{P(\text{occ} \mid D_t, c)}{1 - P(\text{occ} \mid D_t, c)} - L_0(c), \tag{3.7}$$

   where $L_t(c) = \log \frac{P(\text{occupied}|c)}{1-P(\text{occupied}|c)}$ is the log-odds that cell c is occupied.

2. **Planning**: Compute shortest path from current position to goal on the free space of $\mathcal{M}_t$ using A* or Dijkstra's algorithm.

3. **Execution**: Follow the planned path with a low-level controller while updating the map.

This pipeline is **brittle**: depth sensor noise propagates into map errors, dynamic obstacles invalidate planned paths, and actuation noise causes the agent to drift from the planned trajectory. Modern learning approaches replace the entire pipeline with a single neural policy.

3.3.2 End-to-End RL for PointNav

The agent's policy π_θ is a neural network that maps observations directly to actions:

$$a_t \sim \pi_\theta(\cdot \mid o_t^{\text{RGB}}, o_t^{\text{depth}}, o_t^{\text{GPS}}, o_t^{\text{compass}}). \tag{3.8}$$

The policy is trained with Proximal Policy Optimization (PPO) [**?**] to maximize the expected return, where the reward function is:

$$R_t = \underbrace{\Delta d_{\text{geo}}}_{\text{progress}} + \underbrace{r_{\text{success}} \cdot \mathbb{1}[d_{\text{geo}} \leq \varepsilon]}_{\text{success bonus}} - \underbrace{r_{\text{slack}}}_{\text{time penalty}}, \tag{3.9}$$

with $\Delta d_{\text{geo}} = d_{\text{geo}}(p_{t-1}, g) - d_{\text{geo}}(p_t, g)$, the reduction in geodesic distance to the goal, and r_{slack} a small per-step penalty to encourage efficient paths.

3.3.3 DD-PPO and the Scaling Hypothesis

DD-PPO [**?**] demonstrated that PointNav, when trained at sufficient scale, is effectively *solved*—at least with GPS+Compass. The key innovations:

- **Architecture**: A ResNet-50 visual encoder (pretrained on ImageNet) processes o_t^{RGB} and o_t^{depth}. The resulting visual embedding is concatenated with GPS+Compass and fed to a 2-layer GRU with 512 hidden units. The GRU output passes through actor and critic heads.

- **Decentralized training**: DD-PPO distributes PPO across K GPUs with a synchronized update protocol. Each GPU runs rollouts in M parallel environments, collects $M \times N$ steps, computes gradients, and all GPUs average their gradients before the optimizer step.
- **Scale**: Training used 64 GPUs, 2.5 billion environment steps, across hundreds of Matterport3D scenes.

The central empirical finding was that performance improved *monotonically* with training steps—no saturation, no plateau. The success rate on the standard PointNav benchmark rose from 20% (2018 baselines) to >99% (2020 state-of-the-art). This established a scaling law for embodied navigation: *more experience always helps.*

3.3.4 Why GPS+Compass Matters

With perfect GPS+Compass, PointNav reduces to a reactive control problem: move toward the indicated direction while avoiding obstacles. The visual encoder needs only to detect obstacles (walls, furniture) and free space. The long-range planning challenge—reasoning about the connectivity of rooms and hallways—is eliminated because the GPS vector provides a shortcut through the environment's topology.

This observation motivated **ImageNav**: the target is specified as an image I_{goal} rather than a coordinate. The agent must visually recognize when it has reached the goal location. ImageNav is harder because it requires the agent to (a) encode the appearance of a place, (b) maintain spatial memory of where it has been, and (c) reason about visual similarity between current observation and goal image. No GPS+Compass.

3.4 ObjectGoal Navigation (ObjectNav)

3.4.1 Problem Definition

In ObjectNav, the target is a semantic object category $g \in \mathcal{G}$ (e.g., "chair", "toilet", "refrigerator"), and there may be multiple instances of g in the environment. The agent succeeds if it issues `STOP` within ϵ meters of *any* instance of g and the target object is visible in the agent's egocentric view.

Compared to PointNav, ObjectNav requires: (1) **Semantic understanding**: recognizing object categories from visual input. (2) **Exploration**: systematically searching a partially observed environment. (3) **Semantic priors**: knowing that toilets are in bathrooms and refrigerators are in kitchens—a form of common sense.

3.4.2 Semantic Mapping + Frontier Exploration

The strongest approach to ObjectNav is modular:

1. **Semantic mapping**: At each step, the RGB image is processed by a semantic segmentation network to produce per-pixel class labels. These labels are projected into 3D via depth and aggregated into a top-down semantic map $\mathcal{M}_t^{\text{sem}}$. Each cell c holds a distribution over object categories:

$$\mathcal{M}_t^{\text{sem}}(c, g) = P(\text{category } g \text{ at cell } c \mid \tau_t). \tag{3.10}$$

2. **Frontier detection**: A **frontier** is a cell that is free space and is adjacent to at least one unknown cell. Frontiers mark the boundary of explored space and are natural subgoals for exploration. Let $\mathcal{F}_t$ be the set of frontiers at time t.

3. **Semantic frontier selection**: Select the frontier that maximizes the probability of finding the target object. The score for frontier f is:

$$S(f, g) = \max_{c \in \text{near}(f)} \mathcal{M}_t^{\text{sem}}(c, g) \cdot \exp\left(-\lambda \cdot d_{\text{geo}}(p_t, f)\right), \tag{3.11}$$

where $\text{near}(f)$ is the set of cells within a radius of frontier f, and the exponential term penalizes distant frontiers.

3.4.3 SemExp: Learned Semantic Priors

SemExp [?] augments the modular pipeline with learned semantic priors. Rather than hard-coding that chairs are near tables, SemExp learns a spatial prior $P(g_1 \text{ near } g_2)$ from training environments:

$$S_{\text{learned}}(f, g) = \sum_{g' \in \mathcal{G}} \alpha_{g' \to g} \cdot \mathcal{M}_t^{\text{sem}}(c_f, g'), \tag{3.12}$$

where $\alpha_{g' \to g}$ is a learned weight encoding the co-occurrence affinity between categories g' and g. For example, $\alpha_{\text{bed} \to \text{nightstand}}$ is high; $\alpha_{\text{toilet} \to \text{oven}}$ is low. The SemExp policy is trained with RL to select frontiers, with the reward being the eventual success in finding the target.

Formally, SemExp maintains a **Bayesian semantic occupancy** model:

$$P(\text{object } g \text{ at cell } c) = P(\text{semantic label} = g \mid \text{cell } c) \cdot P(\text{spatial prior} \mid g, c). \tag{3.13}$$

The first factor comes from the semantic segmentation network; the second is learned from prior exploration experience. This separation of perception (what is seen) from reasoning (where objects are likely to be) is the key to SemExp's generalization.

3.4.4 ZER: Zero-Shot ObjectNav via CLIP

ZER [?] eliminates the need for ObjectNav training data entirely. Instead, it uses CLIP [?] to bridge vision and language:

$$S_{\text{CLIP}}(o_t, g) = \langle f_{\text{image}}(o_t^{\text{RGB}}),\ f_{\text{text}}(\text{"photo of a } g\text{"}) \rangle. \tag{3.14}$$

The exploration policy is trained with RL using an intrinsic reward that encourages seeing regions with high CLIP similarity to the target category:

$$R_t^{\text{intrinsic}} = S_{\text{CLIP}}(o_t, g) - S_{\text{CLIP}}(o_{t-1}, g). \tag{3.15}$$

At test time, previously unseen object categories can be specified through natural language without any additional training. ZER achieves competitive performance on ObjectNav without using any of the training episodes.

3.4.5 PONI: Potential Functions for ObjectNav

PONI [?] takes a different approach: learn a **potential function** $U(c, g)$ that estimates the geodesic distance from any map cell c to the nearest instance of category g:

$$U(c, g) \approx \mathbb{E}_{\text{env}}\left[d_{\text{geo}}(c, \text{nearest-instance-of-}g)\right]. \tag{3.16}$$

$U(c, g)$ is trained on prior exploration data—episodes where the agent explored environments and recorded object locations. At test time, given a target g, the agent plans to the cell that minimizes the potential:

$$c^* = \arg\min_{c \in \mathcal{M}_t^{\text{free}}} U(c, g). \tag{3.17}$$

The key insight is that U is a spatial prior that captures the distribution of objects in the environment, learned offline from exploration data. It can be evaluated on any map, including the partially observed map at test time.

3.5 Evaluation Metrics

3.5.1 Success Rate

The agent succeeds if (a) it calls `STOP` within ε geodesic distance of a target instance, and (b) the target is visible in the egocentric view at the stop location (for ObjectNav). Let $\mathbb{1}_i$ be the success indicator for episode i:

$$\text{SR} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}_i. \tag{3.18}$$

3.5.2 SPL (Success Weighted by Path Length)

SPL penalizes successful episodes that took longer paths than the optimal shortest path [**?**]:

$$\text{SPL} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}_i \cdot \frac{\ell_i^{\text{opt}}}{\max(\ell_i, \ell_i^{\text{opt}})}, \tag{3.19}$$

where ℓ_i is the agent's actual path length and ℓ_i^{opt} is the shortest-path length from start to goal (computed via the ground-truth top-down map). SPL is the primary metric for navigation benchmarks.

3.5.3 SoftSPL and Progress Metrics

For PointNav, the progress is also measured by the final geodesic distance to the goal: $d_{\text{geo}}^{\text{final}} = d_{\text{geo}}(p_T, g)$. A "soft" success metric counts partial progress.

3.6 Modular vs. End-to-End

A major finding of the navigation literature is that **modular approaches outperform end-to-end policies for ObjectNav**. The reasons:

- Spatial reasoning is inherently geometric: shortest paths, occlusion boundaries, and free-space connectivity are well-modeled by geometric algorithms (A*, frontier detection).
- Learned modules should handle what neural networks are good at: visual recognition (detecting objects, predicting semantics) and learning priors (where are objects likely to be?).
- End-to-end policies must learn geometry, semantics, and planning jointly from reward, which requires far more data and yields worse generalization to new environments.

The SemExp result—97% of the gap to an upper-bound oracle planner compared to 50% for end-to-end RL—provides the empirical evidence. Modularity is the dominant paradigm for ObjectNav; end-to-end RL remains competitive for PointNav *with GPS+Compass*, where the geometric problem is trivialized.

3.7 Connections

Navigation is the substrate upon which higher-level embodied capabilities are built. Vision-and-Language Navigation (Chapter 4) adds natural language instructions to the navigation problem. SLAM (Chapter 5) provides the mapping and localization machinery that modular approaches rely on. Long-horizon task completion (Chapter 8) combines navigation with manipulation.

3.8 Open Problems

1. **Navigation without depth**: Can an agent navigate using only RGB? Monocular depth estimation has improved, but depth prediction errors compound in map construction. Learning to navigate from RGB alone—implicitly learning geometry—remains an open challenge.

2. **Multi-floor and outdoor navigation**: Current benchmarks are single-floor indoor environments. Multi-floor navigation requires staircase detection and elevator usage; outdoor navigation introduces dynamic obstacles, larger scales, and GPS noise.

3. **Efficient exploration**: Frontier-based exploration is provably complete but inefficient. Can learning reduce the number of steps needed to find objects of interest, perhaps by predicting the information content of unexplored regions?

4. **Embodied memory architectures**: The agent must remember what it has seen, where it has been, and what it is looking for. Current architectures (GRU, map) are limited in memory capacity. Transformers with long context windows offer a promising direction.

5. **Zero-shot generalization to new object categories**: ZER with CLIP is a first step, but CLIP's visual representations are trained on internet images, not embodied viewpoints. Training vision-language models on egocentric data is a critical need.

Chapter 4

Vision-and-Language Navigation (VLN)

4.1 Motivation

The ability to follow natural language directions is a hallmark of human intelligence. When a person says "Go down the hallway, turn left at the sofa, and stop in front of the painting", the listener must parse linguistic references, ground them in visual observations, execute a sequence of motor actions, and verify completion by matching the final view against the instruction's expected endpoint. This is **Vision-and-Language Navigation** (VLN): the problem of navigating through a 3D environment by following natural language instructions.

VLN sits at the intersection of computer vision, natural language processing, and embodied control. It requires the agent to: (1) parse compositional linguistic references to objects, spatial relationships, and actions; (2) ground these references in egocentric visual observations; (3) make sequential decisions under partial observability; (4) recognize when the instruction has been completed. This chapter traces the evolution of VLN from sequence-to-sequence baselines to modern transformer architectures, emphasizing the mathematical structures that enable cross-modal grounding.

4.2 Problem Formulation

A VLN episode is defined by a 3D environment E, a starting pose p_0, and a natural language instruction $\mathbf{I} = (w_1, w_2, \ldots, w_L)$ where each w_i is a word token. At time t, the agent receives a panoramic observation O_t consisting of K discrete camera views:

$$O_t = \left\{ v_t^{(1)}, v_t^{(2)}, \ldots, v_t^{(K)} \right\}, \tag{4.1}$$

where each $v_t^{(k)} \in \mathbb{R}^{H \times W \times 3}$ is an RGB image captured from one of K predefined camera angles (typically 36 views: 12 yaw angles $\times$ 3 elevations at $\{-30°, 0°, 30°\}$).

The agent selects an action a_t from a discrete action space:

$$\mathcal{A} = \{\text{MoveForward}, \text{TurnLeft}, \text{TurnRight}, \text{LookUp}, \text{LookDown}, \text{STOP}\}. \tag{4.2}$$

The episode ends when `STOP` is called or the maximum step budget $T_{\max}$ is reached. The agent succeeds if the final position lies within 3 meters of the target location.

4.3 Datasets

4.3.1 R2R: Room-to-Room

The **R2R** dataset [**?**] established VLN as a benchmark. Key statistics:

$$\text{Environments} : 90 \text{ Matterport3D buildings} \tag{4.3}$$

$$\text{Instructions} : 21{,}567 \text{ English instructions (3 per path)} \tag{4.4}$$

$$\text{Vocabulary size} : \sim 3{,}100 \text{ unique words} \tag{4.5}$$

$$\text{Path length} : \text{mean } 10\text{m}, \ 5\text{–}7 \text{ actions per episode.} \tag{4.6}$$

Each path was presented to crowdworkers who wrote instructions describing how to follow it. Three annotators produced instructions for each path, yielding linguistic diversity for the same trajectories.

4.3.2 R4R and RxR

R4R [**?**] extends R2R with longer trajectories and higher linguistic diversity by concatenating shorter R2R paths. **RxR** [**?**] adds multilingual instructions (English, Hindi, Telugu) with more detailed, compositional descriptions. RxR instructions are longer (mean 32 vs. 29 words in R2R) and contain more spatial reasoning language.

4.4 Evaluation Metrics

4.4.1 Success Rate and Path Alignment

Standard VLN metrics include:

$$\text{SR} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[d_{\text{final}} \leq 3\text{m}] \tag{4.7}$$

$$\text{SPL} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}_i \cdot \frac{\ell_i^{\text{opt}}}{\max(\ell_i, \ell_i^{\text{opt}})} \tag{4.8}$$

$$\text{nDTW} = \frac{1}{N}\sum_{i=1}^{N} \frac{\text{DTW}(\tau_i^{\text{pred}}, \tau_i^{\text{ref}}) - \text{DTW}_{\min}}{\text{DTW}_{\max} - \text{DTW}_{\min}}. \tag{4.9}$$

4.4.2 Normalized Dynamic Time Warping (nDTW)

nDTW measures the soft alignment between the predicted trajectory $\tau_{\text{pred}} = (p_0, p_1, \ldots, p_T)$ and the reference trajectory τ_{ref}. Dynamic Time Warping finds the optimal monotonic alignment between the two sequences:

$$\text{DTW}(\tau_{\text{pred}}, \tau_{\text{ref}}) = \min_{\text{alignments}} \sum_{(i,j)\in\text{align}} d_{\text{geo}}(p_i^{\text{pred}}, p_j^{\text{ref}}) \cdot w(i), \tag{4.10}$$

where $w(i)$ is a timestep weight that penalizes early deviations more heavily (because early errors compound). nDTW normalizes against oracle and random trajectories.

4.5 Sequence-to-Sequence Baseline

The earliest VLN models framed the problem as sequence-to-sequence translation: encode the instruction $\mathbf{I}$ into a fixed-length vector, then decode actions conditioned on visual observations.

4.5.1 Instruction Encoding

An LSTM encoder processes the instruction token by token:

$$\mathbf{h}_i^{\text{enc}} = \text{LSTM}_{\text{enc}}(\mathbf{e}(w_i), \mathbf{h}_{i-1}^{\text{enc}}), \tag{4.11}$$

$$\mathbf{z} = \frac{1}{L}\sum_{i=1}^{L} \mathbf{h}_i^{\text{enc}}, \tag{4.12}$$

where $\mathbf{e}(w_i) \in \mathbb{R}^{d_e}$ is the embedding of token w_i, and $\mathbf{z} \in \mathbb{R}^{d_h}$ is the mean-pooled instruction representation. A bidirectional variant concatenates forward and backward encodings.

4.5.2 Visual Encoding

At each step t, the K panoramic views are each encoded by a ResNet-152 pretrained on ImageNet and Places365:

$$\mathbf{f}_t^{(k)} = \text{ResNet}(v_t^{(k)}) \in \mathbb{R}^{2048}, \quad k = 1, \ldots, K. \tag{4.13}$$

The visual features from all K views are concatenated with orientation features (the sine and cosine of each view's heading and elevation relative to the agent's current heading) to form $\mathbf{v}_t \in \mathbb{R}^{K \times (2048+4)}$.

4.5.3 Action Decoding

An LSTM decoder predicts the next action given the instruction encoding $\mathbf{z}$, the visual features $\mathbf{v}_t$, and the previous action:

$$\mathbf{h}_t^{\text{dec}} = \text{LSTM}_{\text{dec}}([\mathbf{z}; \mathbf{v}_t; \mathbf{a}_{t-1}], \mathbf{h}_{t-1}^{\text{dec}}), \tag{4.14}$$

$$P(a_t \mid \mathbf{I}, O_t, a_{t-1}) = \text{softmax}(\mathbf{W}_a \mathbf{h}_t^{\text{dec}} + \mathbf{b}_a). \tag{4.15}$$

The model is trained with teacher forcing: at training time, the ground-truth action is fed as the previous action input; at test time, the model's own prediction is used.

4.5.4 Limitations

The seq2seq baseline struggles with (1) forgetting early instruction parts during long trajectories, (2) overfitting to visual appearance of training environments, and (3) producing identical actions for different instructions in visually similar contexts—three symptoms of insufficient cross-modal grounding.

4.6 Speaker-Follower: Data Augmentation and Pragmatic Reasoning

Speaker-Follower [?] introduced two innovations that became standard in VLN: (1) a speaker model for data augmentation, and (2) pragmatic inference at test time.

4.6.1 Speaker Model

The **Speaker** is an instruction generator trained on human-annotated R2R paths. Given a trajectory $\tau = (v_0, v_1, \ldots, v_T)$, it generates an instruction $\mathbf{I}$:

$$P(\mathbf{I} \mid \tau) = \prod_{i=1}^{L} P(w_i \mid w_{<i}, \tau). \tag{4.16}$$

The speaker encodes the visual sequence with an attention mechanism over the $T + 1$ panoramic observations, then generates word tokens autoregressively.

At training time, the speaker is used to **augment** the R2R dataset: for paths that have only 3 human instructions, the speaker generates additional synthetic instructions, effectively increasing the diversity of language that the follower sees for the same action sequences.

4.6.2 Follower Model

The **Follower** uses an attention-based architecture to align instruction tokens with visual observations:

$$\alpha_{i,t}^{(k)} = \frac{\exp(\langle \mathbf{h}_i^{\text{enc}}, \mathbf{f}_t^{(k)} \rangle / \sqrt{d})}{\sum_{j,k'} \exp(\langle \mathbf{h}_j^{\text{enc}}, \mathbf{f}_t^{(k')} \rangle / \sqrt{d})}, \tag{4.17}$$

$$\mathbf{c}_t = \sum_i \alpha_{i,t} \mathbf{h}_i^{\text{enc}}, \tag{4.18}$$

$$P(a_t \mid \mathbf{I}, O_t) = \text{softmax}(\mathbf{W}[\mathbf{c}_t; \mathbf{v}_t; \mathbf{h}_{t-1}^{\text{dec}}]). \tag{4.19}$$

The attention weights $\alpha_{i,t}^{(k)}$ indicate which words are most relevant to which visual views at the current step—a soft cross-modal grounding mechanism. When the instruction says "turn left at the sofa", the model learns to attend to the word "sofa" when the visual field contains sofa-like features.

4.6.3 Pragmatic Inference

Pragmatic reasoning recognizes that *the speaker would only say certain words if certain visual conditions held*. At test time, the agent considers M candidate action sequences (beam search over action space) and computes:

$$a_t^* = \arg\max_{a_t \in \mathcal{A}} \; P_{\text{follower}}(a_t \mid \mathbf{I}, O_t) \cdot P_{\text{speaker}}(\mathbf{I} \mid \tau_{\text{prefix}} \circ a_t)^{\beta}, \tag{4.20}$$

where τ_{prefix} is the trajectory so far, $\circ$ denotes trajectory extension, and β controls the strength of pragmatic reasoning. This formulation, inspired by Rational Speech Acts theory, reweights actions to favor those that make the instruction *informative*—i.e., those for which the speaker model would be likely to have produced the given instruction.

4.6.4 Environmental Dropout

To prevent overfitting to visual textures of training environments, Speaker-Follower applies **environmental dropout**: random views in the panoramic observation are zeroed out during training. The model learns to rely more on language when vision is unreliable, improving generalization to unseen environments.

4.7 EnvDrop: Robustness via Visual Perturbation

EnvDrop [?] systematizes environmental dropout with a back-translation objective. The model is trained on two tasks simultaneously:

$$\mathcal{L}_{\text{navigation}} = -\log P(a_t^* \mid \mathbf{I}, O_t), \tag{4.21}$$

$$\mathcal{L}_{\text{back-translation}} = -\log P(\mathbf{I} \mid \hat{\tau}), \tag{4.22}$$

where $\hat{\tau}$ is a trajectory generated by the navigation model from a synthetic instruction. The back-translation loss ensures that the model's internal representations are consistent: if the model navigates according to a synthetic instruction, a speaker should be able to recover that instruction from the trajectory.

During training, visual features are randomly shifted, scaled, and masked with probability p_{drop} (typically 0.5). This forces the policy to develop a robust language-conditioned prior that does not collapse when visual input is degraded.

4.8 PREVALENT: Pre-training for VLN

PREVALENT [**?**] adapts the pre-training paradigm from NLP to VLN. The architecture is a multimodal transformer pretrained on a large corpus of VLN-style data, then fine-tuned on R2R.

4.8.1 Pre-training Tasks

PREVALENT defines two pre-training objectives:

1. **Masked Language Modeling (MLM)**: Random instruction tokens are masked, and the model must predict them from the remaining tokens and the paired visual trajectory:

$$\mathcal{L}_{\text{MLM}} = -\sum_{i \in \text{masked}} \log P(w_i \mid \mathbf{I}_{\setminus \text{masked}}, \tau). \tag{4.23}$$

2. **Action Prediction (AP)**: Given the instruction and a partial trajectory, predict the next action:

$$\mathcal{L}_{\text{AP}} = -\sum_{t} \log P(a_t \mid \mathbf{I}, \tau_{<t}). \tag{4.24}$$

The pre-training data is constructed by pairing R2R instructions with the corresponding visual trajectories (without requiring any additional annotation). After pre-training, the model is fine-tuned on R2R with the standard teacher-forcing loss.

PREVALENT demonstrated that pre-training on **in-domain** data (visual trajectories + language from R2R) transfers better to VLN than pre-training on generic image-caption data. The visual representations learned during pre-training are tailored to the viewpoint and panoramic nature of VLN observations.

4.9 HAMT: History-Aware Multimodal Transformer

HAMT [**?**] reformulates VLN as a sequence modeling problem over **full interaction history**. Rather than compressing the past into an LSTM hidden state, HAMT attends over the entire sequence of past observations and actions.

4.9.1 Architecture

The input to the transformer at step t is:

$$\mathbf{X}_t = [\mathbf{h}_{\text{CLS}}, \mathbf{h}_{\mathbf{I}_1}, \ldots, \mathbf{h}_{\mathbf{I}_L}, \mathbf{h}_{o_0}, \mathbf{h}_{a_0}, \mathbf{h}_{o_1}, \mathbf{h}_{a_1}, \ldots, \mathbf{h}_{o_t}], \tag{4.25}$$

where $\mathbf{h}_{\text{CLS}}$ is a learnable classification token, $\mathbf{h}_{\mathbf{I}_i}$ are instruction token embeddings, $\mathbf{h}_{o_t}$ encodes the panoramic observation at time t, and $\mathbf{h}_{a_{t-1}}$ encodes the previous action. The transformer processes this sequence with self-attention, enabling every element to attend to every other element:

$$\mathbf{H}^{(l+1)} = \text{TransformerBlock}^{(l)}(\mathbf{H}^{(l)}). \tag{4.26}$$

The action prediction head reads the CLS token output from the final layer:

$$P(a_t \mid \mathbf{I}, \tau_{<t}) = \text{softmax}(\mathbf{W}_{\text{out}} \mathbf{h}_{\text{CLS}}^{(L)}). \tag{4.27}$$

4.9.2 Long-Term Memory via History Attention

The key advantage of HAMT is **cross-episode attention**. By attending over all past observations, the model can:

- Avoid looping: if the current view matches a view seen 20 steps ago, the attention mechanism can detect the similarity and suppress the action that led back to it.
- Track instruction progress: the model can attend to early instruction tokens at late timesteps, keeping subgoal completion status accessible throughout the episode.
- Leverage visual landmarks: when the instruction says "go past the dining table", the model can attend to the observation that contained the dining table.

HAMT set the state of the art on R2R and RxR, demonstrating that explicit memory mechanisms outperform recurrent hidden states for long-horizon instruction following.

4.10 Cross-Modal Alignment as a Mathematical Challenge

The fundamental difficulty of VLN can be stated precisely: the instruction is a discrete symbolic sequence over a vocabulary $\mathcal{V}$, while the world is a continuous 3D manifold. The agent must learn a mapping:

$$\phi : \mathcal{V}^L \times (\mathbb{R}^3 \times SO(3)) \to \Delta(\mathcal{A}), \tag{4.28}$$

where $\Delta(\mathcal{A})$ is the probability simplex over actions. This mapping is non-convex, high-dimensional, and must generalize across environments, object instances, and linguistic expressions.

The alignment problem decomposes into three sub-problems:

1. **Referential grounding**: Map noun phrases to visual regions. "The wooden chair with armrests" must be matched to the specific chair instance in view. This is the classic visual grounding problem, but with the additional constraint that the agent may need to move to bring the referent into view.
2. **Spatial grounding**: Map spatial prepositions to geometric relationships. "Turn left at the sofa" requires understanding that "left" is relative to the agent's heading; "stop in front of the painting" requires understanding that "in front of" means facing the painting at a certain distance.
3. **Temporal grounding**: Map temporal adverbs and sequencing words to the trajectory structure. "First go straight, then turn right" requires segmenting the trajectory into phases and binding actions to phases.

Each sub-problem involves a different form of cross-modal alignment. Current VLN models handle referential grounding reasonably well (when objects are visible) but struggle with spatial and temporal grounding, especially when the linguistic description is indirect or imprecise.

4.11 Connections

VLN connects to the navigation technologies of Chapter 3 (the agent must navigate), to simulation (Chapter 2, VLN is trained and evaluated in Matterport3D via Habitat or Matterport3D Simulator), and to language-guided manipulation (Chapter 7, where instructions specify object interactions). The cross-modal alignment techniques developed for VLN—attention over instruction tokens, multimodal transformers, pragmatic inference—transfer directly to embodied question answering and interactive dialog.

4.12 Open Problems

1. **VLN in dynamic environments**: Current datasets assume static scenes. Real-world navigation must handle moving people, opened/closed doors, and relocated objects. The agent must distinguish instruction- relevant changes from irrelevant noise.

2. **Dialog-based navigation**: When instructions are ambiguous ("turn at the next door"—which one?), the agent should ask for clarification. This requires generative language capabilities and meta-reasoning about its own uncertainty.

3. **Continuous action VLN**: The discrete panoramic action space (36 views) is an artifact of dataset construction. Real robots operate in continuous $SE(2)$ or $SE(3)$ action spaces. Extending VLN to continuous control is necessary for real-world deployment.

4. **Multilingual and cross-cultural VLN**: RxR provides data in three languages, but the environments are still American apartments. VLN in diverse cultural contexts (markets, temples, outdoor spaces) requires new datasets and models.

5. **Compositional generalization**: Can a VLN agent that has seen "turn left at the sofa" and "go past the dining table" compose them into "turn left at the dining table and go past the sofa"? Current models show limited compositional generalization because they memorize phrase-action correlations rather than learning the underlying grammar of spatial instructions.

6. **Instruction generation as a training signal**: The speaker model in Speaker-Follower generates instructions for data augmentation. Extending this to *generate instructions for exploration*—the agent imagines what instruction would describe a candidate action, then compares it to the given instruction—could provide a richer training signal than reinforcement learning alone.

Chapter 5

SLAM and Spatial Understanding

5.1 Motivation

An embodied agent that cannot answer the question *Where am I?* is fundamentally incapable of purposeful action. Simultaneous Localization and Mapping (SLAM) addresses the intertwined problems of estimating an agent's pose while concurrently constructing a representation of its environment. The problem is elegantly circular: to build a map you need to know where you are; to know where you are, you need a map. This chicken-and-egg structure has made SLAM one of the richest theoretical frameworks in robotics, drawing on probabilistic inference, manifold optimization, and most recently, deep learning.

Why is SLAM hard? The state space grows without bound as the agent explores — each new landmark adds dimensions to the estimation problem. Observational noise accumulates and compounds: a one-degree orientation error at the start of a trajectory becomes a meter-scale position error at the end, absent correction. And the data association problem — determining which landmark is being observed when multiple candidates exist — introduces combinatorial ambiguity that can catastrophically corrupt the estimate.

For embodied AI, spatial understanding extends beyond pure geometry. Consider a robot asked to *"bring me the mug from the kitchen table."* The robot must know not only the metric layout of the house (where is the kitchen?), but also the semantic category of each room (which room is the kitchen?), the likely locations of object classes (mugs are typically on tables or in cabinets), and the navigable connectivity between spaces (can I reach the table?). This chapter develops the probabilistic, optimization-based, and learned foundations of spatial understanding for embodied agents.

5.2 Core Concepts and Key Papers

5.2.1 Probabilistic Formulation of SLAM

At its foundation, SLAM is a problem of Bayesian inference. Let the robot's state at time t be $\mathbf{x}_t \in \mathbb{R}^d$ (encompassing position, orientation, and possibly velocity), the map be $\mathbf{m}$ (which may be a set of landmark positions, an occupancy grid, or a neural field), and the sequence of observations and controls be $\mathbf{z}_{1:t}$ and $\mathbf{u}_{1:t}$ respectively. The objective is to compute the joint posterior

$$p(\mathbf{x}_{1:t}, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) \;=\; \frac{p(\mathbf{z}_{1:t} \mid \mathbf{x}_{1:t}, \mathbf{m})\, p(\mathbf{x}_{1:t} \mid \mathbf{u}_{1:t})\, p(\mathbf{m})}{p(\mathbf{z}_{1:t} \mid \mathbf{u}_{1:t})}. \tag{5.1}$$

The motion model $p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)$ describes the stochastic transition of the robot's state under control $\mathbf{u}_t$, typically modeled as Gaussian noise around a deterministic dynamics function:

$$\mathbf{x}_t = g(\mathbf{x}_{t-1}, \mathbf{u}_t) + \boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_t). \tag{5.2}$$

The observation model $p(\mathbf{z}_t \mid \mathbf{x}_t, \mathbf{m})$ describes the likelihood of perceiving $\mathbf{z}_t$ given the current state and map. For a landmark-based map, observing landmark m_k from pose $\mathbf{x}_t$ produces:

$$\mathbf{z}_t = h(\mathbf{x}_t, \mathbf{m}_k) + \boldsymbol{\delta}_t, \qquad \boldsymbol{\delta}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{R}_t), \tag{5.3}$$

where $h(\cdot)$ predicts range and bearing from the pose to the landmark. The denominator, the marginal likelihood $p(\mathbf{z}_{1:t} \mid \mathbf{u}_{1:t})$, is generally intractable to compute, forcing resort to approximate inference.

The core difficulty of SLAM arises from the *conditional dependence* structure induced by the map $\mathbf{m}$. Even if motion and observation models are conditionally independent given the state, the map couples all poses: observing a landmark at time t provides information about every preceding pose that has observed that same landmark. This is the *SLAM property* — the correlation between pose estimates grows monotonically and never diminishes unless loop closures introduce new constraints that propagate corrections globally. The joint information matrix $\boldsymbol{\Omega} = \boldsymbol{\Sigma}^{-1}$ for the full trajectory-and-map state is almost dense due to this coupling, a fact that algorithms must exploit or circumvent.

5.2.2 EKF-SLAM: The Classical Filtering Approach

The Extended Kalman Filter SLAM formulation (Smith, Self, and Cheeseman, 1990; Leonard and Durrant-Whyte, 1991) represents the joint state as a vector concatenating the robot pose and all landmark positions:

$$\mathbf{s} = \begin{bmatrix} \mathbf{x} \\ \mathbf{m}_1 \\ \vdots \\ \mathbf{m}_L \end{bmatrix} \in \mathbb{R}^{d+3L}, \qquad \mathbf{s} \sim \mathcal{N}(\hat{\mathbf{s}}, \mathbf{P}). \tag{5.4}$$

The covariance matrix $\mathbf{P}$ captures not only uncertainty in the robot pose and landmarks individually, but crucially the cross-correlation blocks $\mathbf{P}_{\mathbf{x}\mathbf{m}_k}$ and $\mathbf{P}_{\mathbf{m}_j\mathbf{m}_k}$ (for $j \neq k$). It is this cross-correlation that makes EKF-SLAM work: updating the robot pose from a landmark observation automatically updates *every* landmark through the Kalman gain mechanism.

The EKF update proceeds in two standard steps. Given a control $\mathbf{u}_t$, the prediction step linearizes the motion model $g(\cdot)$:

$$\bar{\mathbf{s}}_t = g(\hat{\mathbf{s}}_{t-1}, \mathbf{u}_t), \qquad \bar{\mathbf{P}}_t = \mathbf{G}_t \mathbf{P}_{t-1} \mathbf{G}_t^\top + \mathbf{Q}_t, \tag{5.5}$$

where $\mathbf{G}_t = \frac{\partial g}{\partial \mathbf{s}}\big|_{\hat{\mathbf{s}}_{t-1}}$ is the Jacobian and $\mathbf{Q}_t$ is the process noise covariance. When observation $\mathbf{z}_t$ of landmark k arrives, the update step employs the Kalman gain:

$$\mathbf{K} = \bar{\mathbf{P}}_t \mathbf{H}_t^\top \big(\mathbf{H}_t \bar{\mathbf{P}}_t \mathbf{H}_t^\top + \mathbf{R}_t\big)^{-1}, \tag{5.6}$$

$$\hat{\mathbf{s}}_t = \bar{\mathbf{s}}_t + \mathbf{K}\big(\mathbf{z}_t - h(\bar{\mathbf{s}}_t)\big), \qquad \mathbf{P}_t = (\mathbf{I} - \mathbf{K}\mathbf{H}_t)\bar{\mathbf{P}}_t, \tag{5.7}$$

where $\mathbf{H}_t = \frac{\partial h}{\partial \mathbf{s}}\big|_{\bar{\mathbf{s}}_t}$ and $h(\cdot)$ is the observation function. The key observation is that $\mathbf{H}_t$ is sparse — an observation constrains only the robot pose and the specific landmark being observed — but the gain $\mathbf{K}$ is dense, and the covariance update requires $O(L^2)$ operations because $\mathbf{K}\mathbf{H}_t$ is dense. With L landmarks, the per-step complexity is $O(L^2)$, rendering EKF-SLAM intractable for more than a few hundred landmarks.

The information form of the EKF offers a different perspective. Writing the Gaussian in canonical form with information vector $\boldsymbol{\eta} = \boldsymbol{\Omega}\boldsymbol{\mu}$ and information matrix $\boldsymbol{\Omega} = \mathbf{P}^{-1}$, the update becomes *additive* and sparse:

$$\bar{\boldsymbol{\Omega}}_t = \big(\mathbf{G}_t \boldsymbol{\Omega}_{t-1}^{-1} \mathbf{G}_t^\top + \mathbf{Q}_t\big)^{-1}, \qquad \boldsymbol{\Omega}_t = \bar{\boldsymbol{\Omega}}_t + \mathbf{H}_t^\top \mathbf{R}_t^{-1} \mathbf{H}_t. \tag{5.8}$$

The observation update adds a sparse increment $\mathbf{H}_t^\top \mathbf{R}_t^{-1} \mathbf{H}_t$, but the prediction step requires inverting the information matrix — an $O(n^3)$ operation — defeating the purpose. The Sparse Extended Information Filter (SEIF) approximates by enforcing sparsity on $\boldsymbol{\Omega}$, discarding weak links at the cost of overconfidence.

5.2.3 FastSLAM: Rao-Blackwellized Particle Filtering

FastSLAM (Montemerlo et al., 2002) exploits the conditional independence structure of SLAM to achieve logarithmic complexity in the number of landmarks. The key factorization is:

$$p(\mathbf{x}_{1:t}, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) = p(\mathbf{x}_{1:t} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) \prod_{k=1}^{L} p(\mathbf{m}_k \mid \mathbf{x}_{1:t}, \mathbf{z}_{1:t}). \tag{5.9}$$

This identity follows from the observation that, *given* the robot trajectory $\mathbf{x}_{1:t}$, individual landmark posteriors become conditionally independent of one another because the only coupling between landmarks was through their shared dependence on the uncertain trajectory. The decoupling transforms the joint estimation problem into a trajectory estimation problem (solved with a particle filter over M particles) plus L independent landmark estimation problems (each solved with a 3×3 EKF).

Each particle i maintains a sampled robot trajectory $\mathbf{x}_{1:t}^{[i]}$ together with independent EKFs for each landmark — one mean $\hat{\mathbf{m}}_k^{[i]}$ and covariance $\mathbf{\Sigma}_k^{[i]}$ per landmark per particle. When observing landmark k, *only that landmark's* EKF is updated. The particle weight is updated according to the observation likelihood, marginalized over the landmark uncertainty:

$$w_t^{[i]} \propto w_{t-1}^{[i]}\, p(\mathbf{z}_t \mid \mathbf{x}_t^{[i]}, \hat{\mathbf{m}}_k^{[i]}, \mathbf{\Sigma}_k^{[i]}), \tag{5.10}$$

computed via the innovation covariance $S = \mathbf{H}\mathbf{\Sigma}_k^{[i]}\mathbf{H}^\top + \mathbf{R}$ evaluated for the specific particle.

The Rao-Blackwellized structure concentrates Monte Carlo variance on the low-dimensional trajectory while treating landmarks analytically. FastSLAM with M particles and L landmarks has complexity $O(M \log L)$ using tree-based data structures for efficient landmark lookup — a dramatic improvement over $O(L^2)$ for EKF-SLAM. FastSLAM 2.0 further improves by sampling poses from the posterior that incorporates the current observation, using the proposal $p(\mathbf{x}_t \mid \mathbf{x}_{t-1}^{[i]}, \mathbf{u}_t, \mathbf{z}_t, \{\hat{\mathbf{m}}_k^{[i]}\})$, which reduces particle depletion.

5.2.4 Pose Graph SLAM: Optimization on Lie Groups

Modern SLAM systems (Kümmerle et al., 2011; Kaess et al., 2012) cast the problem as nonlinear least-squares optimization over a *pose graph*. Nodes represent robot poses $\mathbf{x}_1, \ldots, \mathbf{x}_T$ with each $\mathbf{x}_i \in \mathrm{SE}(d)$ (for $d = 2, 3$), and edges represent relative pose constraints $\mathbf{z}_{ij}$ between $\mathbf{x}_i$ and $\mathbf{x}_j$. These constraints arise from odometry (consecutive poses), scan matching or visual feature matching (loop closures), or GPS when available.

The optimization objective minimizes the sum of squared residuals, each weighted by the inverse covariance (information matrix) of the corresponding measurement:

$$\mathbf{x}^* = \arg\min_{\mathbf{x}_1, \ldots, \mathbf{x}_T} \sum_{(i,j) \in \mathcal{E}} \|\mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j)\|^2_{\mathbf{\Omega}_{ij}}\,, \tag{5.11}$$

where the error function on SE(3) is:

$$\mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j) = \log_{\mathrm{SE}(3)}\!\big(\mathbf{z}_{ij}^{-1} \circ (\mathbf{x}_i^{-1} \circ \mathbf{x}_j)\big) \in \mathbb{R}^6, \tag{5.12}$$

with $\circ$ denoting the group composition, $\mathbf{z}_{ij}$ the measured relative pose, $\mathbf{x}_i^{-1} \circ \mathbf{x}_j$ the predicted relative pose from current estimates, and $\log_{\mathrm{SE}(3)} : \mathrm{SE}(3) \to \mathfrak{se}(3)$ the logarithmic map that extracts the 6D twist vector from a transformation matrix. The Mahalanobis norm is $\|\mathbf{e}\|^2_{\mathbf{\Omega}} = \mathbf{e}^\top \mathbf{\Omega} \mathbf{e}$ with information $\mathbf{\Omega}_{ij} = \mathbf{\Sigma}_{ij}^{-1}$.

Optimization proceeds via Gauss-Newton or Levenberg-Marquardt on the manifold. The error function is linearized around the current linearization point using the Lie algebra parameterization:

$$\mathbf{e}_{ij}\big(\mathbf{x}_i \exp(\boldsymbol{\xi}_i^\wedge), \mathbf{x}_j \exp(\boldsymbol{\xi}_j^\wedge)\big) \approx \mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j) + \mathbf{J}_i \boldsymbol{\xi}_i + \mathbf{J}_j \boldsymbol{\xi}_j, \tag{5.13}$$

where $\exp(\boldsymbol{\xi}^\wedge)$ maps a twist vector $\boldsymbol{\xi} \in \mathbb{R}^6$ to an SE(3) perturbation via the exponential map, and $\mathbf{J}_i, \mathbf{J}_j \in \mathbb{R}^{6\times 6}$ are Jacobians with respect to the local coordinates. Substituting the linearization into the squared-error objective yields the normal equations:

$$\mathbf{H}\Delta\boldsymbol{\xi} = -\mathbf{b}, \qquad \mathbf{H} = \sum_{(i,j)} \mathbf{J}_{ij}^\top \boldsymbol{\Omega}_{ij} \mathbf{J}_{ij}, \qquad \mathbf{b} = \sum_{(i,j)} \mathbf{J}_{ij}^\top \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij}, \tag{5.14}$$

where $\mathbf{J}_{ij} = [\mathbf{0} \cdots \mathbf{J}_i \cdots \mathbf{J}_j \cdots \mathbf{0}]$ is the full Jacobian with non-zero blocks only at columns i and j. The Hessian $\mathbf{H}$ inherits the sparsity pattern of the pose graph — it is block-tridiagonal for chain graphs with occasional off-diagonal entries from loop closures — enabling efficient solution via sparse Cholesky factorization (e.g., CHOLMOD) in time roughly $O(T)$ for trajectory-like graphs and $O(T^{3/2})$ for graphs with many closures. Variable elimination ordering (COLAMD, AMD) dramatically affects fill-in and runtime.

5.2.5 Learnable SLAM and Beyond

While classical SLAM algorithms are analytically derived and interpretable, they rely on hand-designed feature extraction and data association. Learned SLAM components address these limitations by replacing brittle heuristics with neural networks trained on data.

Learned front-ends replace hand-crafted visual features (SIFT, ORB) with learned keypoint detectors and descriptors. SuperPoint (DeTone et al., 2018) uses a fully convolutional network to jointly detect interest points and compute fixed-length descriptors. Training uses self-supervision: synthetic warps of natural images provide ground-truth correspondences. The keypoint detection head outputs a heatmap $p(u, v)$ of interest-point probability, trained with cross-entropy against pseudo-ground-truth from classical detectors. The descriptor head outputs a D-dimensional vector per pixel, trained with a contrastive loss:

$$\mathcal{L}_{\text{desc}} = \sum_{(i,j)\in\mathcal{P}} \|\mathbf{d}_i - \mathbf{d}_j\|^2 + \sum_{(i,j)\in\mathcal{N}} \max(0, m - \|\mathbf{d}_i - \mathbf{d}_j\|)^2, \tag{5.15}$$

where $\mathcal{P}$ are positive (matching) pairs and $\mathcal{N}$ are negative (non-matching) pairs, and m is a margin.

Learned data association replaces nearest-neighbor matching with graph neural network-based matching. SuperGlue (Sarlin et al., 2020) formulates feature matching as an optimal transport problem. Given two sets of keypoints with descriptors $(\mathbf{D}_A, \mathbf{D}_B)$, a GNN produces a soft assignment matrix $\mathbf{S} \in [0, 1]^{M\times N}$:

$$\mathbf{S} = \text{softmax}\big(\text{Sinkhorn}(\mathbf{D}_A \mathbf{D}_B^\top + \text{GNN}(\mathbf{D}_A, \mathbf{D}_B))\big), \tag{5.16}$$

where the Sinkhorn algorithm iteratively normalizes rows and columns to satisfy the doubly-stochastic constraint of optimal transport. SuperGlue is trained end-to-end on pose estimation accuracy.

5.2.6 From Classical to Learned SLAM Architectures

The line between classical and learned SLAM blurs in systems that retain optimization structure but replace components with neural networks. DeepFactors (Czarnowski et al., 2020) learns a compact codebook of depth bases that can be composed to explain any scene, replacing the explicit depth sensor model with a learned generative model. Similarly, BA-Net (Tang and Tan, 2019) embeds the bundle adjustment optimization into the network architecture, unrolling Levenberg-Marquardt iterations as differentiable layers.

The key tension is between *inductive bias* (the optimization structure of classical SLAM, which provides sample efficiency and interpretability) and *learning capacity* (which provides robustness to domain shift and the ability to handle semantically complex environments). Modern systems increasingly adopt a hybrid philosophy: optimize what you can (pose graph structure), learn what you must (features, data association, and semantic interpretation).

5.3 Key Techniques

5.3.1 Loop Closure Detection

Loop closure — recognizing that the robot has returned to a previously visited location — distinguishes SLAM from pure odometry. Without loop closures, odometric drift accumulates without bound; with loop closures, graph optimization redistributes error globally, producing metrically consistent maps.

Classical approaches use *bag-of-words* (BoW) representations of images, as in ORB-SLAM3 (Campos et al., 2021). Keyframes are described by histograms of ORB visual words: feature descriptors are quantized into a vocabulary tree of 10^4–10^6 words, and each image is represented by a sparse frequency vector over this vocabulary. Loop candidates are retrieved via TF-IDF scoring:

$$s(I_A, I_B) = 1 - \frac{1}{2}\left\| \frac{\mathbf{v}_A}{\|\mathbf{v}_A\|} - \frac{\mathbf{v}_B}{\|\mathbf{v}_B\|} \right\|, \tag{5.17}$$

with verification via geometric consistency (epipolar geometry or 3D point alignment). Efficient retrieval uses inverted indices for constant-time lookup.

Learned alternatives such as NetVLAD (Arandjelović et al., 2016) produce compact vector embeddings of image regions via a trainable VLAD layer:

$$V(j,k) = \sum_{i=1}^{N} \bar{a}_k(\mathbf{x}_i)\big(x_i(j) - c_k(j)\big), \tag{5.18}$$

where $\bar{a}_k(\mathbf{x}_i) = \text{softmax}_k(\mathbf{W}\mathbf{x}_i + \mathbf{b})$ are soft-assignment weights and $\mathbf{c}_k$ are learned cluster centers. The resulting VLAD descriptor is trained with triplet loss to be invariant to viewpoint while discriminative across places. More recently, vision-language models like CLIP have been investigated for open-vocabulary place recognition, where the query can be expressed in natural language.

5.3.2 Learned SLAM: Neural SLAM

Chaplot et al. (2020) introduced *Neural SLAM*, which learns to build top-down semantic and occupancy maps directly from egocentric RGB observations — bypassing explicit depth estimation, feature matching, and pose optimization. This is a radical departure from the geometric pipeline and represents a bet that learned, data-driven end-to-end mapping will eventually surpass hand-engineered pipelines in robustness and semantic richness.

The mapping module receives visual observations encoded by a CNN and projects features into a top-down egocentric map via a differentiable inverse perspective mapping (IPM). The IPM transforms the first-person view into a top-down representation using the camera intrinsics and agent pose:

$$\mathbf{m}_t^{\text{ego}}(u,v) = \sum_{i,j} \alpha_{i,j}(u,v) \cdot \text{CNN}(\mathbf{o}_t)_{i,j}, \tag{5.19}$$

where $\alpha_{i,j}(u,v)$ are differentiable interpolation weights that map image pixels to top-down grid cells. This egocentric map is accumulated over time into an allocentric global map via a recurrent neural update mechanism:

$$\mathbf{m}_t = \mathrm{ReLU}\big(\mathbf{U} * \mathbf{m}_{t-1} + g_\theta(\mathbf{m}_t^{\mathrm{ego}})\big), \tag{5.20}$$

where $*$ denotes convolution and $\mathbf{U}$ is a learned recurrent kernel that propagates information forward in time while allowing previously observed regions to persist.

The training objective combines multiple losses that capture different aspects of map quality:

$$\mathcal{L} = \mathcal{L}_{\mathrm{occ}} + \lambda_{\mathrm{sem}}\mathcal{L}_{\mathrm{sem}} + \lambda_{\mathrm{expl}}\mathcal{L}_{\mathrm{expl}}. \tag{5.21}$$

The occupancy loss $\mathcal{L}_{\mathrm{occ}}$ is binary cross-entropy between predicted and ground-truth occupancy in each grid cell:

$$\mathcal{L}_{\mathrm{occ}} = -\frac{1}{HW}\sum_{u,v}\big[o^{\mathrm{gt}}_{u,v}\log\hat{o}_{u,v} + (1-o^{\mathrm{gt}}_{u,v})\log(1-\hat{o}_{u,v})\big]. \tag{5.22}$$

The semantic loss $\mathcal{L}_{\mathrm{sem}}$ is categorical cross-entropy over per-cell object category predictions:

$$\mathcal{L}_{\mathrm{sem}} = -\frac{1}{HW}\sum_{u,v}\sum_{c=1}^{C} y^{\mathrm{gt}}_{u,v,c}\log\hat{y}_{u,v,c}. \tag{5.23}$$

The exploration loss $\mathcal{L}_{\mathrm{expl}}$ rewards the agent for visiting previously unobserved regions, implemented as the negative count of novel cells observed during a trajectory:

$$\mathcal{L}_{\mathrm{expl}} = -\sum_{c\in\mathcal{C}_{\mathrm{new}}} \mathbb{1}[c \text{ is observed for the first time}]. \tag{5.24}$$

Active Neural SLAM couples the learned mapper with a hierarchical policy. A global policy — trained with reinforcement learning — selects long-term goals (frontier locations on the predicted map), while a local policy computes the shortest path to the goal and executes it using low-level actuation. The global policy's reward is the reduction in map uncertainty, implicitly encouraging exhaustive exploration:

$$R_{\mathrm{global}} = \sum_{c}\big(H(\hat{o}_c^{\mathrm{before}}) - H(\hat{o}_c^{\mathrm{after}})\big), \tag{5.25}$$

where $H(p) = -p\log p - (1-p)\log(1-p)$ is the binary entropy measuring per-cell occupancy uncertainty.

5.3.3 Implicit Representations for Dense SLAM

A paradigm shift in dense SLAM replaces explicit geometric representations (point clouds, surfels, TSDF volumes) with *neural implicit fields*. Methods such as iMAP (Sucar et al., 2021), NICE-SLAM (Zhu et al., 2022), and NeRF-SLAM jointly optimize a coordinate-based neural network $F_\theta : \mathbb{R}^3 \to \mathbb{R}^c \times \mathbb{R}^+$ that maps 3D coordinates to color $\mathbf{c}$ and volume density σ, while simultaneously estimating camera poses $\{\mathbf{T}_t\}$.

The rendering equation for a ray $\mathbf{r}(s) = \mathbf{o} + s\mathbf{d}$ parameterized by depth s is:

$$\mathbf{C}(\mathbf{r}) = \int_{s_n}^{s_f} T(s)\,\sigma(\mathbf{r}(s))\,\mathbf{c}(\mathbf{r}(s))\,ds, \tag{5.26}$$

where the accumulated transmittance is:

$$T(s) = \exp\!\left(-\int_{s_n}^{s}\sigma(\mathbf{r}(u))\,du\right), \tag{5.27}$$

representing the probability that the ray travels from s_n to s without hitting any particle. The rendering loss compares the integrated color against the observed pixel:

$$\mathcal{L}_{\text{render}} = \sum_{\mathbf{r}\in\mathcal{R}} \left\| \hat{\mathbf{C}}(\mathbf{r}) - \mathbf{C}_{\text{gt}}(\mathbf{r}) \right\|^2. \tag{5.28}$$

Crucially, the loss gradient with respect to camera pose $\mathbf{T}$ can be computed via the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{T}} = \frac{\partial \mathcal{L}}{\partial \mathbf{r}} \cdot \frac{\partial \mathbf{r}}{\partial \mathbf{T}} = \frac{\partial \mathcal{L}}{\partial \mathbf{r}} \cdot \frac{\partial}{\partial \mathbf{T}} \big(\mathbf{T} \cdot \mathbf{r}_{\text{cam}}(s)\big). \tag{5.29}$$

This *gradient-based pose estimation* enables simultaneous optimization of scene representation and trajectory, eliminating the need for explicit feature extraction and correspondence matching. NICE-SLAM improves on iMAP by using a hierarchical feature grid that stores learnable features at multiple resolutions, enabling faster convergence and finer detail than a pure MLP representation. The scene at position $\mathbf{x}$ is decoded from multi-level interpolated features:

$$(\mathbf{c}(\mathbf{x}), \sigma(\mathbf{x})) = \text{MLP}\big(\mathbf{x}, \{\phi_\ell(\mathbf{x})\}_{\ell=1}^{L}\big), \tag{5.30}$$

where $\phi_\ell(\mathbf{x})$ are spatially interpolated features from the ℓ-th feature grid level.

A related line uses *signed distance functions* (SDFs). The surface $\mathcal{S}$ of a scene is the zero-level set of a function $\Phi : \mathbb{R}^3 \to \mathbb{R}$:

$$\mathcal{S} = \{\mathbf{x} \in \mathbb{R}^3 \mid \Phi(\mathbf{x}) = 0\}, \tag{5.31}$$

with the convention $\Phi(\mathbf{x}) < 0$ inside objects and $\Phi(\mathbf{x}) > 0$ in free space. Depth observations are fused into a Truncated SDF (TSDF) volume, where truncation with parameter τ limits the integration radius to a narrow band around the surface:

$$\Phi_{\text{TSDF}}(\mathbf{x}) = \min\big(1, \Phi(\mathbf{x})/\tau\big) \cdot \mathbb{1}[|\Phi(\mathbf{x})| \le \tau]. \tag{5.32}$$

Differentiable sphere tracing renders SDFs for gradient-based refinement of geometry and camera poses, forming a self-consistent optimization loop between representation and estimation.

5.3.4 Beyond Geometry: Semantic and Task-Driven Mapping

Spatial understanding for embodied agents transcends geometric reconstruction. A meaningful spatial representation must answer: *Where can I walk? Where are objects likely to be found? What can I do here?*

Semantic mapping augments occupancy grids with object category labels per cell. The joint posterior factorizes as:

$$p(\mathbf{m}_{\text{geo}}, \mathbf{m}_{\text{sem}} \mid \mathbf{z}_{1:t}) = p(\mathbf{m}_{\text{geo}} \mid \mathbf{z}_{1:t})\; p(\mathbf{m}_{\text{sem}} \mid \mathbf{m}_{\text{geo}}, \mathbf{z}_{1:t}). \tag{5.33}$$

This factorization enables separate treatment of geometric and semantic inference, with geometry providing spatial support for semantic predictions. Recent work extends this to *dynamic semantic maps* that track moving entities and model occlusion relationships.

Affordance maps (Grabner et al., 2011) encode what actions are feasible at each location. For a given skill π_k, the map $A_k(\mathbf{x})$ gives the probability of successful execution:

$$A_k(\mathbf{x}) = p(\text{success} \mid \mathbf{x}, \pi_k, \mathbf{m}). \tag{5.34}$$

These maps can be learned from experience via function approximation and used to bias planners toward likely-successful actions. The concept generalizes navigation affordances (traversability) to semantic affordances (an object at location $\mathbf{x}$ is graspable, a surface at $\mathbf{x}$ is placeable).

5.4 Connections

Gaussian Splatting: 3D Gaussian Splatting (3DGS; Kerbl et al., 2023) provides an alternative to NeRF for real-time differentiable rendering. A scene is represented as a collection of anisotropic 3D Gaussians, each with position $\boldsymbol{\mu}$, covariance $\boldsymbol{\Sigma} = \mathbf{RSS}^\top\mathbf{R}^\top$, color modeled as spherical harmonics, and opacity α. The rendering is projection-based rather than ray-marching, achieving real-time frame rates. SLAM systems using 3DGS (e.g., SplaTAM, Gaussian-SLAM) combine rapid rendering with explicit geometric proxy for camera tracking.

Representation learning for navigation: ObjectNav and ImageNav (Anderson et al., 2018; Batra et al., 2020) train policies that implicitly build spatial representations in neural network activations. The implicit-explicit divide — should maps be decoded as geometric entities or represented as latent vectors? — remains unresolved and connects SLAM to broader questions in representation learning.

Active perception: SLAM is intrinsically active — the agent chooses where to look and move to maximize information gain. The expected information gain for a future action a can be estimated as the mutual information between map and future observation:

$$I(\mathbf{m}; \mathbf{z} \mid a) = H(\mathbf{m}) - H(\mathbf{m} \mid \mathbf{z}, a) = \mathbb{E}_{p(\mathbf{z}|a)}\left[\mathrm{KL}\big(p(\mathbf{m} \mid \mathbf{z}, a) \,\|\, p(\mathbf{m})\big)\right]. \tag{5.35}$$

Greedy or planning-based approaches use this to guide exploration, connecting SLAM to reinforcement learning and optimal experimental design.

Simultaneous localization, mapping, and manipulation: Interactive settings demand joint estimation and control. Opening doors, moving furniture, and grasping objects change the environment. The map must track object states while the agent is both observer and actor — a coupled estimation-and-control problem bridging SLAM and manipulation.

5.5 Open Problems

1. **Lifelong SLAM:** Current systems assume sessions of hours. Real agents operate over weeks and years, requiring persistent maps that adapt to gradual environmental changes (furniture rearranged, seasons changing, construction). The recursive filter structure is ill-suited to long-term operation; information-theoretic forgetting and map management strategies are essential.

2. **Multi-agent SLAM:** Multiple robots mapping a shared environment must reconcile their individual coordinate frames. Decentralized pose-graph optimization over ad hoc networks introduces challenging consensus problems. Communication bandwidth constraints dictate what compressed information — submaps, feature embeddings, learned descriptors — should be shared.

3. **Uncertainty-aware downstream use:** SLAM produces a distribution over maps, not a point estimate. Downstream tasks (planning, grasping, navigation) should be uncertainty-aware, reasoning about worst-case or expected utility under the posterior. Collapsing the posterior to a point estimate discards valuable information about what the robot does and does not know.

4. **Scene dynamics:** Most SLAM systems treat the world as static between observations. In inhabited environments, people, pets, and vehicles move. Robust SLAM must detect, segment, and track dynamic entities — incorporating them into motion models rather than treating them as outliers — and distinguish between permanent and transient structure.

5. **Semantic SLAM with open vocabularies:** CLIP and vision-language models enable open-vocabulary object recognition. Building maps with open-vocabulary semantic labels

— allowing queries like "where is an object I can use to reach something high" — is an open frontier bridging spatial understanding and language grounding.

6. **Robustness to perceptual aliasing:** Distinct places can appear identical (identical corridors in a hospital, repetitive facades). Incorrect loop closures are catastrophic, tearing the pose graph apart. Robust methods that maintain multi-hypothesis data associations via nonparametric belief propagation or multi-hypothesis iSAM remain an important research area.

7. **Generalization across sensor modalities:** SLAM must work with diverse sensors — monocular, stereo, RGB-D, LiDAR, event cameras, thermal — and handle sensor degradation (motion blur, low light, rain). Sensor-agnostic SLAM that abstracts away modality-specific preprocessing is a long-term goal.

Part III

Manipulation & Interaction

Chapter 6

Robotic Grasping

6.1 Motivation

The ability to grasp and manipulate objects is the defining physical competence of embodied intelligence. While humans grasp objects effortlessly — adapting grip force, finger placement, and approach direction to an astonishing variety of shapes, materials, and task contexts — robotic grasping remains a frontier problem. The challenge spans geometric reasoning (finding a 6-DoF gripper configuration that makes stable contact), physical prediction (will the object slip, tilt, or break?), and semantic understanding (which grasp is appropriate for the intended task: a mug grasped by its rim is stable but useless for drinking).

The past decade has witnessed a transformation: grasping has moved from slow, analytical, model-based approaches that required known object CAD models and careful calibration, to fast, data-driven, model-free approaches that work on novel objects from a single depth image. This chapter develops the mathematical foundations of grasp quality and surveys the deep learning methods that have made robust grasping a reality on physical robots.

6.2 Core Concepts and Key Papers

6.2.1 The Grasp Synthesis Problem

Given a perception of the scene — typically a depth image $\mathbf{D} \in \mathbb{R}^{H \times W}$ or a partial point cloud $\mathcal{P} = \{\mathbf{p}_i \in \mathbb{R}^3\}_{i=1}^{N}$ — the grasp synthesis problem is to find a set of candidate grasps $\{\mathbf{g}_j\}$ that, when executed by the robot's end-effector, result in a stable hold of the target object. For a parallel-jaw gripper, each grasp is parameterized as:

$$\mathbf{g} = (\mathbf{p}_{\text{center}}, \mathbf{R}, w), \qquad \mathbf{p}_{\text{center}} \in \mathbb{R}^3, \quad \mathbf{R} \in \text{SO}(3), \quad w \in \mathbb{R}^+, \tag{6.1}$$

where $\mathbf{p}_{\text{center}}$ is the 3D position of the gripper center, $\mathbf{R}$ is the 3×3 rotation matrix encoding approach direction and finger orientation, and w is the opening width between the fingers.

The *antipodal grasp* principle provides a geometric heuristic: stable grasps require two contact points whose surface normals are approximately opposite:

$$\mathbf{n}_1 + \mathbf{n}_2 \approx \mathbf{0}, \qquad |\mathbf{n}_1^\top (\mathbf{p}_2 - \mathbf{p}_1)| \approx \|\mathbf{p}_2 - \mathbf{p}_1\|\,. \tag{6.2}$$

The first condition ensures the contact forces are collinear; the second ensures the line connecting the contacts is approximately parallel to the surface normals. While antipodal geometry is neither necessary nor sufficient for a stable grasp, it dramatically reduces the search space of candidate configurations.

6.2.2 Force Closure: The Foundation of Grasp Stability

A grasp is *force-closed* if it can resist any arbitrary external wrench (force and torque) applied to the object, assuming sufficient actuator strength and bounded friction. This concept, rooted in screw theory, provides a rigorous mathematical criterion for grasp quality that has guided both analysis and synthesis for decades.

Consider K contact points between the gripper fingers and the object. At each contact i, the force $\mathbf{f}_i \in \mathbb{R}^3$ that the finger can exert must lie within a *Coulomb friction cone*:

$$\mathcal{F}_i = \{\mathbf{f}_i \in \mathbb{R}^3 \mid \left\|\mathbf{f}_i^t\right\| \leq \mu f_i^n,\ f_i^n \geq 0\}, \tag{6.3}$$

where $\mathbf{f}_i = (\mathbf{f}_i^t, f_i^n)$ is decomposed into tangential and normal components relative to the contact surface, and μ is the coefficient of friction. This quadratic cone can be linearized as a polyhedral cone with M generators $\{\mathbf{g}_{i,j}\}_{j=1}^{M}$ for computational tractability:

$$\mathbf{f}_i = \sum_{j=1}^{M} \alpha_{i,j}\mathbf{g}_{i,j}, \qquad \alpha_{i,j} \geq 0. \tag{6.4}$$

The *grasp matrix* $\mathbf{G}_i \in \mathbb{R}^{6\times 3}$ maps each contact force (expressed in the object's body frame) to the resulting wrench on the object:

$$\mathbf{w}_i = \mathbf{G}_i\mathbf{f}_i = \begin{bmatrix} \mathbf{I}_{3\times 3} \\ [(\mathbf{p}_i - \mathbf{p}_{\text{com}})]_\times \end{bmatrix} \mathbf{f}_i, \tag{6.5}$$

where $\mathbf{p}_i$ is the contact position, $\mathbf{p}_{\text{com}}$ is the object's center of mass, and $[\mathbf{a}]_\times \in \mathbb{R}^{3\times 3}$ is the skew-symmetric matrix satisfying $[\mathbf{a}]_\times\mathbf{b} = \mathbf{a} \times \mathbf{b}$. Stacking all K contacts, the total wrench applied to the object is:

$$\mathbf{w} = \mathbf{G}\mathbf{f} = \begin{bmatrix} \mathbf{G}_1 & \cdots & \mathbf{G}_K \end{bmatrix} \begin{bmatrix} \mathbf{f}_1 \\ \vdots \\ \mathbf{f}_K \end{bmatrix}, \qquad \mathbf{G} \in \mathbb{R}^{6\times 3K}. \tag{6.6}$$

A grasp is formally force-closed if and only if two conditions hold: (i) $\mathbf{G}$ has full row rank (the grasp can generate wrenches spanning all of $\mathbb{R}^6$ — the unit wrench space), and (ii) there exists a strictly internal force $\mathbf{f} > \mathbf{0}$ within the friction cones such that $\mathbf{G}\mathbf{f} = \mathbf{0}$ (the grasp can squeeze without producing a net wrench on the object). This internal force condition ensures that friction can be actively modulated without disturbing the object, a property necessary for resisting disturbances from any direction.

6.2.3 Grasp Quality Metrics

Grasp quality metrics quantify how robust a grasp is, enabling optimization and comparison. The *manipulability* or *wrench* perspective defines the grasp wrench space (GWS) — the convex hull of all primitive wrenches that can be exerted by unit forces at the extremes of the friction cones:

$$\text{GWS} = \text{Conv}\left(\bigcup_{i=1}^{K} \{\mathbf{G}_i\mathbf{g}_{i,1}, \ldots, \mathbf{G}_i\mathbf{g}_{i,M}\}\right) \subset \mathbb{R}^6. \tag{6.7}$$

The *Ferrari-Canny* ε-metric (Q_1 metric) measures the radius of the largest origin-centered 6D sphere that fits entirely within the GWS:

$$Q_\varepsilon = \sup_{\varepsilon \geq 0} \{\varepsilon \mid \mathcal{B}_\varepsilon \subseteq \text{GWS}\}, \qquad \mathcal{B}_\varepsilon = \{\mathbf{w} \in \mathbb{R}^6 : \|\mathbf{w}\|_2 \leq \varepsilon\}. \tag{6.8}$$

A larger ε means the grasp can resist larger disturbances in all directions. Computing Q_ε requires solving a linear program (LP) for each grasp:

$$\begin{aligned} \min_{\varepsilon,\boldsymbol{\alpha}} \quad & -\varepsilon \\ \text{s.t.} \quad & \sum_{i,j} \alpha_{i,j} (\mathbf{G}_i \mathbf{g}_{i,j})_k \geq \varepsilon, \quad k = 1, \ldots, 6, \\ & \sum_{i,j} \alpha_{i,j} (\mathbf{G}_i \mathbf{g}_{i,j})_k \leq -\varepsilon, \quad k = 1, \ldots, 6, \\ & \sum_{i,j} \alpha_{i,j} = 1, \quad \alpha_{i,j} \geq 0. \end{aligned} \tag{6.9}$$

The *volume* metric Q_{vol} measures the full volume of the GWS:

$$Q_{\text{vol}} = \text{Vol}(\text{GWS}) = \int_{\text{GWS}} d\mathbf{w}. \tag{6.10}$$

In practice, this volume is approximated by computing the convex hull of primitive wrenches and evaluating its 6-dimensional volume using QuickHull. The volume metric is more expensive than ε but provides a more nuanced measure of directional robustness.

6.2.4 Dex-Net: Data-Driven Grasp Quality at Scale

The Dex-Net family (Mahler et al., 2017–2019) represents a landmark in bringing deep learning to grasp synthesis. The central insight is elegantly simple: analytic grasp metrics, while computationally prohibitive at runtime, can be precomputed offline on a massive synthetic dataset and used to train a convolutional neural network that predicts grasp quality from depth images in real time. The simulation-to-reality transfer is surprisingly effective because depth sensing is relatively invariant to visual domain shift.

Dex-Net 2.0 (Mahler et al., 2017) focuses on planar (4-DoF) top-down grasps, a simplified but practically important setting for bin picking. The Grasp Quality CNN (GQ-CNN) takes as input a depth image centered on a grasp candidate and outputs a scalar robustness prediction:

$$\hat{Q} = f_\theta(\mathbf{D}_{\text{local}}) \in [0, 1], \qquad \mathbf{D}_{\text{local}} \in \mathbb{R}^{32 \times 32}. \tag{6.11}$$

Training data generation proceeds as follows: sample 3D object meshes from a database of thousands of CAD models; generate random stable poses on a planar surface via rigid-body simulation; compute antipodal grasp candidates using geometric heuristics (intersecting the object silhouette with parallel lines); label each grasp with the ε-metric via LP; render synthetic depth images from a virtual top-down camera; and train the GQ-CNN to regress Q using an L_2 loss. With 6.7 million synthetic grasps covering over 1,500 object models, Dex-Net 2.0 achieves 99.3% grasp success rate on novel objects never seen during training, demonstrating that simulation-trained models generalize robustly to physical hardware.

Dex-Net 4.0 (Mahler et al., 2019) extends to full 6-DoF grasping. The GQ-CNN now operates on a point cloud: the local point cloud within a bounding volume around the grasp candidate is transformed into the gripper frame and processed by a PointNet-like architecture to predict robustness:

$$\hat{Q} = f_\theta\big(\mathcal{P}_{\text{local}}, \mathbf{g}\big), \qquad \mathcal{P}_{\text{local}} = \{\mathbf{T}_{\mathbf{g}}^{-1} \mathbf{p}_i \mid \mathbf{p}_i \in \mathcal{P}, \|\mathbf{p}_i - \mathbf{p}_{\mathbf{g}}\| < r\}, \tag{6.12}$$

where $\mathbf{T}_{\mathbf{g}}$ transforms points to the gripper frame. The network learns to implicitly evaluate force-closure conditions directly from raw point data, bypassing explicit surface normal estimation, friction cone construction, and linear programming at runtime.

6.2.5 GG-CNN: Generative Grasping

While Dex-Net samples candidate grasps and scores them with a CNN, the Generative Grasping CNN (GG-CNN) by Morrison et al. (2018) takes a radically more direct approach: from a single depth image, it produces *pixel-wise* predictions of grasp quality, angle, and width in one forward pass — no candidate sampling, no iterative refinement, no multiple network evaluations. The network runs at approximately 20ms per image, fast enough for closed-loop reactive grasping on moving objects.

For each pixel (u, v) in the input depth image, GG-CNN simultaneously predicts three quantities:

$$Q(u,v) \in [0,1] \quad \text{(grasp quality — probability of successful grasp)}, \tag{6.13}$$

$$\Phi(u,v) = \frac{1}{2}\arctan\left(\frac{\sin_{\text{pred}}(u,v)}{\cos_{\text{pred}}(u,v)}\right) \in [-\tfrac{\pi}{2}, \tfrac{\pi}{2}] \quad \text{(grasp angle)}, \tag{6.14}$$

$$W(u,v) \in [0, W_{\max}] \quad \text{(gripper opening width)}. \tag{6.15}$$

The angle is predicted via the two-argument arctangent applied to regressed $\sin(2\theta)$ and $\cos(2\theta)$ components, which elegantly avoids the discontinuity at $\pm\pi/2$ that plagues direct angle regression. The training loss combines three terms with task-specific weighting:

$$\mathcal{L} = \mathcal{L}_Q + \lambda_\Phi \mathcal{L}_\Phi + \lambda_W \mathcal{L}_W, \tag{6.16}$$

where $\mathcal{L}_Q = \frac{1}{HW}\sum_{u,v}(Q(u,v) - Q_{\text{gt}}(u,v))^2$ is MSE on grasp quality, $\mathcal{L}_\Phi = \frac{1}{HW}\sum_{u,v}(1 - \cos(2(\hat{\Phi} - \Phi_{\text{gt}})))$ is angular distance, and $\mathcal{L}_W$ is smooth L_1 on width. The ground-truth labels are derived from antipodal grasp annotations on the Cornell Grasping Dataset.

The generative formulation has profound implications: the network can represent *multi-modality* — in a scene with multiple graspable objects or multiple valid grasps on the same object, the quality map $Q(\cdot)$ can have multiple peaks, and any of them is a valid output. This contrasts with discriminative approaches that commit to a single output per candidate.

6.3 Key Techniques

6.3.1 6-DoF Grasp Detection from Point Clouds

Full 6-DoF grasp detection in SE(3) presents a substantial computational challenge: the configuration space is a 6-dimensional manifold, and naive grid sampling across position ($\mathbb{R}^3$) and orientation (SO(3)) requires exponentially many samples as resolution increases. Modern approaches decouple *proposal* (generating candidate grasps via geometric heuristics that prune the search space) from *scoring* (evaluating each candidate with a learned model that captures physics implicitly).

Grasp Pose Detection (GPD) by ten Pas et al. (2017) generates candidates by sampling points on the observed surface, estimating local principal curvatures, and aligning a hand geometry model to regions of sufficient concavity. Each candidate is represented as a multi-view projection — three orthographic depth images rendered from orthogonal viewpoints around the grasp — which are classified as graspable by a shallow CNN.

PointNetGPD (Liang et al., 2019) replaces the multi-view rendering with PointNet (Qi et al., 2017), a permutation-invariant network that directly consumes unordered point sets. The network applies a shared MLP $h : \mathbb{R}^3 \to \mathbb{R}^D$ to each point independently, aggregates via channel-wise max-pooling to enforce permutation invariance, and maps to a scalar score via a decoder γ:

$$f(\mathcal{P}_{\text{local}}) = \gamma\left(\max_{i=1,\ldots,N} h(\mathbf{p}_i)\right). \tag{6.17}$$

The local point cloud around each grasp candidate is transformed into the gripper frame via $\mathbf{T}_{\mathbf{g}}^{-1}$ before being fed to PointNet, providing approximate invariance to the grasp orientation. This eliminates the need for multi-view rendering and allows the network to learn the geometric signatures of stable grasps directly from 3D data.

AnyGrasp (Fang et al., 2023) pushes toward the goal of grasping any object at any location on its surface. It combines geometric antipodal constraints with a learned scoring function trained on a large-scale dataset of simulated grasps spanning diverse object categories. The key architectural element is orientation-aware point feature extraction: before point pooling, features are enriched with the approach direction and grasp orientation, ensuring the network conditions on the 6-DoF grasp parameters.

6.3.2 Contact-GraspNet and Implicit Grasp Representations

An alternative to sampling-then-scoring is to predict grasps *directly* as a function of the scene geometry. Contact-GraspNet (Sundermeyer et al., 2021) predicts per-point grasp proposals: for each point in a scene point cloud, the network outputs a 6-DoF grasp pose (if any), enabling dense sliding-window-free grasp detection.

The representation is implicit: rather than outputting an explicit grasp parameter vector, the network outputs a contact point on the object surface and a grasp direction in the local frame, from which the full 6-DoF pose is reconstructed geometrically. The loss supervises the distance between predicted and ground-truth grasp poses:

$$\mathcal{L}_{\text{grasp}} = \sum_{p \in \mathcal{P}^+} \|\mathbf{g}_{\text{pred}}(p) - \mathbf{g}_{\text{gt}}(p)\|^2_{\text{SE(3)}}, \tag{6.18}$$

where $\mathcal{P}^+$ are points that belong to valid grasp regions, and $\|\cdot\|_{\text{SE(3)}}$ is a weighted combination of translation and rotation errors. This per-point prediction paradigm aligns with the broader trend in 3D deep learning toward dense prediction and away from proposal-based pipelines.

6.3.3 Task-Oriented and Semantic Grasping

Grasping is not merely about stability — it is about *function*. A spoon grasped by its bowl is stable but useless for eating; grasped by its handle, it can be used. *Task-oriented grasping* incorporates the downstream purpose into grasp selection:

$$\mathbf{g}^* = \arg\max_{\mathbf{g} \in \mathcal{G}} \Big[\alpha\, Q_{\text{stability}}(\mathbf{g}, \mathcal{P}) + (1-\alpha)\, Q_{\text{task}}(\mathbf{g} \mid \text{task}, \ell)\Big], \tag{6.19}$$

where $Q_{\text{stability}}$ is a conventional quality metric and Q_{task} scores the grasp's suitability for the intended manipulation. The task may be specified by a language description ℓ (e.g., "grasp the mug by the handle to pour"), a demonstration, or a predefined task taxonomy.

Semantic grasping uses object part recognition to bias grasps toward functional regions. Affordance segmentation networks (Do et al., 2018) predict per-point or per-pixel labels — *graspable*, *cuttable*, *scoopable*, *pourable* — providing a task-conditioned prior over grasp locations. The affordance map $A(\mathbf{x} \mid \text{task})$ encodes the probability that an action at $\mathbf{x}$ will succeed:

$$A(\mathbf{x} \mid \text{task}) = p(\text{success} \mid \mathbf{x}, \text{task}, \mathcal{P}). \tag{6.20}$$

This connects task-oriented grasping to language-conditioned manipulation: the language instruction provides the task context, semantic perception identifies the relevant object parts, and geometric grasp synthesis finds a stable configuration on those parts.

6.3.4 The Complete Grasping Pipeline

A complete robotic grasping system integrates several subsystems in a real-time loop, with the end-to-end latency determining whether the system can react to moving objects and recover from failures:

1. **Perception:** A depth camera produces a point cloud $\mathcal{P}$. Segmentation (learned or geometric) isolates the target object from the background. For cluttered scenes, instance segmentation networks (Mask R-CNN, SAM) separate individual objects.

2. **Grasp detection:** Candidate grasps $\{\mathbf{g}_i\}_{i=1}^{N}$ are proposed (via sampling or dense prediction) and scored by a learned quality function. The highest-quality grasp that is collision-free and kinematically reachable is selected:

$$\mathbf{g}^* = \arg\max_{\mathbf{g}\in\mathcal{G}} \; Q(\mathbf{g}, \mathcal{P}) \quad \text{s.t.} \quad \text{collision-free}(\mathbf{g}, \mathcal{P}) \wedge \text{reachable}(\mathbf{g}, \mathbf{q}_{\text{current}}). \tag{6.21}$$

3. **Motion planning:** A collision-free arm trajectory $\mathbf{q}(t)$ from the current joint configuration $\mathbf{q}_{\text{start}}$ to a pre-grasp configuration $\mathbf{q}_{\text{pre-grasp}}$ (offset 10–15 cm along the approach vector) and then to the final grasp configuration $\mathbf{q}_{\text{grasp}}$ is planned. Sampling-based planners (RRT*, PRM*) or trajectory optimizers (CHOMP, TrajOpt, STOMP) are commonly used.

4. **Execution:** The arm follows the planned trajectory. At the grasp pose, the gripper closes until force feedback or motor current exceeds a threshold indicating firm contact. Janaflex or soft fingertips provide compliance that absorbs residual pose error.

5. **Verification:** After lifting the object (a short vertical motion), force/torque sensors or visual tracking confirm that the object is stably held. If verification fails — the object slips or doesn't move — the system replans.

6.4 Connections

Grasping and SLAM: Accurate grasping requires knowing the object's pose in the robot's base frame. This depends on reliable hand-eye calibration and, for mobile manipulators, on SLAM-estimated robot pose. A 5cm error in robot localization translates directly to a 5cm offset at the end-effector, potentially causing grasp failure. Joint optimization of localization and manipulation — closing the loop between mapping and acting — is an emerging research direction.

Grasping and reinforcement learning: RL-based grasping (QT-Opt by Kalashnikov et al., 2018) trains vision-based closed-loop policies from hundreds of thousands of real-world grasp attempts. The policy $\pi(a \mid \mathbf{o}, \mathbf{s})$ maps visual observations and robot state to continuous action commands, trained via off-policy Q-learning with a sparse reward (1 for successful lift, 0 otherwise). QT-Opt's 580,000 grasp attempts over several months across 7 robots underscores both the data appetite of RL and its potential: after sufficient data, the policy outperforms scripted and heuristic baselines.

Dexterous hands: The parallel-jaw abstraction, while dominant in practice, is severely limited. Multi-fingered dexterous hands enable in-hand manipulation — rotating, sliding, reorienting an object without regrasping — but the higher-dimensional action space ($\sim$20 DoF vs. 1 DoF for a parallel jaw) and complex contact dynamics make control exponentially harder. Dexterous grasping merges grasping with continuous manipulation, currently among the hardest open problems in physical robot control.

Grasping as generative modeling: Recent work reframes grasp synthesis as conditional generative modeling. Diffusion policies (Chi et al., 2023) or flow-matching models learn the full

distribution $p(\mathbf{g} \mid \mathcal{P})$ of successful grasps, enabling diverse sampling that captures the multi-modality intrinsic to grasping. The generative perspective connects grasping to the broader AI toolkit: denoising diffusion, normalizing flows, and energy-based models.

6.5 Open Problems

1. **Generalization beyond rigid objects:** Current systems excel on rigid, convex objects with distinct geometry. Deformable objects (cloth, cables, food), transparent (glass), and specular objects (metal) violate the surface assumptions of depth sensors and geometric grasp metrics. Non-rigid grasp simulation and multimodal perception (combining depth, tactile, and proprioceptive sensing) are essential next steps.

2. **Dexterous in-hand manipulation:** Transitioning from parallel-jaw grasping to dexterous manipulation introduces a combinatorial explosion in contact modes — sliding, rolling, breaking, and making contact. The mathematical frameworks (hybrid dynamical systems, contact-implicit trajectory optimization) exist but lack the robustness and speed for real-time control on physical hardware.

3. **Sample-efficient grasp learning:** While QT-Opt required hundreds of thousands of real-world trials, humans learn to grasp novel objects from a handful of observations. Sample-efficient grasp learning — via meta-learning for rapid adaptation, strong inductive biases from analytic physics, or few-shot imitation — remains a foundational challenge.

4. **Commonsense and task-oriented grasping:** Current systems succeed at "grasp this object" but fail at "grasp this object by the handle." Integrating commonsense knowledge about object function (from vision-language models or knowledge graphs) with geometric grasp synthesis requires bridging language, vision, and physics in a unified framework.

5. **Grasping in dense clutter:** In cluttered scenes, objects occlude one another and are physically constrained by neighbors. Singulation — isolating one object — often requires non-prehensile actions (pushing, nudging, toppling). Combined prehensile and non-prehensile planning in clutter has combinatorial complexity resistant to efficient solutions.

6. **Human-robot handover:** When a robot hands an object to a human, the grasp must be stable, the trajectory predictable, and the release timing responsive to human cues (eye gaze, hand approach, verbal signal). This involves social perception, intention prediction, and compliant control — a rich multi-agent interaction problem.

7. **Tactile feedback for grasp adjustment:** Vision-only grasping is fundamentally limited by occlusions and ambiguous geometry at contact points. Integrating tactile sensing — high-resolution tactile arrays (GelSight, DIGIT) — for real-time grasp adjustment and slip detection promises to close the perception-action loop at the crucial moment of contact.

Chapter 7

Language-Conditioned Manipulation

7.1 Motivation

Natural language is the most expressive and intuitive interface humans have for communicating intent. An instruction such as *"pick up the red block and place it on the blue plate"* conveys a multi-step manipulation goal with remarkable economy — specifying objects by their visual properties (color), actions by their expected outcomes (transporting an object from one location to another), and spatial relations between entities (on top of). Building robots that can interpret and faithfully execute such instructions requires solving the *language grounding* problem: mapping discrete symbolic linguistic expressions to continuous perceptual representations and ultimately to motor commands.

The challenge is multifaceted. Language is compositional: novel instructions are formed by recombining known words in new ways. Language is referential: words point to entities in the physical world that the robot must identify from sensory data. Language is intentional: instructions encode *goals*, not trajectories — it is the robot's job to determine *how* to achieve them. And language is situated: the same instruction means different things in different contexts ("pick it up" depends on what "it" refers to in the current scene). This chapter examines the spectrum of approaches to language-conditioned manipulation, from end-to-end visuomotor policies conditioned on text embeddings to compositional systems that leverage large language models for semantic planning.

7.2 Core Concepts and Key Papers

7.2.1 CLIPort: Language-Grounded Transporter Networks

CLIPort (Shridhar et al., 2022) elegantly combines two independently powerful ideas: CLIP's (Radford et al., 2021) vision-language representations for semantic grounding, and Transporter Networks' (Zeng et al., 2021) spatial action prediction for efficient manipulation learning. The result is a system that can execute language-specified pick-and-place tasks with strong compositional generalization.

Transporter Networks address tabletop rearrangement through visual correspondence. Given a current observation $\mathbf{o}_t$ and a goal observation $\mathbf{o}_g$ (showing the desired final arrangement), two ResNet-based encoder streams compute dense feature maps:

$$\mathbf{\Phi}_{\text{pick}} = f_{\text{pick}}(\mathbf{o}_t, \mathbf{o}_g) \in \mathbb{R}^{H \times W \times D}, \qquad \mathbf{\Phi}_{\text{place}} = f_{\text{place}}(\mathbf{o}_t, \mathbf{o}_g) \in \mathbb{R}^{H \times W \times D}. \tag{7.1}$$

The pick heatmap is produced by a small convolutional head applied per spatial location:

$$\mathcal{H}_{\text{pick}}(u, v) = \sigma\big(h_{\text{pick}}(\mathbf{\Phi}_{\text{pick}}(u, v))\big) \in [0, 1], \tag{7.2}$$

and the pick location is $(u^*, v^*) = \arg\max_{u,v} \mathcal{H}_{\text{pick}}(u, v)$. For the place prediction, a spatial crop of the place feature map around (u^*, v^*) is correlated with the full map via cross-correlation:

$$\mathcal{H}_{\text{place}} = \big(\mathbf{\Phi}_{\text{pick}}(u^*, v^*)\big) \star \mathbf{\Phi}_{\text{place}}, \tag{7.3}$$

where $\star$ denotes cross-correlation across the spatial dimensions. The place location is the argmax of $\mathcal{H}_{\text{place}}$. This cross-correlation mechanism naturally captures the spatial displacement

between pick and place. Moreover, Transporter Networks exploit rotational equivariance: rotating the input image by θ and cross-correlating with rotated feature maps is equivalent to rotating the place prediction by θ, giving the model an architectural inductive bias that dramatically improves sample efficiency for tabletop tasks.

CLIPort replaces the goal image $\mathbf{o}_g$ with a CLIP-encoded natural language instruction. The instruction ℓ is tokenized and encoded through CLIP's text transformer:

$$\mathbf{e}_\ell = \text{CLIP}_{\text{text}}(\ell) \in \mathbb{R}^{D_{\text{CLIP}}}. \tag{7.4}$$

The fusion of language and vision happens through *pixel-wise dot-product attention*. The visual feature map is projected into a shared space, and the language embedding modulates each spatial location via multiplicative interaction:

$$\mathbf{\Phi}_{\text{fused}}(u, v) = \mathbf{\Phi}_{\text{vis}}(u, v) \odot \big(\mathbf{W}_{\text{fuse}}\, \mathbf{e}_\ell\big), \tag{7.5}$$

where $\mathbf{W}_{\text{fuse}} \in \mathbb{R}^{D \times D_{\text{CLIP}}}$ projects the language embedding to the visual feature dimension and $\odot$ denotes Hadamard (element-wise) multiplication. The fused feature map selectively enhances regions corresponding to the referenced object — the dot product acts as a learned similarity measure between the visual appearance at each pixel and the linguistic description of the target.

The training objective is behavioral cloning from expert demonstrations — a dataset $\{(\mathbf{o}_t^{(i)}, \ell^{(i)}, a^{(i)})\}$ of observation-instruction-action triplets, where each action is a pick-and-place pair $a = ((u_{\text{pick}}, v_{\text{pick}}), (u_{\text{place}}, v_{\text{pla}}$ The loss combines binary cross-entropy terms for pick and place heatmaps:

$$\mathcal{L} = \text{BCE}(\mathcal{H}_{\text{pick}}, \mathcal{H}^{\text{gt}}_{\text{pick}}) + \lambda\, \text{BCE}(\mathcal{H}_{\text{place}}, \mathcal{H}^{\text{gt}}_{\text{place}}). \tag{7.6}$$

Crucially, CLIP's pre-trained representations enable compositional generalization: if the demonstrations include "pick the red block" and "place on the blue plate" separately, the model can generalize to "pick the red block and place it on the blue plate" because CLIP's embedding space encodes the semantic compositionality learned from 400M image-text pairs.

7.2.2 SayCan: Grounding Language in Affordances

While CLIPort directly maps language to actions in a single feedforward pass, SayCan (Ahn et al., 2022) introduces a compositional architecture that separates *semantic planning* (what to do) from *physical feasibility* (whether it can be done). This decomposition addresses a fundamental limitation of using LLMs directly for robot control: LLMs possess impressive semantic knowledge but lack embodied experience — they may confidently propose physically impossible actions in the current environment.

The robot's capability is defined by a repertoire of K *skills* $\{\pi_k\}_{k=1}^{K}$, each a parameterized policy for a primitive action: `pick(obj)`, `place(receptacle)`, `open(drawer)`, `navigate(location)`, etc. Each skill π_k maps from the current observation (which may encode state history) to a sequence of low-level motor commands.

For each skill and observation context, a learned *affordance function* (a value function) estimates the probability of successful execution:

$$V_{\pi_k}(s) = p(\text{success} \mid s, \pi_k). \tag{7.7}$$

These value functions are trained on the robot's historical experience data — a dataset of (s, π_k, r) tuples where $r \in \{0, 1\}$ indicates whether the skill succeeded — using a combination of temporal-difference learning and behavioral cloning. The state s encodes not just the current sensor reading but a history of past observations and actions, forming an *episodic context* that helps the value function disambiguate situations (e.g., a fridge observation means different things depending on whether the robot has already retrieved the item it needs).

The LLM is prompted with the natural language instruction ℓ together with the sequence of already-executed skills. It outputs a log-probability for each candidate next skill, representing the LLM's estimate of how semantically appropriate that skill is given the instruction and execution history:

$$P_{\text{LLM}}(\pi_k \mid \ell, \pi_{1:t-1}) \propto \exp\big(\text{logit}_{\text{LLM}}(\pi_k \mid \text{prompt}(\ell, \pi_{1:t-1}))\big). \tag{7.8}$$

The key innovation is the *combined scoring* that multiplicatively integrates semantic and physical estimates:

$$\pi^* = \arg\max_{\pi_k} \Big[\log P_{\text{LLM}}(\pi_k \mid \ell, \pi_{1:t-1}) + \log V_{\pi_k}(s)\Big]. \tag{7.9}$$

In probability space, this is the product of two numbers — the LLM's semantic score and the affordance function's feasibility score:

$$\text{score}(\pi_k) = P_{\text{LLM}}(\pi_k \mid \ell, \pi_{1:t-1}) \cdot V_{\pi_k}(s). \tag{7.10}$$

If either factor is near zero, the skill is rejected. This mechanism elegantly handles impossible requests: if the LLM proposes "pick up the apple" but no apple is visible, $V_{\text{pick(apple)}}(s) \approx 0$, and the system gracefully reports that the action cannot be executed rather than attempting it and failing. Deployed on real kitchen mobile manipulators, SayCan achieved 84% success on long-horizon instructions like *"bring me a coke from the fridge"* and *"clean up the spilled chips."*

7.2.3 RT-1: Robotics Transformer

RT-1 (Brohan et al., 2023) embodies the scaling hypothesis: with sufficient diverse, multi-task demonstration data, a single large model can perform many manipulation tasks from free-form language instructions without task-specific engineering. The emphasis is on data scale and diversity rather than architectural sophistication.

RT-1 processes a short history of images (6 RGB frames at 3 Hz, concatenated channel-wise) together with a natural language instruction, and outputs a sequence of *discretized* action tokens: 7-DoF arm movement (end-effector 3D position, 3D orientation as Euler angles, and gripper opening) each quantized into 256 bins, plus a discrete base motion and termination command.

The architecture proceeds through three stages:

Stage 1 — FiLM-conditioned visual encoding: Each image frame is processed by an EfficientNet-B3 backbone pretrained on ImageNet-21k. The language instruction is embedded via a pretrained Universal Sentence Encoder (USE) to a 512-dimensional vector $\mathbf{e}_\ell$. Every convolutional layer in EfficientNet is modulated by FiLM (Feature-wise Linear Modulation), which applies an affine transformation conditioned on language:

$$\text{FiLM}(\mathbf{x} \mid \mathbf{e}_\ell) = \gamma(\mathbf{e}_\ell) \odot \mathbf{x} + \beta(\mathbf{e}_\ell), \tag{7.11}$$

where $\gamma : \mathbb{R}^{512} \to \mathbb{R}^C$ and $\beta : \mathbb{R}^{512} \to \mathbb{R}^C$ are learned MLPs that produce channel-wise scale and shift parameters. This per-layer language conditioning enables the visual pathway to selectively attend to instruction-relevant visual features at every scale of abstraction.

Stage 2 — TokenLearner compression: The dense visual feature grid $\mathbf{F} \in \mathbb{R}^{H \times W \times D}$ from the visual backbone is adaptively compressed into a small fixed set of $T = 8$ learned tokens using TokenLearner (Ryoo et al., 2021). TokenLearner computes a spatial attention map and aggregates:

$$\boldsymbol{\tau}_t = \sum_{u,v} \alpha_t(u,v) \cdot \mathbf{F}(u,v), \qquad \alpha_t(u,v) = \text{softmax}_{u,v}\big(\text{MLP}_t(\mathbf{F}(u,v))\big). \tag{7.12}$$

Each of the T heads learns to attend to different spatial regions, creating a compact bottleneck that dramatically reduces Transformer sequence length.

Stage 3 — Transformer policy: The 8 visual tokens, a CLS token, and a sinusoidal position encoding are concatenated into a sequence and processed by a decoder-only Transformer (8 layers, self-attention). The output CLS token is projected to action logits. Actions are predicted auto-regressively:

$$p(a_t \mid \mathbf{o}_{<t}, \ell) = \prod_{d=1}^{7} p(a_t^{(d)} \mid a_t^{(<d)}, \mathbf{o}_{<t}, \ell), \tag{7.13}$$

where each action dimension is predicted sequentially, allowing the model to capture dependencies among action components.

RT-1 was trained on 130,000 real robot demonstrations collected across 700+ distinct tasks from a fleet of 13 robots over 17 months. The training data includes diverse instructions ("pick up the green can," "move the sponge to the left") and diverse object instances, lighting conditions, and backgrounds. The principal finding is that *data diversity trumps architectural sophistication*: a straightforward Transformer policy trained on massive multi-task data outperforms more sophisticated architectures trained on narrow, task-specific datasets. The paper demonstrates 97% success on seen tasks and 76% on unseen tasks, with strong generalization to novel objects, backgrounds, and environments.

7.2.4 InstructRL: LLM-Generated Reward Functions

An orthogonal approach uses LLMs not as planners or action predictors but as *reward engineers*. InstructRL (Yu et al., 2023; Ma et al., 2023) prompts an LLM with the task description and the robot's observation and action specification, and the LLM generates executable Python code implementing a reward function:

$$\text{prompt}(\ell, \text{obs_spec}, \text{act_spec}) \xrightarrow{\text{LLM}} \texttt{def reward(obs, action) -> float:...} \tag{7.14}$$

This generated reward function is then used to train a reinforcement learning policy via standard algorithms (SAC, PPO):

$$\pi_\ell^* = \arg\max_\pi \; \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \, R_\ell(s_t, a_t) \right]. \tag{7.15}$$

The approach automates reward engineering — traditionally a labor-intensive, domain-expert-dependent, and brittle component of RL system design. The LLM brings commonsense knowledge to bear: given "the robot should place the red block on the blue plate," it can generate code that checks object positions, computes distances, and provides shaping rewards for intermediate progress (approaching the block, lifting it, moving toward the plate).

In practice, the generated rewards are refined through an iterative loop: the LLM observes execution traces (simulation rollouts with the current reward) and refines the reward code, much like a human reward engineer iterating on the shaping terms. The approach enables rapid repurposing of the same robotic system for new tasks without manual reward design — bridging language, code generation, and reinforcement learning into a unified pipeline.

7.2.5 RT-2: Vision-Language-Action Models

RT-2 (Brohan et al., 2023) co-trains a large vision-language model (PaLI-X or PaLM-E) on both internet-scale vision-language data and robot demonstration data, producing a *vision-language-action* (VLA) model that directly outputs robot actions. The key insight is that internet-scale

pre-training provides rich visual and semantic representations that transfer to robotic control with minimal fine-tuning.

The model is trained to output text tokens — but the action space is *tokenized* into the model's vocabulary: continuous action values are discretized into bins assigned to special tokens (e.g., "<action_0_128>" for the midpoint of the first action dimension). At inference time, the model takes an image and a language instruction as input and generates a sequence of action tokens, which are decoded to joint commands.

The co-training loss combines next-token prediction on both web data and robot data:

$$\mathcal{L} = \mathbb{E}_{(\mathbf{x},\mathbf{y})\sim\mathcal{D}_{\text{web}}}\big[-\log p(\mathbf{y} \mid \mathbf{x})\big] + \lambda\, \mathbb{E}_{(\mathbf{o},\ell,a)\sim\mathcal{D}_{\text{robot}}}\big[-\log p(a \mid \mathbf{o}, \ell)\big]. \tag{7.16}$$

The key empirical finding is that co-training *does not* degrade performance on either domain — the model learns to switch between modes based on context — and that the web-scale pre-training substantially improves generalization to novel objects, scenes, and instructions not seen in the robot data.

7.3 Key Techniques

7.3.1 Language Grounding via Multimodal Fusion

The core technical problem in language-conditioned manipulation is fusing discrete symbolic information (language) with continuous perceptual representations (images, point clouds, robot proprioception). Several fusion paradigms have proven effective, representing points on a spectrum from early to late integration:

Early fusion (FiLM): Language modulates visual processing at every layer through affine transformations, as in RT-1. This allows language to gate visual features early, suppressing task-irrelevant information (e.g., color when the instruction references shape) and amplifying task-relevant features.

Mid-level fusion (cross-attention): Language and visual features interact through cross-attention layers, where queries from one modality attend to keys and values from the other. Multi-head cross-attention enables the model to learn multiple types of cross-modal relationships:

$$\mathbf{F}_{\text{fused}} = \text{Concat}\big(\text{head}_1, \ldots, \text{head}_H\big)\mathbf{W}^O, \qquad \text{head}_i = \text{Attention}(\mathbf{Q}_i\mathbf{F}_{\text{lang}}, \mathbf{K}_i\mathbf{F}_{\text{vis}}, \mathbf{V}_i\mathbf{F}_{\text{vis}}). \tag{7.17}$$

Late fusion (dot product): As in CLIPort, both modalities are independently encoded to a shared dimension and combined via inner product at the output level. This is computationally efficient but assumes the modalities can be aligned in a shared embedding space — a reasonable assumption when using a pre-trained multimodal encoder like CLIP.

Multiplicative interaction: A general bilinear form of multimodal fusion computes:

$$f(\mathbf{x}_{\text{vis}}, \mathbf{x}_{\text{lang}}) = \sum_{i,j,k} w_{ijk}\, x_{\text{vis}}^{(i)}\, x_{\text{lang}}^{(j)}\, z^{(k)}, \tag{7.18}$$

where $\mathbf{z}$ is an optional latent factor. Since the full tensor $\mathbf{W}$ has $O(D^3)$ parameters, low-rank approximations (MUTAN, MLB, MFB) factor $\mathbf{W}$ into smaller matrices, trading expressiveness for parameter efficiency.

7.3.2 Compositional Generalization

A key property for language-conditioned manipulation is *compositional generalization*: executing novel combinations of known concepts. If the robot has learned to "pick up the red cube" and

"place on the blue plate," it should succeed at "pick up the green cube and place on the red plate" without explicit training on that exact instruction.

CLIPort achieves this through CLIP's pre-trained representations, which encode visual and textual concepts in a shared metric space where semantic compositionality emerges from contrastive pre-training on internet-scale data. SayCan achieves it through the LLM, which has learned compositional linguistic reasoning from text alone. Formally, compositional generalization requires that the policy's distribution factorizes approximately over the instruction's constituents:

$$p(a \mid \text{action}, \text{obj}_1, \text{rel}, \text{obj}_2) \approx f\big(g(\text{action}, \text{obj}_1),\ h(\text{rel}, \text{obj}_2)\big), \tag{7.19}$$

for some factorization into sub-policies g and h that can be recombined for novel instructions. End-to-end networks that achieve this factorization *implicitly* — through shared parameters and multi-task training — remain an active research frontier.

7.3.3 Behavioral Cloning and the Compounding Error Problem

Most language-conditioned manipulation systems are trained via behavioral cloning (BC): minimizing the negative log-likelihood of expert actions on a demonstration dataset:

$$\mathcal{L}_{\text{BC}} = -\mathbb{E}_{(\mathbf{o},\ell,a)\sim\mathcal{D}}\big[\log \pi_\theta(a \mid \mathbf{o}, \ell)\big]. \tag{7.20}$$

BC is simple, stable, and efficient, but suffers from *covariate shift*: during execution, small prediction errors accumulate, driving the policy into states not represented in the training data, where errors compound further. DAGGER (Ross et al., 2011) mitigates this by iteratively collecting on-policy data with expert corrections, but requires an interactive expert.

Action chunking — predicting a sequence of H future actions rather than a single action — provides temporal smoothing that partially mitigates compounding errors. The policy outputs:

$$\pi(a_t, a_{t+1}, \ldots, a_{t+H-1} \mid \mathbf{o}_t, \ell), \tag{7.21}$$

and the robot executes the first $k < H$ actions before replanning, creating an implicit model-predictive control loop. This approach, used in ACT (Zhao et al., 2023) and adopted by RT-1, trades off reactivity against temporal consistency.

7.4 Connections

Language-conditioned manipulation and visual grounding: Both require linking linguistic expressions to visual entities. Techniques from visual grounding — detection-based (Faster R-CNN + language-conditioned attention), attention-based (MDETR, GLIP), and segmentation-based (referring expression segmentation) — transfer directly to manipulation. The critical difference: manipulation requires an *action* as output, not merely a bounding box or mask.

LLMs as world models: LLMs encode substantial commonsense knowledge about physical interactions ("you need to open the fridge before taking something out," "a knife should be grasped by the handle, not the blade"). Whether this knowledge can substitute for, augment, or bootstrap learned world models in robotic planning is an open empirical question. SayCan's decomposition — LLM for semantics, learned models for physics — represents one answer; RT-2's co-training represents another.

Multilingual and culturally aware manipulation: Current systems operate almost exclusively in English. Extending language-conditioned manipulation to low-resource languages and culturally specific tasks (different utensils, different grasping conventions across cultures) connects robotics to computational linguistics and cultural computing, with significant implications for global deployability.

Instruction following and intent inference: Natural instructions are often underspecified ("get me something to write with" could be a pen, pencil, or marker). Handling ambiguity requires the robot to infer latent intent, ask clarifying questions, or produce a distribution over interpretations. This connects language-conditioned manipulation to Bayesian theory of mind and Gricean pragmatics.

7.5 Open Problems

1. **Grounding fine-grained spatial relations:** Instructions like "place it to the left of the plate, behind the cup" require understanding spatial frames of reference (egocentric vs. allocentric, relative vs. absolute), handling occlusions, and resolving referential ambiguity. Current systems often fail on fine-grained spatial reasoning.

2. **Temporal language:** Instructions like "stir until it thickens, then pour" encode temporal structure — sequencing, preconditions, duration, termination conditions. Parsing and executing temporally extended language requires integrating linguistic temporal reasoning with real-time multimodal sensor feedback and continuous state monitoring.

3. **Ambiguity resolution via dialogue:** When an instruction is ambiguous (multiple red blocks visible, unclear destination), the robot should ask for clarification rather than guess. Building dialogue-capable manipulation systems that proactively resolve ambiguity through natural back-and-forth interaction is largely unexplored.

4. **Cross-embodiment generalization:** Language instructions are embodiment-agnostic ("pick up the cup" applies to parallel-jaw grippers, suction cups, dexterous hands). Can a single language-conditioned policy generalize across embodiments, or must each morphology learn its own language-to-action mapping from scratch?

5. **Safety and instruction alignment:** Language-conditioned policies can be prompted with dangerous instructions. Ensuring that manipulation respects safety constraints — through constitutional training, formal verification of language-conditioned plans, or learned safety critics — is essential for deployment in shared human environments.

6. **Zero-shot skill acquisition from language:** The frontier goal is a robot that learns new manipulation skills from language descriptions alone, without demonstrations. This requires combining language as a semantic prior with autonomous exploration and reinforcement learning for physical skill acquisition — few-shot or zero-shot skill learning via language that remains beyond current capabilities.

7. **Causal understanding in language-conditioned manipulation:** Language often encodes causal relationships ("if the door is closed, open it first"). End-to-end policies lack explicit causal models, making them brittle to distribution shift. Integrating causal reasoning — structural causal models learned from interaction data — with language-conditioned manipulation is a nascent but critical direction.

Chapter 8

Long-Horizon Task Completion

8.1 Motivation

The tasks that matter in the real world are not atomic. *"Clean the kitchen"* decomposes into dozens of sub-goals spanning navigation, perception, and manipulation — find the sponge, wet the sponge, wipe each counter surface, dispose of trash, sweep the floor — executed across multiple rooms over several minutes. The challenge of long-horizon task completion is not primarily the difficulty of any single action, but rather the combinatorial explosion of possible action sequences, the compounding of errors over extended time horizons, and the necessity of robust error recovery when intermediate steps inevitably fail.

Long-horizon tasks expose the limits of both classical and learned approaches. Classical task planning (STRIPS, PDDL) provides formal guarantees on plan correctness but requires hand-designed symbolic abstractions of continuous sensorimotor reality — a brittle and labor-intensive requirement. End-to-end learned policies can handle perceptual noise and physical variability but struggle with the credit assignment problem over hundreds of time steps, where the signal from a sparse terminal reward is washed out by the combinatorial depth of the action tree.

The emerging consensus points toward hybrid architectures: learned perception and low-level control for robustness to perceptual and physical variation, combined with structured planning — either symbolic (PDDL) or neural (LLM-generated) — for efficient exploration of the combinatorial action space. This chapter develops the mathematical foundations of task and motion planning, surveys end-to-end and modular approaches to long-horizon manipulation, and examines the skill chaining and hierarchical reinforcement learning frameworks that enable temporal abstraction.

8.2 Core Concepts and Key Papers

8.2.1 The ALFRED Benchmark and the Anatomy of Difficulty

ALFRED (Shridhar et al., 2020) crystallized the long-horizon embodied task challenge in a standardized, reproducible benchmark. Agents operate in AI2-THOR simulated indoor environments — kitchens, living rooms, bedrooms, bathrooms — and must execute natural language instructions drawn from a diverse set of household tasks.

A prototypical ALFRED instruction is: *"Pick up the heated apple and place it in the fridge."* Executing this requires: (1) navigating from the starting location to a room containing an apple, (2) searching for and visually identifying the apple among distractors, (3) picking it up, (4) navigating to a room with a microwave, (5) opening the microwave, (6) placing the apple inside, (7) closing the microwave and activating it, (8) waiting, (9) retrieving the heated apple, (10) navigating to a room with a fridge, (11) opening the fridge, (12) placing the apple inside, (13) closing the fridge. This is a sequence of 13+ atomic actions spanning navigation, object search, and 7 distinct manipulation types (pick, place, open, close, slice, clean, toggle on/off).

The benchmark comprises 25,000 language instructions spanning 120+ object types across 7 action types, rendered in 120 distinct indoor scenes. Evaluation is along two complementary axes:

$$\text{Task Success Rate (TSR)} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\text{all sub-goals completed for task } i], \tag{8.1}$$

$$\text{Goal-Condition Success (GCS)} = \frac{1}{N}\sum_{i=1}^{N} \frac{\#\text{sub-goals completed}_i}{\#\text{total sub-goals}_i}. \tag{8.2}$$

TSR is unforgiving: a single failed sub-goal results in task failure, even if 12 of 13 steps succeeded. GCS is more forgiving, measuring partial progress and thus providing a finer-grained signal for methods that make partial headway. The benchmark also reports per-action-type success rates, revealing which manipulations are hardest: cleaning objects (requiring finding a sink + holding the object + executing the correct action) and slicing objects (requiring finding a knife + positioning) consistently have the lowest success rates, exposing the limits of current methods.

8.2.2 Task and Motion Planning: The Classical Synthesis

Task and Motion Planning (TAMP) represents the classical AI and robotics approach to long-horizon reasoning. It marries *task planning* over discrete symbolic state representations with *motion planning* over continuous configuration spaces, unified through *semantic attachments* — geometric feasibility predicates that ground symbolic operators in continuous geometry.

The task planning layer operates in a STRIPS-like (Stanford Research Institute Problem Solver) representation. A planning domain is defined by:

- A set of predicates $\mathcal{P}$ — boolean functions of the continuous world state (e.g., `at(robot, kitchen)`, `holding(robot, apple)`, `open(fridge)`).
- A set of operators $\mathcal{O}$, each a parameterized action schema.
- An initial state $s_0 \subseteq \mathcal{P}$ (all true predicates at the start).
- A goal specification $g \subseteq \mathcal{P}$ (predicates that must be true at termination).

Each operator $o \in \mathcal{O}$ is a triple:

$$o = \langle \text{Pre}(o), \text{Eff}^+(o), \text{Eff}^-(o) \rangle, \tag{8.3}$$

where $\text{Pre}(o)$ is a conjunction of predicates that must hold for o to be applicable, $\text{Eff}^+(o)$ is the set of predicates that become true (add effects), and $\text{Eff}^-(o)$ is the set that become false (delete effects). For example:

$$\text{Pre}(\texttt{go_to_kitchen}) = \{\texttt{at(robot, living_room)}\}, \tag{8.4}$$

$$\text{Eff}^+(\texttt{go_to_kitchen}) = \{\texttt{at(robot, kitchen)}\}, \tag{8.5}$$

$$\text{Eff}^-(\texttt{go_to_kitchen}) = \{\texttt{at(robot, living_room)}\}. \tag{8.6}$$

A task plan is a sequence of grounded operators $\pi_{\text{symb}} = (o_1, o_2, \ldots, o_T)$ that transforms s_0 to a state satisfying g. The symbolic planner (e.g., Fast Downward using heuristic search with h^{FF} or $h^{\text{LM-Cut}}$) finds such a sequence.

The *semantic attachment* (also called a *refinement* or *grounding*) maps each symbolic operator to a continuous motion planning problem. For instance, the operator `go_to_kitchen` requires a collision-free arm trajectory from the current configuration to a configuration satisfying the kinematic constraint of being in the kitchen region:

$$\exists \mathbf{q}(t) \in \mathcal{C}_{\text{free}},\ t \in [0,1] \quad \text{s.t.} \quad \mathbf{q}(0) = \mathbf{q}_{\text{current}},\ \ \text{FK}(\mathbf{q}(1)) \in \mathcal{R}_{\text{kitchen}}, \tag{8.7}$$

where $\mathcal{C}_{\text{free}} \subset \mathbb{R}^d$ is the collision-free configuration space and $\text{FK} : \mathbb{R}^d \to \text{SE}(3)$ is the forward kinematics. For manipulation operators like `pick(apple)`, the motion planning problem additionally requires an inverse-kinematics solution for a grasp pose and a grasp execution trajectory.

The TAMP algorithm interleaves symbolic search with geometric verification in a generate-and-test loop:

1. The symbolic planner proposes a candidate plan π_{symb}.
2. For each operator $o_t \in \pi_{\text{symb}}$, the motion planner attempts to find a feasible trajectory.
3. If any motion planning sub-problem fails, the symbolic planner backtracks and tries an alternative plan.
4. If all motion plans succeed, the full hybrid plan is returned.

This architecture provides formal guarantees — if TAMP returns a plan, it is guaranteed to be geometrically feasible — but at a steep cost: the worst-case search complexity is the product of symbolic branching factor and the cost of geometric verification. In practice, the backtracking penalty can be severe, and heuristic guidance (biasing the symbolic planner away from geometrically difficult operators) is essential.

The fundamental limitation of TAMP is its reliance on hand-designed symbolic abstractions. Defining the predicate vocabulary, operator preconditions and effects, and semantic attachments requires painstaking domain engineering that does not transfer across environments or tasks. A kitchen TAMP cannot be dropped into a workshop without re-engineering the entire abstraction stack.

8.2.3 FILM: LLM-Guided Modular Execution

FILM (Following Instructions with Language Models; Min et al., 2022) advances a hybrid architecture that replaces the hand-designed symbolic planner with a frozen large language model (GPT-3 or PaLM) for semantic planning, while retaining learned neural modules for perception and low-level control. This preserves the modularity and interpretability of the classical pipeline while leveraging LLMs' commonsense reasoning to generate plans without per-domain engineering.

Plan generation: Given a natural language instruction ℓ, a prompted LLM translates it into a sequence of structured sub-goals composed from a predefined vocabulary of predicates and action types:

$$\begin{aligned} \ell = \text{“put the cleaned apple in the fridge”} \\ \xrightarrow{\text{LLM}}\ & [\texttt{Find(Apple)}, \texttt{Pick(Apple)}, \texttt{Find(Sink)}, \texttt{Clean(Apple)}, \\ & \texttt{Find(Fridge)}, \texttt{Open(Fridge)}, \texttt{Put(Apple, Fridge)}, \texttt{Close(Fridge)}]. \end{aligned} \tag{8.8}$$

The LLM is *not* trained or fine-tuned on robot data — it is a frozen, general-purpose language model prompted with in-context examples of instruction-to-plan mappings.

Execution modules: Three learned, reusable modules ground the symbolic sub-goals in perception and action:

Semantic map module f_{map}: Maintains a top-down allocentric semantic map that integrates egocentric RGB-D observations over time:

$$\mathbf{m}_{\text{sem}}^{(t)} = f_{\text{map}}\big(\mathbf{m}_{\text{sem}}^{(t-1)}, \mathbf{o}_t, \mathbf{p}_t\big), \tag{8.9}$$

where $\mathbf{o}_t$ is the egocentric visual observation and $\mathbf{p}_t$ the agent's pose. $\mathbf{m}_{\text{sem}}$ encodes per-cell probabilities for each object category c:

$$\mathbf{m}_{\text{sem}}(u, v, c) = p(\text{object } c \text{ present at cell } (u, v) \mid \mathbf{o}_{1:t}). \tag{8.10}$$

Semantic search module π_{search}: Given a target object category c, navigates the agent toward cells where $\mathbf{m}_{\text{sem}}(u, v, c)$ is highest. The search policy can be implemented as frontier-based exploration with semantic guidance or a learned navigation module trained via imitation or RL.

Manipulation module π_{manip}: Given an action type and target object (e.g., `Pick(Apple)`), executes the low-level motor commands from egocentric vision. This module is trained via behavioral cloning on task-specific manipulation demonstrations.

The critical empirical finding is an ablation comparing planning sources:

- **Template plans** (always the same fixed sequence regardless of instruction): 26% TSR.
- **LLM-generated plans** (FILM): 34% TSR.
- **Human-written plans** (expert-authored sequences): 42% TSR.

The gap between LLM and human planning — 34% vs 42% — highlights that while LLMs capture substantial commonsense structure, they still produce sub-optimal or physically inconsistent plans. The gap between template and LLM — 26% vs 34% — demonstrates the value of flexible, instruction-conditioned planning. Critically, the gap between human-written plans (42%) and perfect execution (100%) underscores that plan quality is only one factor; perception and low-level execution errors dominate the failure modes.

8.2.4 Episodic Transformer: End-to-End Memory-Based Policy

The Episodic Transformer (Pashevich et al., 2021) challenges the modular architecture by training an end-to-end transformer that maps the full history of past observations and actions directly to future actions and sub-goal completion predictions. The hypothesis is that a sufficiently expressive sequence model with access to the complete episode history can *implicitly* learn the planning, perceptual grounding, and error recovery that modular systems implement *explicitly* — and may do so more robustly by avoiding the information bottlenecks of modular interfaces.

The model receives a temporally interleaved sequence of visual features and previous action embeddings:

$$\mathbf{H} = \text{Transformer}\big([\mathbf{o}_1, \mathbf{a}_1, \mathbf{o}_2, \mathbf{a}_2, \ldots, \mathbf{o}_t]; \mathbf{P}_{\text{temporal}} + \mathbf{P}_{\text{modal}}\big), \tag{8.11}$$

where $\mathbf{o}_i \in \mathbb{R}^D$ are features from a frozen ResNet-18 encoder, $\mathbf{a}_i \in \mathbb{R}^D$ are learned embeddings of discrete action tokens, and $\mathbf{P}_{\text{temporal}}$ and $\mathbf{P}_{\text{modal}}$ are learned positional encodings that distinguish time steps and modalities (observation vs. action). The Transformer produces contextualized representations $\{\mathbf{h}_1^{\text{obs}}, \mathbf{h}_1^{\text{act}}, \ldots\}$.

From the last observation token, two prediction heads operate:

$$p(a_{t+1} \mid \mathbf{o}_{1:t}, a_{1:t}) = \text{softmax}\big(\text{ActionHead}(\mathbf{h}_t^{\text{obs}})\big), \tag{8.12}$$

$$p(\text{sub-goal}_k \text{ completed} \mid \mathbf{o}_{1:t}, a_{1:t}) = \sigma\big(\text{ProgressHead}^{(k)}(\mathbf{h}_t^{\text{obs}})\big), \tag{8.13}$$

where the progress head has one output per sub-goal in the instruction, and the action head predicts the next action from the discrete action vocabulary.

The training objective combines multiple auxiliary losses that provide additional supervision signals:

$$\mathcal{L} = \mathcal{L}_{\text{action}} + \lambda_{\text{obj}}\mathcal{L}_{\text{object}} + \lambda_{\text{prog}}\mathcal{L}_{\text{progress}} + \lambda_{\text{mask}}\mathcal{L}_{\text{mask}}, \tag{8.14}$$

where $\mathcal{L}_{\text{object}}$ is an object detection loss (per-frame multi-label classification of visible objects), $\mathcal{L}_{\text{progress}}$ is the binary cross-entropy on sub-goal completion, and $\mathcal{L}_{\text{mask}}$ is a self-supervised masked observation prediction loss that encourages the model to learn a predictive world representation. These auxiliary losses provide dense training signal that mitigates the sparse-reward credit assignment problem endemic to long-horizon tasks.

The key architectural insight is that *full episode context resolves perceptual aliasing.* Consider the observation of looking at a fridge. If the agent is holding an apple, the correct action is likely `Put(Apple, Fridge)`. If the agent is empty-handed and searching for an apple, the correct action is likely `Open(Fridge)` to look inside. The same visual stimulus demands different responses depending on the history — and the Transformer's long-context attention provides the memory to disambiguate such situations without explicit state representations.

8.3 Key Techniques

8.3.1 Skill Chaining and the Options Framework

The *options framework* (Sutton, Precup, and Singh, 1999) provides a mathematically principled formalism for temporal abstraction in reinforcement learning. An option ω is a temporally extended course of action, defined as a triple:

$$\omega = \langle \mathcal{I}_\omega, \pi_\omega, \beta_\omega \rangle, \tag{8.15}$$

where $\mathcal{I}_\omega \subseteq \mathcal{S}$ is the initiation set (states from which the option may be started), $\pi_\omega : \mathcal{S} \to \Delta(\mathcal{A})$ is the intra-option policy that selects primitive actions, and $\beta_\omega : \mathcal{S} \to [0, 1]$ is the termination condition (the probability of terminating the option in each state).

Skill chaining builds a library of options (discovered from data or hand-designed) and composes them for novel long-horizon tasks. Given a task goal state s_g, a high-level policy ϕ (the "conductor") selects among the available options:

$$\omega_t \sim \phi(\cdot \mid s_t, s_g), \tag{8.16}$$

while the selected option executes its intra-option policy until termination. This hierarchical decomposition reduces the effective planning horizon: the high-level policy operates at the granularity of skills (steps of 50–200 primitive actions), while the low-level policies need only solve local, temporally short sub-problems.

Play-based skill discovery (Lynch et al., 2020; Play-LMP) learns options from unlabeled "play" data — long trajectories where a human teleoperates the robot performing varied, goal-agnostic behaviors. The raw trajectory is segmented into windows of length H, and a variational autoencoder is trained to encode each window into a latent skill code $\mathbf{z} \in \mathbb{R}^{d_z}$ and reconstruct the actions:

$$\mathcal{L}_{\text{play}} = \mathbb{E}_{\tau\sim\mathcal{D}}\mathbb{E}_{q_\phi(\mathbf{z}|\tau)}\left[\sum_{t=0}^{H-1} \log \pi_\theta(a_t \mid s_t, \mathbf{z})\right] - \beta\ \mathrm{KL}\big(q_\phi(\mathbf{z} \mid \tau) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I})\big). \tag{8.17}$$

The first term is the action reconstruction loss — given the latent code $\mathbf{z}$, can the decoder reproduce the teleoperated actions? The second term is a β-VAE-style KL regularizer that encourages the latent codes to be compact and well-distributed. After training, the latent space $\mathbf{z}$ encodes a continuum of reusable skills. New long-horizon tasks are solved by chaining skills:

a high-level planner selects a sequence of skill codes $\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_K$, each of which is executed by conditioning the decoder $\pi_\theta(a \mid s, \mathbf{z}_k)$ on the corresponding code.

The learned skill space often exhibits interpretable structure: nearby codes correspond to semantically similar behaviors, and linear interpolation in $\mathbf{z}$-space produces smooth behavioral transitions. This property — that learned skill representations are semantically meaningful — is critical for downstream use in planning and instruction following.

8.3.2 Progress Monitoring and Recovery

A defining challenge of long-horizon execution is that failures cascade. If the agent drops the apple between the sink and the fridge, blindly continuing the plan (opening the fridge, attempting to place nothing inside) produces chaotic and unrecoverable states. *Progress monitoring* trains classifiers to predict, from the history, which sub-goals have been completed:

$$p(\text{sub-goal}_k \text{ completed} \mid \mathbf{o}_{1:t}, a_{1:t}) = \sigma\big(f^{(k)}_{\text{prog}}(\mathbf{h}_t)\big). \tag{8.18}$$

When the progress monitor detects that the agent is stuck — the probability of any sub-goal being completed has not increased over a threshold number of steps — a recovery mechanism intervenes. Recovery strategies span a spectrum:

- **Replanning:** The high-level planner is re-invoked from an estimated current symbolic state, potentially discovering an alternative path that circumvents the failure (e.g., finding a different sink).
- **Exploratory recovery:** The agent deploys an information-gathering policy to re-localize itself and re-identify the relevant objects.
- **Backtracking:** The agent attempts to reverse the last successful action (re-enter the room, re-approach the object) and retry. This requires reversible skills — actions whose effects can be approximately undone.

The effectiveness of recovery depends critically on the diversity of the skill repertoire. An agent with only forward skills (pick, place) cannot recover from a drop; an agent with search skills can re-find the object; an agent with backtracking skills can return to a known-good state and restart.

8.3.3 Hierarchical Reinforcement Learning

Hierarchical Reinforcement Learning (HRL) provides a learning-theoretic foundation for long-horizon task completion. The central idea is to learn policies at multiple temporal scales, with higher-level policies setting sub-goals for lower-level policies:

$$\pi_{\text{high}}(g_{\text{sub}} \mid s, g_{\text{task}}) : \text{select intermediate sub-goals every } K \text{ steps}, \tag{8.19}$$

$$\pi_{\text{low}}(a \mid s, g_{\text{sub}}) : \text{execute primitive actions to achieve the current sub-goal.} \tag{8.20}$$

The low-level policy is trained via goal-conditioned RL. The reward for achieving sub-goal g from state s is typically a negative distance in a learned embedding space:

$$r^{\text{low}}(s_t, g) = -\,\|\phi(s_t) - \phi(g)\|_2^2\,, \tag{8.21}$$

where $\phi : \mathcal{S} \to \mathbb{R}^d$ is a state embedding function (e.g., a VAE encoder or a contrastively learned representation). The low-level policy is trained to minimize the time-to-goal via standard actor-critic methods, with the distance-based reward providing dense training signal.

Hindsight Experience Replay (HER; Andrychowicz et al., 2017) is essential for training goal-conditioned policies at scale. Failed trajectories — where the agent did not reach the commanded sub-goal g — are *relabeled*: the trajectory is retroactively assigned the goal that was *actually* achieved:

$$\mathcal{D}_{\text{HER}} = \{(s_t, a_t, r'_t, s_{t+1}, g') \mid g' = \phi(s_T) \text{ for some } T \geq t\}. \tag{8.22}$$

This relabeling converts every trajectory into a successful demonstration of *some* goal, dramatically increasing the effective training data and making sparse-reward goal-reaching feasible. HER is the algorithmic workhorse behind much of the progress in goal-conditioned manipulation.

The high-level policy is trained with a reward that depends on task progress — typically the negative distance to the terminal goal, sparse upon task completion, or learned via inverse RL from demonstrations. The two policies can be trained jointly in a hierarchical actor-critic framework (e.g., HIRO, HAC), or sequentially (pre-train low-level policies, freeze them, and train the high-level policy on top).

8.3.4 Plan Verification and Monitoring via Language Models

Beyond generating plans, LLMs can serve as *plan verifiers* and *execution monitors*. Given a candidate plan $\pi = (o_1, \ldots, o_T)$ and the instruction ℓ, an LLM can be prompted to evaluate whether the plan is semantically valid — does it achieve the stated goal? — and whether it respects commonsense ordering constraints:

$$p(\text{plan valid} \mid \pi, \ell) = \sigma\big(\text{LLM_logit}(\texttt{yes} \mid \text{prompt}(\pi, \ell))\big). \tag{8.23}$$

This verification can be applied as a filter after plan generation, rejecting implausible plans before execution. Similarly, during execution, an LLM can monitor the observation stream to detect anomalies — "the robot was supposed to pick up the apple, but the apple is still on the table" — and trigger replanning. This connects LLM-based planning to the broader monitoring and fault-detection literature in robotics.

8.4 Connections

Long-horizon manipulation and classical AI planning: The STRIPS formalism, developed at SRI in the early 1970s, remains remarkably relevant to modern embodied AI. Heuristic search planners like Fast Downward routinely solve problems with thousands of operators by exploiting delete-relaxation heuristics ($h^{\text{FF}}, h^{\text{add}}, h^{\max}$). The frontier is closing the loop: perception systems produce symbolic state estimates, classical planners search over them, and learned policies ground symbolic operators in continuous actions.

Learning from play and intrinsic motivation: Children learn through undirected, intrinsically motivated play long before they follow explicit multi-step instructions. Play-based learning (Play-LMP, Playful) seeks to replicate this: collect diverse, goal-agnostic interaction data through curiosity-driven exploration and extract reusable skills via unsupervised segmentation. The connection to intrinsic motivation — empowerment, information gain, novelty — suggests that the data collection process itself can be automated without human supervision.

Memory architectures for long horizons: The Episodic Transformer's reliance on full-trajectory memory connects to broader research on long-context sequence models. Efficient attention mechanisms — linear attention (Performer), sparse attention (BigBird, Longformer), and state-space models (Mamba, S4) — are essential for scaling to episodes with thousands of steps without quadratic memory and compute growth.

Planning as inference: The control-as-inference framework (Levine, 2018; Toussaint, 2009) reframes planning as probabilistic inference in a graphical model where binary optimality

variables $\mathcal{O}_t \in \{0, 1\}$ encode the reward: $p(\mathcal{O}_t = 1 \mid s_t, a_t) \propto \exp(r(s_t, a_t))$. The posterior over actions given that all $\mathcal{O}_t = 1$ recovers the optimal policy. This probabilistic perspective unifies classical planning (deterministic inference in a deterministic model), model-based RL (approximate inference in a learned model), and learned planners (amortized inference via a policy network).

Foundation models for planning: The emergence of general-purpose vision-language-action models (RT-2, Octo, OpenVLA) trained on massive multi-embodiment datasets suggests a future where a single pre-trained model can plan and execute long-horizon tasks zero-shot from language, much as GPT-4 can perform complex multi-step reasoning zero-shot from text. However, the gap between text-based reasoning (where errors have no physical consequences) and physical execution (where errors compound and cause irreversible state changes) remains vast.

8.5 Open Problems

1. **Scaling horizon without scaling error:** As the task horizon grows, open-loop plan execution becomes infeasible due to compounding errors. Developing planners that produce *closed-loop* plans — policies that react to observations rather than fixed action sequences — while maintaining computational tractability over hundreds of steps is a central challenge.

2. **Symbol grounding for open-world planning:** Classical planning requires a predefined set of predicates and operators. In open-world environments, novel objects, properties, and action types appear. Automatically learning to ground novel concepts from perceptual experience without human annotation — open-vocabulary symbol grounding — would enable planning in previously unseen domains.

3. **Interleaving planning and learning:** Planning is sample-efficient but relies on accurate world models; learning is robust to model error but data-hungry. Architectures that fluidly interleave planning (for novel situations where the learned policy is uncertain) and learning (for familiar situations where the policy is reliable), gradually shifting from model-based to model-free as experience accumulates, mirror human skill acquisition.

4. **Multi-agent long-horizon coordination:** When multiple agents collaborate on long-horizon tasks ("you set the table while I cook"), the joint plan space grows exponentially with agent count. Decentralized planning under partial observability with communication constraints — where agents must negotiate task allocation and coordinate temporally extended actions — remains largely unsolved.

5. **Explainable and verifiable plans:** In safety-critical domains (elder care, surgical assistance), the robot must explain its plan and provide verifiable guarantees that it respects constraints. Combining neural planners with formal verification — proving that a learned plan satisfies temporal logic specifications (LTL, STL) — is a nascent area with immense practical importance.

6. **Lifelong and continual skill acquisition:** A robot deployed over years should continuously expand its skill repertoire, learning from successes and failures without forgetting previously acquired competencies. Continual learning in the context of long-horizon planning — where new skills are integrated into the planning model and old skills are retained — integrates challenges from continual learning, hierarchical RL, and open-ended learning.

7. **Uncertainty-aware hierarchical planning:** Both LLM-generated and classical plans are typically deterministic — a single sequence of actions. In reality, the environment is

stochastic and partially observable. *Branching* plans — trees or policies rather than sequences — that explicitly reason about multiple possible futures and select actions that are robust across them, while maintaining computational efficiency, remain an open research frontier.

Part IV

Learning Paradigms

Chapter 9

Imitation Learning for Robotics

9.1 Motivation

Reinforcement learning in the physical world is punitive. Each interaction demands wall-clock time measured in seconds, minutes, or hours. A robot learning to grasp from scratch might require thousands of real-world episodes, each potentially ending in hardware damage. Meanwhile, humans can demonstrate complex tasks with ease—a surgeon guiding a scalpel, a chef folding dough, a machinist operating a lathe. The demonstrations do not come with reward annotations. They come as trajectories of states and actions, silent records of expertise. The question imitation learning poses is deceptively simple: can we learn directly from these demonstrations, bypassing the need for explicit reward engineering and unsafe exploration?

The mathematical formulation is elegant. Given a dataset of expert trajectories $\mathcal{D} = \{\tau_1, \tau_2, \ldots, \tau_N\}$ where each $\tau = (s_0, a_0, s_1, a_1, \ldots, s_T)$, drawn from an unknown expert policy π^*, the goal is to produce a policy π_θ whose behavior is indistinguishable from the expert's. But imitation is not merely supervised learning over state-action pairs. It is learning a dynamical system that must remain stable under its own feedback—a fundamentally harder class of problems. A policy that agrees with the expert 99% of the time on expert-visited states can still diverge catastrophically when its own errors steer it into unfamiliar regions of state space.

9.2 Behavioral Cloning and Covariate Shift

9.2.1 Formulation

The simplest form of imitation learning reduces the problem to supervised learning. Given a dataset of state-action pairs $\mathcal{D} = \{(s_i, a_i)\}_{i=1}^{M}$ collected by rolling out the expert policy π^*, behavioral cloning seeks the policy π_θ that minimizes expected loss over the empirical distribution of expert data:

$$\theta^* = \arg\min_\theta \; \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\ell\big(\pi_\theta(s), a\big) \right]. \tag{9.1}$$

For continuous action spaces, ℓ is typically the mean squared error: $\ell(\hat{a}, a) = \|\hat{a} - a\|^2$. For discrete actions, cross-entropy: $\ell(\hat{p}, a) = -\log \hat{p}(a)$. The optimization is straightforward: stochastic gradient descent on a static dataset. There is no environment interaction, no reward signal, and no temporal credit assignment. The only requirement is a dataset of expert behavior.

9.2.2 The Covariate Shift Problem

Behavioral cloning appears deceptively effective in open-loop evaluation—the policy achieves low validation loss on held-out expert data. The failure emerges at deployment time, when the learned policy interacts with the environment in closed loop. At step t, the policy π_θ is queried on state $s_t \sim d_{\pi_\theta}$, the *rollout distribution* induced by π_θ itself. This distribution diverges from the training distribution d_{π^*} because $\pi_\theta \neq \pi^*$. The state distribution shift is *covariate shift* in the context of dynamical systems: the distribution of inputs to the policy changes as a consequence of the policy's own predictions.

Consider the compounding mechanism. At step $t = 0$, the initial state $s_0 \sim \rho_0$ is drawn from the same distribution for both expert and learner. The learner makes a small prediction error: $a_0^{\text{learner}} = \pi^*(s_0) + \varepsilon_0$. The environment transitions to $s_1' \sim \mathcal{P}(\cdot \mid s_0, a_0^{\text{learner}})$. If the expert

would have visited s_1 (under action a_0^*), then $\|s_1' - s_1\| > 0$. The policy is now evaluated on a state s_1' that lies off the manifold of expert-visited states. Its prediction error at step 1, ε_1, is larger than ε_0 because the training data contains no examples near s_1'. This larger error pushes s_2' even further from the expert manifold, and the errors compound.

9.2.3 Error Bound Analysis

Under a Lipschitz continuity assumption on the policy and dynamics, we can derive the classical error bound. Suppose the expert policy is λ_π-Lipschitz and the dynamics are $\lambda_\mathcal{P}$-Lipschitz in state. Let ε bound the per-step supervised error: $\mathbb{E}_{(s,a)\sim d_{\pi^*}}[\|\pi_\theta(s) - \pi^*(s)\|] \le \varepsilon$. Then after T steps, the expected cumulative error scales as:

$$\mathbb{E}\left[\sum_{t=0}^{T} \|s_t^{\text{learner}} - s_t^{\text{expert}}\|\right] \le \varepsilon T + \mathcal{O}\big(\varepsilon^2 T^2\big). \tag{9.2}$$

The first term εT is the linear accumulation of independent per-step errors. The second term $\mathcal{O}(\varepsilon^2 T^2)$ is the consequence of *compounding*—each error shifts the state, which causes larger future errors, which shift the state further. For a horizon of $T = 1000$ steps and per-step error rate $\varepsilon = 0.01$, the quadratic term dominates, and the bound exceeds the scale of the state space. In practice, this manifests as the policy driving the robot into configurations never seen during training, where the policy's behavior becomes essentially random.

The covariate shift analysis reveals that behavioral cloning's performance is fundamentally limited by the effective horizon of the task and the Lipschitz constants of the dynamics. Tasks with fast, sensitive dynamics (small time constants, large Lipschitz constants) are particularly vulnerable. Tasks with resetting or self-correcting dynamics (where errors naturally decay) are more forgiving.

9.3 DAgger: Dataset Aggregation

9.3.1 The Core Algorithm

Ross, Gordon, and Bagnell (2011) identified the root cause of compounding error: the training distribution is static while the test distribution is dynamic and policy-dependent. Their algorithm, DAgger (Dataset Aggregation), iteratively aligns the two distributions by having the expert label the states that the learner actually visits.

The procedure iterates for N rounds. At round k:

1. Train π_{θ_k} on the current aggregate dataset $\mathcal{D}_k$ using supervised learning.
2. Roll out π_{θ_k} in the environment to collect a trajectory $(s_1^{(k)}, \dots, s_T^{(k)})$.
3. Query the expert at each visited state: $a_t^* = \pi^*(s_t^{(k)})$.
4. Augment the dataset: $\mathcal{D}_{k+1} \leftarrow \mathcal{D}_k \cup \{(s_t^{(k)}, a_t^*)\}_{t=1}^{T}$.

Optionally, the first round uses pure expert trajectories (behavioral cloning baseline), and subsequent rounds mix the learner's distribution with the expert's through a mixing parameter β_k: at each step, with probability β_k the expert chooses the action (and the state is recorded for labeling), and with probability $1 - \beta_k$ the learner acts. As k increases, $\beta_k \to 0$, and the learner is increasingly in control of the state distribution.

9.3.2 Theoretical Guarantee

The intuition behind DAgger can be formalized. Let ℓ be a strongly convex loss function. Define the average per-step loss of the best policy in hindsight as $\varepsilon_N = \min_{\theta\in\Theta} \frac{1}{N}\sum_{k=1}^{N} \mathbb{E}_{s\sim d_{\pi_{\theta_k}}}[\ell(\pi_\theta(s), \pi^*(s))]$. Under the assumption that the expert's actions are deterministic and the learner's policy space is rich enough, DAgger achieves:

$$\mathbb{E}\left[\frac{1}{T}\sum_{t=1}^{T} \ell(\pi_{\hat{\theta}}(s_t), \pi^*(s_t))\right] \leq \varepsilon_N + \mathcal{O}\left(\frac{1}{N}\right), \tag{9.3}$$

where $\hat{\theta}$ is the policy returned (typically the best or the last). This bound shows that DAgger converges to the best policy in the considered class, with the average loss decreasing as $\mathcal{O}(1/N)$. The total amount of expert feedback required scales as $\mathcal{O}(T^2 \log(1/\varepsilon))$, which, while superlinear in T, is polynomial and avoids the exponential compounding of behavioral cloning.

The practical limitation of DAgger is the requirement for an interactive expert—one that can be queried online during the learner's rollouts. In many real-world settings (surgery, driving, delicate manipulation), the expert cannot safely intervene during the learner's mistakes, or the cost of expert time is prohibitive. Offline variants of DAgger, which use learned models to simulate the learner's state distribution, partially address this limitation.

9.4 Inverse Reinforcement Learning

9.4.1 The IRL Framework

Behavioral cloning and DAgger copy actions directly. Inverse reinforcement learning takes the indirect but more principled route: infer the reward function that the expert is implicitly optimizing, then solve the forward RL problem with that inferred reward. The motivation is twofold. First, reward functions are compact, transferable representations of task knowledge—a reward for "grasp the cup" generalizes across environments, whereas a state-action policy may not. Second, recovering the reward exposes the causal structure of the task, enabling transfer and compositional reasoning.

Given an MDP $\mathcal{M} \setminus \{\mathbb{R}\}$ (everything except the reward function) and expert trajectories $\mathcal{D}$, IRL seeks $\mathbb{R}^*$ such that the expert's policy is optimal for $\mathbb{R}^*$:

$$\pi^* \in \arg\max_\pi \mathbb{E}_{\tau\sim\pi}\left[\sum_t \gamma^t \mathbb{R}^*(s_t, a_t)\right]. \tag{9.4}$$

The fundamental challenge is under-specification: the expert's behavior is consistent with infinitely many reward functions. The degenerate solution $\mathbb{R}(s, a) = 0$ for all (s, a) makes every policy optimal and therefore "explains" any observed behavior. Resolving this ambiguity requires an inductive bias—a principle for selecting among reward functions that equally well explain the data.

9.4.2 Maximum Entropy IRL

Ziebart et al. (2008) introduced the principle of maximum entropy as the inductive bias for IRL. The postulate: among all reward functions consistent with the expert's behavior, choose the one that maximizes the entropy of the induced trajectory distribution. This is the least-committed choice—the reward function that explains the expert without imposing unnecessary structure.

The trajectory distribution under a reward function $\mathbb{R}$ is modeled as a Boltzmann distribution:

$$P(\tau \mid \mathbb{R}) = \frac{1}{Z(\mathbb{R})} \exp\left(\sum_{t=0}^{T} \mathbb{R}(s_t, a_t) \right), \tag{9.5}$$

where $Z(\mathbb{R}) = \int \exp(\sum_t \mathbb{R}(s_t, a_t))\, d\tau$ is the partition function. The maximum entropy IRL objective maximizes the log-likelihood of the expert trajectories under this distribution:

$$\mathbb{R}^* = \arg\max_{\mathbb{R}} \; \mathcal{L}(\mathbb{R}) = \arg\max_{\mathbb{R}} \; \mathbb{E}_{\tau \sim \pi^*} \left[\sum_t \mathbb{R}(s_t, a_t) - \log Z(\mathbb{R}) \right]. \tag{9.6}$$

The gradient of the log-likelihood with respect to the reward parameters (when $\mathbb{R}$ is parameterized linearly as $\mathbb{R}(s, a) = \mathbf{w}^\top \phi(s, a)$) has a particularly intuitive form:

$$\nabla_{\mathbf{w}} \mathcal{L} = \mathbb{E}_{\tau \sim \pi^*} \left[\sum_t \phi(s_t, a_t) \right] - \mathbb{E}_{\tau \sim P(\cdot \mid \mathbb{R}_{\mathbf{w}})} \left[\sum_t \phi(s_t, a_t) \right]. \tag{9.7}$$

This is a moment-matching condition: the optimal reward function makes the expected feature counts under the model distribution equal to the empirical feature counts under the expert distribution. The algorithm alternates between (a) estimating the feature expectations under the current reward by solving the forward RL problem (computing the soft optimal policy under $\mathbb{R}_{\mathbf{w}}$), and (b) adjusting $\mathbf{w}$ to reduce the gap between model and expert feature expectations.

The partition function $Z(\mathbb{R})$ is computationally challenging because it requires integrating over the enormous space of all possible trajectories. In discrete state spaces, dynamic programming can compute $Z(\mathbb{R})$ exactly via the soft value iteration backup:

$$V(s) = \operatorname{softmax}_a \big(\mathbb{R}(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot \mid s, a)} [V(s')] \big), \tag{9.8}$$

where $\operatorname{softmax}_a f(a) = \log \sum_a \exp(f(a))$. The partition function is then $Z(\mathbb{R}) = \sum_s V(s)$. In continuous spaces, approximations based on sampling or importance weighting are necessary.

9.4.3 Generative Adversarial Imitation Learning

Ho and Ermon (2016) forged a deep connection between IRL and generative adversarial networks, yielding an algorithm that eliminates the need to explicitly recover or optimize over reward functions. The Generative Adversarial Imitation Learning (GAIL) framework is a two-player minimax game between a policy generator and a discriminator:

$$\min_{\pi_\theta} \max_{D_\phi} \; \mathbb{E}_{\tau \sim \pi_\theta} \big[\log D_\phi(s, a) \big] + \mathbb{E}_{\tau \sim \pi^*} \big[\log \big(1 - D_\phi(s, a) \big) \big]. \tag{9.9}$$

The generator π_θ is the learned policy; the discriminator $D_\phi : \mathcal{S} \times \mathcal{A} \to [0, 1]$ is a classifier trained to distinguish expert state-action pairs from learner-generated pairs. The discriminator is trained to maximize classification accuracy; the generator is trained to produce state-action pairs that the discriminator misclassifies as expert.

The occupancy measure connection. Define the *occupancy measure* of a policy π as the discounted distribution of state-action visitations:

$$\rho_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \, \mathbf{1}_{\{(s_t, a_t) = (s, a)\}} \right]. \tag{9.10}$$

The occupancy measure is a sufficient statistic for the policy's long-term behavior: two policies with the same occupancy measure have the same expected return under any reward function. There is a one-to-one mapping between policies and their occupancy measures, and the set of valid occupancy measures is a convex polytope defined by the Bellman flow constraints.

At the optimum of the inner maximization over D_ϕ, the discriminator converges pointwise to:

$$D^*(s,a) = \frac{\rho_{\pi^*}(s,a)}{\rho_{\pi^*}(s,a) + \rho_{\pi_\theta}(s,a)}. \tag{9.11}$$

Substituting D^* into the outer minimization reveals that GAIL minimizes the Jensen-Shannon divergence between the occupancy measures of the learner and the expert:

$$\min_{\pi_\theta} \; D_{\mathrm{JS}}\big(\rho_{\pi_\theta} \,\big\|\, \rho_{\pi^*}\big)\,. \tag{9.12}$$

The training procedure. In practice, GAIL alternates between:

1. Sampling trajectories $\tau \sim \pi_\theta$ and computing discriminator targets: expert pairs $(s,a) \sim \pi^*$ get label 1; learner pairs get label 0.
2. Updating the discriminator parameters via gradient ascent on the binary classification objective.
3. Using the discriminator's output as a learned reward signal: $\tilde{\mathbb{R}}(s,a) = -\log\big(1 - D_\phi(s,a)\big)$ or equivalently $\tilde{\mathbb{R}}(s,a) = \log D_\phi(s,a)$.
4. Updating the policy via any RL algorithm (TRPO, PPO, or SAC) with the learned reward.

Connection to f-divergences. GAIL's objective can be generalized. For any convex function $f : \mathbb{R}^+ \to \mathbb{R}$ with $f(1) = 0$, the f-divergence between occupancy measures is:

$$D_f(\rho_{\pi_\theta} \| \rho_{\pi^*}) = \mathbb{E}_{(s,a)\sim\rho_{\pi^*}}\left[f\left(\frac{\rho_{\pi_\theta}(s,a)}{\rho_{\pi^*}(s,a)} \right) \right]. \tag{9.13}$$

GAIL corresponds to $f(u) = -(u+1)\log\frac{1+u}{2} + u\log u$ (Jensen-Shannon). Other choices of f yield different imitation objectives: χ^2-divergence (least-squares GAIL), reverse KL (AIRL), and total variation. Each choice induces a different discriminator training objective and a different learned reward function, with trade-offs in sample efficiency and stability.

AIRL (Adversarial Inverse RL). Fu et al. (2018) extended GAIL with the insight that the discriminator can be structured to recover an explicit reward function that is disentangled from environment dynamics. In AIRL, the discriminator takes the form:

$$D_\phi(s,a,s') = \frac{\exp(f_\phi(s,a,s'))}{\exp(f_\phi(s,a,s')) + \pi_\theta(a \mid s)}, \tag{9.14}$$

where $f_\phi(s,a,s') = g_\phi(s,a) + \gamma h_\phi(s') - h_\phi(s)$ is a learned function that decomposes into a reward approximant g_ϕ and a shaping term $\gamma h_\phi(s') - h_\phi(s)$. The shaping term, being a potential-based difference, does not affect the optimal policy. The stabilized discriminator training in AIRL produces reward functions that transfer across dynamics, addressing a key limitation of standard GAIL.

9.5 Learning from Play

Lynch et al. (2020) challenged a fundamental assumption of imitation learning: that demonstrations must be task-specific. They proposed learning from *play*—unstructured, goal-free interaction data where a human teleoperates the robot in open-ended exploration. No task labels, no predefined goals, no segmented trajectories. Just continuous streams of interaction spanning diverse object manipulations.

The model is a goal-conditioned policy $\pi(a \mid s, s_g)$ where s_g is a goal observation specifying what the robot should achieve. A visual encoder f_ϕ maps both the current observation and the goal observation to a shared latent space via a siamese architecture:

$$z = f_\phi(s), \qquad z_g = f_\phi(s_g). \tag{9.15}$$

The policy conditions on both representations, typically through concatenation or attention-based fusion:

$$a \sim \pi_\theta\big(a \mid s, \mathrm{fuse}(z, z_g)\big). \tag{9.16}$$

Training proceeds by extracting random temporal windows from the play data. For any window spanning time steps $[t_0, t_1]$, the observation at t_0 serves as the current state s, the observation at t_1 as the goal s_g, and the actions in between become supervision for behavioral cloning:

$$\mathcal{L}_{\text{play}} = \sum_{i=0}^{t_1 - t_0 - 1} \big\| \pi_\theta(s_{t_0+i}, s_{t_1}) - a_{t_0+i} \big\|^2. \tag{9.17}$$

Crucially, the windows are not restricted to task boundaries. A single play session of teleoperated interaction produces thousands of windows—every pair of timesteps defines a valid (current, goal) pair. The data efficiency is staggering: an hour of teleoperation can produce millions of training windows.

Latent plan formulation. A powerful extension treats the learned representation z_g as a *latent plan* that the policy follows. At test time, given a desired goal image s_g, the encoder produces $z_g = f_\phi(s_g)$, and the policy executes actions to realize that plan. Since z_g was trained on windows from diverse play data, it encodes a vocabulary of reusable motor skills—approach, grasp, lift, place, push, slide—that can be composed for novel goals at test time.

The play paradigm aligns with the self-supervised learning philosophy that proved transformative in NLP and vision. The model learns an embodied visual representation as a byproduct of the imitation objective: features that are predictive of the actions connecting two observations must capture object affordances, physics, and motor control information. These features transfer across tasks and provide a powerful initialization for downstream goal-conditioned policies.

9.6 Action Chunking with Temporal Ensembles

Zhao et al. (2023) identified a complementary source of compounding error: the temporal granularity of action prediction. Standard behavioral cloning predicts one action at a time, requiring T sequential predictions for a horizon of T. Each prediction is subject to error; the errors accumulate across the temporal sequence.

Their Action Chunking Transformer (ACT) reduces the effective horizon by a factor of k by predicting chunks of k future actions at each step:

$$(\hat{a}_t, \hat{a}_{t+1}, \ldots, \hat{a}_{t+k-1}) = \pi_\theta(s_t). \tag{9.18}$$

A Transformer encoder processes the observation and action history; a Transformer decoder produces the chunk autoregressively or in parallel. Predicting chunks rather than single actions means the policy makes only $\lceil T/k \rceil$ predictions per trajectory instead of T, dramatically reducing the compounding error.

At deployment, predictions from overlapping chunks are combined via *temporal ensemble*. At time step t, the chunk predicted at step t contributes $\hat{a}_t^{(t)}$, the chunk predicted at step $t-1$ contributed $\hat{a}_t^{(t-1)}$, and so on back to $t-k+1$. These multiple predictions for the same target action are averaged with exponential weighting:

$$a_t^{\text{ensemble}} = \frac{\sum_{i=0}^{k-1} w_i \cdot \hat{a}_t^{(t-i)}}{\sum_{i=0}^{k-1} w_i}, \qquad w_i = \exp(-\lambda i). \tag{9.19}$$

The decay constant λ controls how much trust is placed in older predictions. Recent predictions (small i) receive higher weight because they are conditioned on more up-to-date observations. The temporal ensemble provides two benefits: it smooths the action trajectory (reducing jitter from individual prediction noise) and it provides robustness against occasional mispredictions (a single bad chunk prediction is averaged with several other predictions).

ACT has demonstrated state-of-the-art performance on fine-grained bimanual manipulation tasks, including assembling a chain of beads, flipping a pancake, and opening an airtight container—tasks that require precise, coordinated motion over horizons of hundreds of steps.

9.7 Key Techniques and Practical Considerations

The imitation learning toolkit has expanded considerably beyond behavioral cloning. The choice among methods depends on the form and cost of expert supervision:

- **DAgger** provides a principled reduction from interactive imitation to supervised learning. Its theoretical guarantees require an interactive expert and strongly convex losses. In practice, it works well when the expert can safely intervene and labeling cost is low.
- **GAIL and AIRL** eliminate the need for interactive experts by using a learned discriminator as a reward proxy. They require environment interaction for policy optimization (RL), which is expensive in the real world but feasible in simulation. The learned reward can transfer across environments and tasks.
- **Maximum entropy IRL** produces interpretable reward functions that are useful for analysis and transfer. The computational cost of repeatedly solving forward RL makes it impractical for online settings; it is more commonly used in offline or batch settings.
- **Learning from play** is ideal when task-specific demonstration is expensive but unstructured teleoperation is cheap. The self-supervised nature of the training windows provides data abundance.
- **Action chunking** is complementary to all of the above—it can be combined with any imitation learning algorithm to reduce effective horizon and smooth predictions.

Data augmentation plays a crucial role in practical imitation learning. Random cropping, color jitter, and background replacement on image observations improve the visual robustness of imitated policies. More sophisticated augmentations include: action noise injection during training (making the policy robust to its own errors), domain randomization of visual properties (lighting, textures, camera pose), and temporal data augmentation (time warping, subsampling, adding state noise).

9.8 Connections to Other Fields

Imitation learning sits at the intersection of supervised learning, reinforcement learning, and causal inference. The covariate shift problem in behavioral cloning is a special case of distribution shift in dynamical systems—the same phenomenon studied in off-policy RL (where the behavioral policy differs from the target policy), domain adaptation (where training and test distributions differ), and time-series forecasting (where errors propagate through autoregressive models).

The connection between GAIL and f-divergence minimization links imitation learning to the vast literature on generative modeling. The discriminator in GAIL is estimating the density ratio $\rho_{\pi^*}/(\rho_{\pi^*} + \rho_{\pi_\theta})$, which is a form of density ratio estimation. The duality between f-divergences and variational forms connects GAIL to variational inference, GANs, and contrastive learning. The same mathematical machinery—the Fenchel conjugate, the variational representation of divergences—appears in all of these settings.

The maximum entropy principle is a thread running through multiple disciplines:

$$\mathcal{H}(\pi) = -\mathbb{E}_{\tau\sim\pi}[\log \pi(\tau)]\,. \tag{9.20}$$

In statistical mechanics, the Boltzmann distribution maximizes entropy subject to energy constraints. In entropy-regularized RL, the softmax policy $\pi(a \mid s) \propto \exp(Q(s,a)/\tau)$ maximizes entropy-regularized return. In MaxEnt IRL, the principle selects the least-committed reward function. In variational inference, the ELBO can be derived as maximum entropy subject to moment-matching constraints. The common mathematical structure is profound: when faced with uncertainty or under-specification, choose the least informative distribution consistent with the observed constraints.

The visual encoder trained in learning from play connects imitation to self-supervised representation learning. The encoder f_ϕ learns features that are predictive of the actions taken between two observations. This is a form of *inverse dynamics modeling*—learning what actions were taken given the state change—which is a well-established self-supervised objective in reinforcement learning. Pre-trained representations like R3M and VC-1 extend this idea to large-scale egocentric video data, learning visual features that are broadly useful for downstream manipulation tasks.

9.9 Open Problems

One-shot and few-shot imitation. Current methods require tens to hundreds of demonstrations per task. Humans can imitate from a single demonstration by parsing it into reusable skill components and composing them for the new context. Bridging this gap requires models that decompose demonstrations into skills, recognize analogies between new and previously seen tasks, and adapt skill parameters to novel object geometries and dynamics. Meta-imitation learning—learning to imitate from few demonstrations by training across a distribution of tasks—provides a formal framework for this problem.

Cross-embodiment imitation. The internet contains billions of hours of humans performing tasks with human hands. Transferring this data to robots with different morphologies requires solving the embodiment mapping problem: learning a shared representation of task intent and motion that abstracts away the specifics of the body. The kinematic mapping $\phi : \mathcal{A}_{\text{human}} \to \mathcal{A}_{\text{robot}}$ is non-trivial and task-dependent. Learned retargeting via cycle-consistency and domain-adversarial training are promising directions.

Interactive imitation and online correction. A human supervisor who can intervene when the robot makes mistakes can dramatically improve the sample complexity of imitation. The challenge is minimizing the burden on the human. Uncertainty-aware policies that request help only when confidence is low, shared autonomy frameworks that blend autonomous and human control, and algorithms that propagate corrections backwards in time (to fix the root cause, not just the symptom) are active research areas.

Offline and batch imitation. In many settings (autonomous driving, surgical robotics), expert data exists but interactive expert querying is impossible. Offline imitation learning must extract a policy from a fixed dataset of trajectories, without environment interaction. The challenge is distribution shift: without online feedback, the learner cannot estimate its own test-time state distribution. Conservative approaches that penalize deviation from the data distribution, similar to those in offline RL, are necessary.

Toward a unified framework. The field currently offers a menu of methods with unclear relationships. A unified theoretical framework that characterizes imitation learning as divergence minimization between trajectory distributions would clarify trade-offs: behavioral cloning minimizes the forward KL divergence $D_{\mathrm{KL}}(d_{\pi^*} \| d_{\pi_\theta})$ over states; GAIL minimizes the JS divergence over occupancy measures; MaxEnt IRL minimizes the reverse KL divergence. Understanding which divergence is appropriate for which setting, and developing algorithms that can interpolate among them, remains open.

9.10 Summary

Imitation learning transforms the robot learning problem from "discover through trial and error" to "learn by observing." Behavioral cloning provides a simple baseline that fails dramatically under distribution shift. DAgger patches this failure through online expert querying, at the cost of requiring an interactive expert. Maximum entropy IRL recovers reward functions that explain expert behavior, with elegant connections to statistical mechanics and variational inference. GAIL unifies IRL and RL into a single adversarial objective, connecting imitation to the rich theory of f-divergences and GANs. Learning from play and action chunking address the practical realities of data collection and temporal consistency. The frontier lies in few-shot generalization, cross-embodiment transfer, interactive correction, and a unified theoretical understanding that can guide the choice of imitation objective for a given domain.

Chapter 10

Embodied Reinforcement Learning

10.1 Motivation

If imitation learning is learning by watching, reinforcement learning is learning by doing. An embodied RL agent is dropped into a physical world—or a simulation thereof—and must discover, through trial and error, a policy that achieves its goals. The body introduces constraints that are absent in board games, Atari, or text-based environments: high-dimensional continuous state spaces (vision, proprioception, tactile sensing), continuous action spaces (joint torques, end-effector velocities), sparse reward signals (success at the end of a long sequence), and the relentless forward march of physical time.

The promise of embodied RL is autonomy without demonstration. The challenge is that the trial-and-error loop, when each trial demands seconds or minutes of real-world interaction, scales poorly. Simulation offers an escape hatch: train in a fast, parallelized virtual world and transfer the learned policy to reality. But simulation is always an approximation, and the gap between simulation and reality—the *sim-to-real gap*—is the central problem of embodied RL.

10.2 Core Concepts and Key Papers

10.2.1 The Embodied Markov Decision Process

We model the robot's interaction with the world as a Markov Decision Process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathbb{R}, \gamma, \rho_0)$. In the embodied setting, the state $\mathcal{S}$ is high-dimensional: RGB images from head-mounted or wrist-mounted cameras, depth maps, joint positions and velocities, end-effector poses, and tactile readings. The action space $\mathcal{A}$ is continuous—joint torques for torque-controlled robots, desired joint angles for position-controlled robots, or Cartesian end-effector displacements. The transition dynamics $\mathcal{P}$ are the equations of rigid-body physics with contacts, friction, and deformation. The reward $\mathbb{R}(s_t, a_t)$ is task-specific and often sparse: 1 when the peg is inserted, 0 otherwise. The discount factor $\gamma \in [0, 1)$ and initial state distribution ρ_0 complete the specification.

The policy $\pi_\theta(a \mid s)$ maps observations to action distributions. The standard RL objective is the expected discounted return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta, \rho_0} \left[\sum_{t=0}^{\infty} \gamma^t \, \mathbb{R}(s_t, a_t) \right]. \tag{10.1}$$

In practice, the agent rarely has access to the full state s_t. Camera sensors suffer from occlusions, limited field of view, and noise. We therefore operate in the more realistic **P**artially **O**bservable MDP (POMDP) formulation, where the agent receives an observation $o_t = \mathcal{O}(s_t)$ rather than s_t itself. The optimal policy in a POMDP conditions on the entire observation-action history, or equivalently on the *belief state*:

$$b_t = P(s_t \mid o_{1:t}, a_{1:t-1}). \tag{10.2}$$

Recurrent neural architectures (LSTMs, GRUs, or Transformer decoders with causal masks) approximate this belief state by maintaining a hidden state h_t that is updated at each step: $h_t = f(h_{t-1}, o_t, a_{t-1})$. The policy then conditions on h_t: $\pi_\theta(a_t \mid h_t)$. This is the design principle behind recurrent RL agents, which dominate in visually-rich, partially observable embodied tasks.

10.2.2 Sim-to-Real Transfer: The Central Problem

Let $\mathcal{M}_{\text{sim}} = (\mathcal{S}, \mathcal{A}, \mathcal{P}_{\text{sim}}, \mathbb{R}, \gamma, \rho_0)$ be the simulated MDP and $\mathcal{M}_{\text{real}} = (\mathcal{S}, \mathcal{A}, \mathcal{P}_{\text{real}}, \mathbb{R}, \gamma, \rho_0)$ the real MDP. The state and action spaces are shared; the dynamics differ. A policy trained in simulation sees $\mathcal{P}_{\text{sim}}$; deployed in reality, it encounters $\mathcal{P}_{\text{real}} \neq \mathcal{P}_{\text{sim}}$. The performance drop is often catastrophic.

The robust policy objective formulates this as a worst-case optimization:

$$\theta^* = \arg\max_\theta \min_{\mathcal{P} \in \mathfrak{D}} \mathbb{E}_{\tau \sim \mathcal{P}, \pi_\theta} \left[\sum_t \gamma^t \, \mathbb{R}(s_t, a_t) \right], \tag{10.3}$$

where $\mathfrak{D}$ is an uncertainty set of possible dynamics around the simulator. This is a minimax problem: the policy maximizes return against the most adversarial dynamics within $\mathfrak{D}$. In the worst case, the adversary can make small perturbations to mass, friction, or geometry that completely defeat the policy. Direct minimax optimization is intractable for the continuous, high-dimensional dynamics of robotics; we need approximations.

Domain Randomization

Domain randomization replaces the worst-case adversary with a distributional opponent. Instead of min over dynamics, we maximize the expectation over a distribution of simulator parameters:

$$\theta^* = \arg\max_\theta \mathbb{E}_{\xi \sim \Xi} \left[\mathbb{E}_{\tau \sim \mathcal{P}_\xi, \pi_\theta} \left[\sum_t \gamma^t \, \mathbb{R}(s_t, a_t) \right] \right]. \tag{10.4}$$

The randomization parameters ξ encompass everything that might differ between simulation and reality:

- **Physics:** friction coefficients (μ), object masses (m), joint damping (b), stiffness (k), restitution coefficients.
- **Visuals:** lighting direction, intensity, and color temperature; object textures and materials; camera pose and intrinsics; background clutter.
- **Actuation:** motor gains, latency between commanded and actual torques, sensor noise and delay.

At each training episode, a new parameter vector $\xi \sim \Xi$ is sampled and the simulator is configured accordingly. The policy sees a different "reality" every episode and must learn strategies that work across the entire support of Ξ. The crucial theoretical intuition: if the real dynamics fall within the support of Ξ (i.e., $\mathcal{P}_{\text{real}} \in \text{support}(\Xi)$), then a policy trained to maximize expected return over Ξ will also perform well under $\mathcal{P}_{\text{real}}$. In practice, Ξ is a designer-chosen distribution—uniform over hand-specified ranges, or Gaussian around nominal values.

The breakthrough demonstration of domain randomization came with OpenAI's robotic hand solving a Rubik's cube (Akkaya et al., 2019). The entire training occurred in simulation with randomized visual properties, physics parameters, and actuation noise. The trained policy transferred to a physical Shadow Hand, which could solve a physical Rubik's cube despite never having touched one during training. Key engineering: **A**utomatic **D**omain **R**andomization (ADR). ADR dynamically expands the randomization ranges as the agent improves. If the agent achieves high success rate with friction in $[0.5, 1.5]$, ADR widens the range to $[0.3, 1.7]$. This curriculum pushes the boundary of the support outward, gradually encompassing a wider range of real-world conditions.

System Identification

An alternative to randomization is estimation. If we can identify the parameters of $\mathcal{P}_{\text{real}}$ from real data, we can train exclusively on a simulator configured with those identified parameters. Given trajectory data $\{(s_t^{\text{real}}, a_t, s_{t+1}^{\text{real}})\}$ from the real robot, system identification solves:

$$\xi^* = \arg\min_\xi \; \mathbb{E}\Big[\big\| s_{t+1}^{\text{real}} - f_{\text{sim}}(s_t^{\text{real}}, a_t; \xi)\big\|^2\Big]. \tag{10.5}$$

This is a supervised regression problem: predict the real next state given the simulator's parameterized forward dynamics. The identified parameters ξ^* are then fixed, and a policy is trained in the calibrated simulator. The limitation is practical: the space of physical parameters is vast, correlations among parameters create identifiability issues, and unmodeled effects (compliance in cables, temperature-dependent friction) cannot be expressed within a rigid-body simulator's parameterization. System identification provides a point estimate of the simulator parameters, but the real world always has residual mismatch.

Domain Adaptation at the Feature Level

A third approach circumvents dynamics matching altogether by learning a feature representation that is invariant to the sim-to-real distinction. Let $\phi : \mathcal{S} \to \mathbb{R}^d$ be a learned encoder. We train ϕ such that the distribution of $\phi(s)$ is indistinguishable between simulation and reality. A GAN-based method achieves this: a discriminator $D_\psi : \mathbb{R}^d \to [0, 1]$ is trained to classify whether a feature vector came from simulation or reality, while the encoder ϕ is trained adversarially to fool the discriminator. The aligned feature space then serves as the input to a downstream policy, which is trained only on simulation features.

The formulation is:

$$\min_\phi \max_\psi \; \mathbb{E}_{s\sim\text{sim}}\big[\log D_\psi(\phi(s))\big] + \mathbb{E}_{s\sim\text{real}}\big[\log(1 - D_\psi(\phi(s)))\big]. \tag{10.6}$$

At convergence, the feature distributions $p(\phi(s) \mid \text{sim})$ and $p(\phi(s) \mid \text{real})$ are matched. Since the policy operates on $\phi(s)$, it becomes independent of the visual domain. This is an instance of the broader framework of domain-adversarial training, which has been applied to sim-to-real transfer for visual robot manipulation and navigation.

10.2.3 Asymmetric Actor-Critic

Pinto et al. (2017) introduced a conceptually elegant solution to the partial observability of realistic robot sensors. In simulation, we have access to the full privileged state s_t^{priv}—exact object poses, velocities, contact forces, and distances. A real robot never has this information; it only receives o_t. The asymmetric actor-critic architecture exploits this asymmetry during training only.

The critic $V_\omega(s_t^{\text{priv}})$ receives the privileged state and learns an accurate value function from it. The actor $\pi_\theta(a_t \mid o_t)$ receives only the realistic observation and is trained via gradients from the privileged critic:

$$\nabla_\theta J \approx \mathbb{E}\Big[\nabla_\theta \log \pi_\theta(a_t \mid o_t) \cdot A(s_t^{\text{priv}}, a_t)\Big], \tag{10.7}$$

where $A(s, a) = Q(s, a) - V(s)$ is the advantage computed from privileged information. At deployment, the critic is discarded; only the actor, which has learned to map realistic observations to actions informed by a high-quality value function, is used. The critic provides better training signal because it can evaluate actions against the true state; the actor learns to approximate the same behavior from impoverished observations. This approach is a form of *information asymmetry* during training, enabling the policy to learn behaviors that would be unidentifiable from observations alone.

10.2.4 Recurrent RL for Partial Observability

In visually randomized environments, the agent cannot infer the full state from a single image—the image is corrupted by randomized lighting, textures, and camera parameters. A recurrent policy $h_t = \text{LSTM}(h_{t-1}, \phi(o_t))$ accumulates information over time, approximating the belief state update:

$$h_t \approx b_t = P(s_t \mid o_{1:t}, a_{1:t-1}). \tag{10.8}$$

The recurrent state h_t encodes the history of observations that the agent has integrated. With enough steps of observation, the agent can infer physical parameters that are constant across the episode (e.g., the friction coefficient of the floor, the mass of an object) and adapt its policy accordingly. This is *online system identification* performed implicitly by the recurrent network. The approach is often called RRL (Recurrent RL) or DRQ (Deep Recurrent Q-learning) and is a standard component of sim-to-real RL stacks.

10.3 Key Techniques

10.3.1 Exploration in Sparse Reward Settings

Sparse rewards—receiving a positive signal only upon task completion—are the norm in embodied tasks. RL agents with undirected exploration policies can take astronomically long to discover the rewarding region. Three families of solutions have emerged.

Intrinsic motivation via prediction error. Random Network Distillation (RND) by Burda et al. (2019) provides a curiosity signal without requiring a learned dynamics model. A randomly initialized target network f is fixed throughout training. A predictor network $\hat{f}_\psi$ is trained to match f on all visited states. The intrinsic reward is the prediction error:

$$r_t^{\text{int}} = \left\| \hat{f}_\psi(s_t) - f(s_t) \right\|^2. \tag{10.9}$$

States that are novel—far from the training distribution of $\hat{f}_\psi$—produce high prediction error and thus high intrinsic reward. As the agent revisits similar states, $\hat{f}_\psi$ learns to predict them accurately, and the intrinsic reward decays. This creates a natural exploration curriculum: the agent is drawn to novelty and eventually masters each region of the state space.

Count-based exploration. Discretizing the continuous state space via hashing (SimHash, autoencoder hashing) allows counting state visitations. States with low visitation counts receive an exploration bonus $b(s) \propto 1/\sqrt{N(s)}$, where $N(s)$ is the pseudo-count. This is the continuous-state analogue of UCB exploration in bandits and tabular MDPs.

Goal-conditioned exploration with Hindsight Experience Replay. Andrychowicz et al. (2017) proposed HER, a remarkably simple method for multi-goal RL with sparse rewards. The key idea: after each episode, regardless of whether the agent achieved the commanded goal g, it *did* achieve *some* state s_T. Relabel the episode's goal as s_T and assign $+1$ reward. Every trajectory becomes a successful trajectory for its retrospectively assigned goal.

Formally, for each episode $\tau = (s_0, a_0, \ldots, s_T)$ originally executed with goal g, HER adds a relabeled version with goal $g' = s_T$ and reward $\mathbb{R}^{\text{HER}}(s_t, a_t, g') = \mathbf{1}_{\{s_t \approx g'\}}$. The agent learns that reaching any reachable state is valuable when that state is the goal. This converts the sparse-reward exploration problem into a dense supervised learning problem: the replay buffer contains many "successful" trajectories, each showing how to reach some state from some starting point.

10.3.2 Multi-Task Embodied RL

A single-task robot is a fragile robot. Multi-task RL trains a single policy $\pi_\theta(a \mid s, g)$ to perform many tasks, where the goal descriptor g encodes the task identity (a language instruction, a goal

image, or a one-hot task index). The benefits are threefold: shared representations across tasks improve sample efficiency, the policy learns compositional skills, and training on a diverse task distribution prevents catastrophic forgetting.

MT-Opt (Kalakrishnan et al., 2021) demonstrated multi-task RL at scale: 7 real robots performing 10+ manipulation tasks, continuously learning over months. The system used a distributed architecture where robots asynchronously collected data, a central learner updated the policy, and the updated policy was deployed back to all robots. The policy was goal-conditioned with goal images; it learned to grasp diverse objects, place them in containers, and rearrange tabletop scenes.

The training objective for multi-task RL is:

$$J(\theta) = \mathbb{E}_{g\sim\mathcal{G}}\mathbb{E}_{\tau\sim\pi_\theta(\cdot|\cdot,g)}\left[\sum_t \gamma^t \, \mathbb{R}_g(s_t, a_t)\right], \tag{10.10}$$

where $\mathcal{G}$ is the task distribution and $\mathbb{R}_g$ is the task-specific reward. In practice, this is optimized with off-policy algorithms (SAC, TD3) that reuse data across tasks via the replay buffer, possibly with task-conditional critics and actors.

10.4 Connections

Embodied RL inherits the full machinery of deep RL—policy gradients, Q-learning, actor-critic methods, entropy regularization—and adapts it to the physical domain. The sim-to-real transfer problem connects RL to robust optimization (adversarial training), system identification (parameter estimation), and domain adaptation (distribution matching). Domain randomization is effectively *data augmentation for dynamics*: just as random crops and flips improve visual generalization, randomized physics parameters improve dynamical generalization.

The asymmetric actor-critic architecture is a form of *teacher-student* training: the privileged critic is a teacher with access to ground-truth state; the actor is a student that learns from the teacher's evaluations using only realistic observations. This pattern recurs across embodied AI—whenever privileged information is available in training but not at test time, asymmetric architectures provide a natural inductive bias.

Exploration in sparse-reward embodied tasks connects to the broader challenge of efficient exploration in high-dimensional MDPs. RND relates to count-based exploration through the lens of pseudo-counts; HER relates to hindsight reasoning and counterfactual learning.

10.5 Open Problems

Closing the sim-to-real gap. Domain randomization works by brute force: train on everything, hope reality is within the distribution. This is statistically inefficient and provides no formal guarantees. How can we quantify the sim-to-real gap for a given policy? Can we provide probabilistic bounds on the real-world performance drop based on the policy's sensitivity to simulator parameters? This requires advances in sensitivity analysis, uncertainty quantification, and calibrated simulators.

Sample efficiency. State-of-the-art sim-to-real RL still requires millions or billions of simulated episodes. For tasks where simulation is itself slow (soft-body dynamics, fluid simulation, deformable objects), this is infeasible. Model-based RL, which learns a dynamics model from limited interaction and plans within that model, holds promise but suffers from model-bias compounding. Learned world models that are accurate enough for planning in contact-rich settings remain elusive.

Safe exploration. Real robots break. Exploration strategies must be constrained to safe regions of state-action space while still discovering novel behaviors. Safe RL formalizes this with

constrained MDPs: maximize return subject to $\mathbb{E}[\sum_t C(s_t, a_t)] \leq \delta$, where C encodes costs (e.g., joint torque limits, collision penalties). Lagrangian methods, trust-region methods, and learned safety critics are active research directions.

Long-horizon credit assignment. When a task takes 1000 steps and the reward arrives only at the end, assigning credit to individual actions is extremely challenging. Temporal difference learning with discount factors attenuates the signal. Hindsight relabeling (HER) helps but does not solve the fundamental credit assignment problem for tasks where intermediate progress is hard to measure. Hierarchical RL, with temporal abstractions learned or specified, may provide a path forward.

Continual and lifelong learning. A deployed robot will encounter new tasks, new objects, and new environments throughout its lifetime. It must learn these without forgetting previous skills and without full retraining. Elastic weight consolidation, progressive networks, and memory-based approaches have been demonstrated in simulation; deploying them on physical robots for months or years remains an open engineering challenge.

Hardware-aware policy design. Most RL algorithms assume idealized actuation: instantaneous, accurate torque delivery. Real actuators have backlash, saturation, rate limits, and stiction. Policies must be robust to these effects, and the RL formulation should incorporate actuation constraints directly into the optimization. Constrained policy parameterizations (energy-based policies, differentiable filters) and actuator-aware simulators are under-explored.

10.6 Summary

Embodied reinforcement learning is the science of learning to act through physical interaction. The MDP formalism provides a clean mathematical foundation; the POMDP extension acknowledges the reality of partial observability. Sim-to-real transfer is the defining challenge, addressed through domain randomization (train on everything), system identification (estimate and match), and domain adaptation (learn invariant features). Asymmetric actor-critic and recurrent architectures exploit privileged information during training to produce robust policies for deployment. Exploration strategies based on intrinsic motivation, counts, and hindsight relabeling overcome the sparsity of real-world rewards. The field is converging on a standard toolkit—domain randomization, entropy-regularized actor-critic, recurrent encoders, and hindsight relabeling—that has achieved remarkable sim-to-real results in dexterous manipulation, locomotion, and mobile manipulation.

Part V

Foundation Models & Frontiers

Chapter 11

Foundation Models for Robotics

11.1 Motivation

The classical robotics stack is modular by necessity: one module for visual perception, one for state estimation, one for task planning, one for motion planning, one for low-level control. Each module is designed for a specific robot, a specific sensor suite, and a specific set of tasks. Changing any component requires re-engineering the interfaces between modules. This architecture has produced remarkable systems in factory automation, but it does not scale to the diversity and unpredictability of unstructured human environments.

A different vision has crystallized in the last three years: a single, large neural model that maps sensory observations and natural language instructions directly to robot actions, trained on internet-scale data and fine-tuned on robot demonstration data. This model would generalize across tasks, objects, environments, and even robot embodiments—the "GPT moment" for robotics. The models pursuing this vision are called **VLAs (Vision-Language-Action models)** or simply **foundation models for robotics**. They share a common architecture: a vision encoder to process images, a language model backbone to fuse vision and language, and an action head to produce control commands.

11.2 Core Concepts and Key Papers

11.2.1 PaLM-E: Embodied Multimodal Language Model

Driess et al. (2023) introduced PaLM-E, the first large-scale demonstration that a language model could ingest multimodal embodied observations and directly output actions. The architecture is elegantly compositional.

Input encoding. Three modalities are projected into a shared token space:

- **Images:** Each camera frame is processed by a Vision Transformer (ViT), producing a sequence of patch embeddings. These are linearly projected to the embedding dimension of the language model.
- **State estimates:** Continuous proprioceptive readings—joint angles, end-effector pose, gripper state—are encoded by a learned MLP into a small number of embedding vectors.
- **Text:** The language instruction and any task context are tokenized via the standard language model tokenizer.

All tokens are concatenated into a single sequence:

$$\mathbf{x} = \big[\text{img}_1, \ldots, \text{img}_K, \text{state}_1, \ldots, \text{state}_M, \text{text}_1, \ldots, \text{text}_N\big]. \tag{11.1}$$

Decoder backbone. PaLM-E uses a decoder-only Transformer (PaLM) that autoregressively predicts the next token in the sequence. The model is trained with the standard language modeling objective—next-token prediction with cross-entropy loss:

$$\mathcal{L} = -\sum_{t} \log P_\theta(x_t \mid x_{<t}). \tag{11.2}$$

Action representation. Actions are treated as text. For a robot arm with a parallel-jaw gripper, an action is a string: `"128 64 200 50 0 0 1"` representing the target joint angles,

end-effector displacement, and gripper open/close command. Continuous values are discretized into bins (e.g., 256 bins per degree of freedom). The Transformer predicts each action token just as it predicts any other text token.

Training data mixture. The breakthrough of PaLM-E lies in its training recipe, which combines three qualitatively distinct datasets:

1. **Internet-scale vision-language data:** Web images with captions, visual question answering, image-text pairs from the public internet. This teaches the model general visual concepts, object recognition, spatial relations, and language grounding.

2. **Robot demonstration data:** Observation-action trajectories collected from real robots performing manipulation tasks. This teaches the model the mapping from visual states + instructions to motor commands.

3. **Scene description data:** Images annotated with object bounding boxes and labels rendered as text ("there is a red block at (0.3, 0.5) and a blue cup at (0.7, 0.2)"). This teaches spatial reasoning and object-level understanding.

Key result—co-training synergy. A model trained on all three datasets outperforms models trained on any subset. The internet vision-language data provides rich visual representations; the robot data provides embodiment-specific motor skills; the scene descriptions provide structured spatial reasoning. The model that has encountered "red apple" and "grasp the cup" in millions of captioned images can recognize a red apple in a robot's camera feed and know to grasp it. Knowledge transfers across modalities and across domains.

PaLM-E demonstrated zero-shot generalization to unseen object combinations, the ability to follow multi-step language instructions, and emergent planning behaviors such as "if I knock over the cup, I should pick it up and place it upright."

11.2.2 RT-2: Vision-Language-Action Model

Brohan et al. (2023) pushed the vision-language-action paradigm further with RT-2, a direct successor to RT-1. The core methodology is conceptually straightforward: take a state-of-the-art vision-language model (PaLI-X or PaLM-E), co-fine-tune it on both internet-scale vision-language data and robot trajectory data, and treat actions as language tokens. The result is a single model that can answer visual questions, caption images, and control a robot—all with the same weights.

Action tokenization. RT-2's action space is the Cartesian displacement of the end-effector in 6 degrees of freedom: $\Delta x, \Delta y, \Delta z$ (translation), $\Delta r_x, \Delta r_y, \Delta r_z$ (rotation). Each dimension is discretized into 256 uniformly spaced bins, producing 6 integer tokens. A seventh binary token controls the gripper (1 for close, 0 for open). A trajectory command is thus a sequence of 7 integer tokens.

The radical implication: the same Transformer parameters that predict `"the"` after `"The cat sat on"` also predict `"128 64 200 50 10 5 1"` after `"pick up the apple"`. The distinction between language tokens and action tokens is erased in training; both are just tokens in a vocabulary.

Co-fine-tuning. A critical design choice is *not* to fine-tune solely on robot data. Doing so would catastrophically forget the visual and linguistic knowledge acquired during internet pre-training. Instead, RT-2 interleaves batches of robot trajectory data with batches of the original VLM data (image captioning, VQA). The model continues to practice visual recognition and language understanding throughout the fine-tuning process. This co-fine-tuning prevents forgetting while enabling the model to learn the new action-prediction task.

Emergent capabilities. RT-2 exhibited behaviors not present in its training data:

- **Symbol understanding:** The instruction "move the block to the smiley face" succeeds even when smiley-face-labeled objects never appeared in robot training. The model learned to recognize smiley faces from internet image data and to move objects from robot data; it composes these skills at test time.
- **Semantic reasoning:** "Push the cola can towards the blue block" where cola cans were seen in internet images but never in robot training. The VLM's semantic knowledge of cola cans transfers to the manipulation domain.
- **Chain-of-thought reasoning:** The model can emit intermediate reasoning in natural language before producing action tokens: `"I should identify the red apple. The apple is at the center of the table. I will move my arm to (0.3, 0.5)..."` followed by action tokens. This explicit reasoning improves long-horizon task performance by breaking the task into logical subgoals.

The scaling hypothesis. RT-2 embodies a bet: more internet data yields better visual recognition, which in turn yields better robot generalization. More robot data yields better motor skills. The combination produces capabilities that neither data source alone can support. RT-2 was trained on a web-scale VLM (billions of image-text pairs) plus 130,000 real robot trajectories collected over 17 months from a fleet of 13 robots.

11.2.3 Octo: An Open-Source Generalist Policy

Ghosh et al. (2024) released Octo as an open-source, Transformer-based generalist robot policy trained on the **Open X-Embodiment (OXE)** dataset—the largest and most diverse robot dataset ever assembled, containing over 1 million trajectories from 22 distinct robot embodiments across 20 research institutions. Octo represents a shift toward community-driven, reproducible foundation models for robotics.

The cross-embodiment challenge. Different robots have different action spaces: some are 7-DoF arms, others are 6-DoF arms with mobile bases, yet others use suction grippers or parallel-jaw grippers. Camera configurations differ: wrist-mounted vs. head-mounted vs. third-person views. Control frequencies range from 10 Hz to 50 Hz. A generalist policy must handle this heterogeneity.

Octo's approach. Actions are predicted via a diffusion head conditioned on observation tokens. Rather than discretizing actions, Octo uses continuous action prediction with a conditional diffusion model: noise is iteratively denoised across K diffusion steps to produce the action trajectory. The observation context (images, proprioception) and task specification (language instruction or goal image) condition the denoising process via cross-attention.

All robots' action spaces are padded to a universal maximum dimension (e.g., 12 DoF). For a 7-DoF robot, the first 7 dimensions are active and the remaining 5 are masked. The model learns which dimensions correspond to which embodiment through the data distribution.

Action chunking. Octo predicts chunks of future actions (typically 10–25 steps) rather than individual actions, following the ACT paradigm. This reduces the effective control frequency and smooths the action trajectory through temporal ensembling. At deployment, predictions from overlapping chunks are combined with an exponential weighting scheme.

Design philosophy. Octo is designed as a *pre-trained base model* for robotics—analogous to a pre-trained language model like Llama. Users fine-tune Octo on their specific robot and tasks with a small amount of demonstration data (50–200 demonstrations). The pre-training provides strong visual representations, a repertoire of basic motor skills, and an understanding of object affordances. Fine-tuning adapts these general capabilities to the specific kinematics and tasks of the target robot.

11.2.4 OpenVLA: An Open Vision-Language-Action Model

Kim et al. (2024) further democratized VLA models with OpenVLA, a 7-billion-parameter model built from completely open components: a SigLIP vision encoder (ViT-SO400M) plus the Llama 2 language model backbone. The entire model is fine-tuned using LoRA (Low-Rank Adaptation), making it feasible to train and deploy on academic hardware.

Architecture. The vision encoder produces patch embeddings that are projected through a lightweight MLP into the Llama 2 embedding space. These projected visual tokens, concatenated with text tokens (the language instruction), are fed into the Llama decoder. The model predicts actions as discretized tokens, following RT-2's convention. The entire pipeline is:

$$\text{(Image)} \xrightarrow{\text{SigLIP}} \text{(Visual tokens)} \xrightarrow{\text{MLP}} \text{(Projected tokens)} \xrightarrow{\text{Llama 2 + LoRA}} \text{(Action tokens)}. \tag{11.3}$$

Training efficiency. LoRA freezes the pre-trained Llama 2 weights and inserts low-rank adapters (pairs of matrices $A \in \mathbb{R}^{d\times r}, B \in \mathbb{R}^{r\times d}$ with $r \ll d$) at each attention layer. Only these adapter parameters and the vision projector are trained, reducing the trainable parameter count by orders of magnitude. Training OpenVLA on the full OXE dataset takes approximately 2–3 days on 16 A100 GPUs—accessible to academic labs.

Deployment. OpenVLA can be deployed on real robots with standard inference frameworks. At inference time, the model receives an RGB image, proprioceptive state, and a language instruction ("place the carrot on the cutting board"). It autoregressively generates action tokens at 5–10 Hz. The model has been validated on multiple commercial robot platforms, including Franka Emika Panda and WidowX arms.

11.2.5 π_0: Flow Matching for Continuous Actions

The Physical Intelligence team (2024) introduced π_0, which addresses a conceptual limitation of token-based action representations: discretization introduces unnecessary quantization error and loses the natural continuity of physical motion. π_0 generates continuous actions via **flow matching**, a generative modeling framework closely related to diffusion models but with simpler training dynamics.

Flow matching formulation. Let $x_0 \sim \mathcal{N}(0, I)$ be a noise sample and x_1 be a target action from the training data. Flow matching learns a time-dependent vector field $v_\theta(x_t, t \mid o, \ell)$ conditioned on observations o and language instruction ℓ, where $x_t = (1-t)x_0 + tx_1$ is a linear interpolation. The training objective is:

$$\mathcal{L}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1],(o,\ell,x_1)\sim\mathcal{D},x_0\sim\mathcal{N}}\Big[\big\|v_\theta(x_t, t \mid o, \ell) - (x_1 - x_0)\big\|^2\Big]. \tag{11.4}$$

The target vector field $x_1 - x_0$ is the constant velocity that transports noise samples to data samples along straight trajectories. At inference, starting from noise $x_0 \sim \mathcal{N}(0, I)$, we solve the ordinary differential equation $\frac{dx}{dt} = v_\theta(x, t \mid o, \ell)$ to obtain the predicted action x_1.

Advantages over discretization. Flow matching produces inherently smooth, continuous action trajectories. The sampling process can be truncated early with a smaller number of integration steps, trading quality for speed. This is critical for real-time robot control where inference latency must be under 50–100 milliseconds. π_0 has demonstrated dexterous tasks including folding laundry, bussing tables, and assembling components—tasks that require precise, smooth, and adaptive motion.

Heterogeneous training. Like Octo, π_0 is trained on a mixture of robot data from diverse embodiments. The flow matching framework naturally handles variable-dimensional action spaces through padding and masking. The model is pre-trained on a large heterogeneous dataset and fine-tuned for specific embodiments and tasks.

11.2.6 The Common Architecture

All these models share a unified formula:

1. **Start with a pre-trained model.** A vision-language model (PaLI, PaLM, Llama) with billions of parameters trained on internet-scale data provides the foundation.

2. **Add a vision encoder.** ViT or SigLIP transforms camera images into token sequences.

3. **Represent actions as predictable outputs.** Either discretized tokens (RT-2, OpenVLA) or continuous signals via diffusion/flow matching (Octo, π_0).

4. **Co-train or co-fine-tune.** Mix robot data with original pre-training data to prevent catastrophic forgetting and enable knowledge transfer.

5. **Generalization through pre-training.** The model generalizes to new tasks because it already understands objects, language, and spatial relations from its pre-training data.

11.3 Key Techniques

The training of foundation models for robotics requires careful handling of heterogeneous data. Key techniques include:

Action normalization and padding. Different robots produce actions in different ranges and units. Normalizing all actions to zero mean and unit variance per dimension, and padding to a universal dimension, allows a single model to handle all embodiments.

Observation tokenization. Images are tokenized via ViT; proprioceptive states are encoded via small MLPs; language instructions are tokenized via the language model's native tokenizer. The challenge is balancing the token budget—too many image tokens drown the action and text tokens; too few lose visual detail.

Co-training strategies. Data from different sources appear in different proportions. Balancing the mixture to prevent any single dataset from dominating requires careful tuning. Techniques include: per-dataset temperature scaling, gradient accumulation with separate losses, and progressive training (starting with easier data, adding harder data later).

Inference-time techniques. Chain-of-thought prompting enables models to reason through tasks before acting. Temporal ensembling smooths action predictions across time. Classifier-free guidance (from diffusion literature) balances fidelity to the instruction with diversity of actions.

11.4 Connections

Foundation models for robotics unify several threads in machine learning. They are direct descendants of large language models, borrowing the Transformer architecture and the pre-then-fine-tune paradigm. They connect to vision-language models (CLIP, Flamingo, PaLI) via shared visual encoders and multimodal fusion techniques. They connect to generative modeling (diffusion, flow matching) through continuous action generation. And they connect to imitation learning and offline RL through their training on demonstration data.

The theoretical underpinning is the *manifold hypothesis* for embodied interaction: the space of useful robot behaviors, across tasks and embodiments, lies on a low-dimensional manifold embedded in the high-dimensional space of sensory observations and actions. Pre-training on diverse data discovers this manifold; fine-tuning maps a specific embodiment onto it. The empirical evidence—that co-training on internet data improves robot performance—suggests that the visual-linguistic knowledge captured in VLM pre-training partially spans this manifold.

11.5 Open Problems

Data scaling. Robot demonstration data is measured in hundreds of thousands of trajectories. Language data is measured in trillions of tokens. The six-order-of-magnitude gap is the fundamental limitation of current robot foundation models. Closing it requires: more robots collecting data, better simulation-based data generation, mining of human video data, and data-efficient learning algorithms that extract maximal value from each trajectory.

Cross-embodiment generalization. Current models handle different embodiments through padding and masking—a practical but unsatisfying solution. A true generalist model would understand embodiment differences at a semantic level: "I have a suction gripper, not fingers; I should approach the object from above rather than from the side." This requires learning an explicit model of the robot's own embodiment and reasoning about how it constrains action selection.

Real-time control. The compute requirements of 7B+ parameter Transformers are at odds with the 50–100 Hz control loops required for dexterous manipulation. Distillation into smaller models, speculative decoding, and specialized hardware (edge TPUs, Jetson) are necessary for practical deployment. Model quantization and pruning can reduce latency at the cost of performance; finding the Pareto frontier is an active engineering problem.

Reliability and safety. A foundation model that succeeds 85% of the time on a benchmark is not deployable in a home or factory, where failures can damage property or injure people. Formal verification of neural policies is in its infancy. Runtime monitoring—detecting when the model is uncertain or producing anomalous actions—combined with fallback to safe controllers is a practical mitigation, but not a solution.

Evaluation. Current benchmarks for robot foundation models are task-specific and laboratory-bound. A standardized, community-maintained benchmark spanning diverse embodiments, tasks, and environments does not yet exist. Without one, comparing models and measuring progress is difficult. Efforts like OXE and the OpenVLA evaluation suite are steps in this direction.

Understanding emergent behavior. RT-2's chain-of-thought reasoning and PaLM-E's planning behaviors were not explicitly programmed; they emerged from scale. We lack a scientific framework for predicting, controlling, and verifying emergent capabilities in embodied models. Interpretability tools for the action-generation pathway are rudimentary compared to those for language models.

11.6 Summary

The foundation model paradigm has transformed the robot learning landscape in under three years. PaLM-E demonstrated that language models could ingest multimodal observations and output actions. RT-2 proved that co-fine-tuning on vision-language and robot data produces emergent generalization. Octo and OpenVLA made the technology accessible through open-source releases. π_0 advanced the frontier with continuous action generation via flow matching. The common thread—pre-train on internet data, fine-tune on robot data, represent actions as predictable tokens or continuous signals—has produced the most general-purpose robot policies ever built. The remaining challenges are data scale, cross-embodiment reasoning, real-time inference, and safety. The trajectory is clear: foundation models are the most promising path toward robots that can operate in the unstructured, diverse, and unpredictable environments of human daily life.

Chapter 12

Frontiers and Future Directions

12.1 The State of Embodied AI

As of 2025–2026, embodied AI stands at a peculiar juncture. Laboratory demonstrations are spectacular: robots fold laundry, solve Rubik's cubes, navigate office buildings, and follow natural language instructions. Yet no general-purpose home robot exists. The gap between laboratory capability and real-world deployment is the central tension of the field. This chapter surveys what we can do, what we cannot, and where the field is heading.

12.2 Capability Gaps

12.2.1 Dexterous Manipulation

Two-finger parallel-jaw grasping of rigid objects in tabletop settings is largely solved. A standard robot arm with a wrist-mounted camera and a learned grasping policy can pick up novel objects with 90%+ success rates. But the human hand has 21 degrees of freedom, not 1. Tasks involving in-hand manipulation—rotating an object between fingers, reorienting a key to insert it into a lock, threading a needle—remain beyond the reach of current systems.

The contact dynamics challenge. Contact is the fundamental obstacle. When a robot finger touches an object, the dynamics equations change discontinuously: a previously free object becomes constrained; a previously constrained object slips. These discontinuities create sharp ridges in the reward landscape that gradients cannot cross. Reinforcement learning algorithms struggle to discover contact-rich strategies because the path from an initial random policy to a policy that makes and breaks contacts intentionally passes through regions of extremely steep (or zero) gradient.

Differentiable simulation offers one route forward. Simulators like Isaac Gym, Brax, and MJX compute exact or approximate gradients through contact dynamics, enabling gradient-based optimization of dexterous policies with massive parallelization (thousands of environments simultaneously on a single GPU). Coupled with domain randomization for sim-to-real transfer, differentiable simulation has produced policies for in-hand rotation, assembly, and dexterous tool use that transfer to physical hardware.

Tactile sensing. Vision cannot see through the robot's own fingers. Tactile sensors—GelSight, DIGIT, and their successors—provide high-resolution contact images that reveal slip, texture, and force distribution. Learned policies that fuse vision and touch outperform vision-only policies on contact-rich tasks by detecting incipient slip and adjusting grip force adaptively. The challenge is that tactile sensing is absent from most robot datasets; scaling tactile data collection is an unmet need.

12.2.2 Mobile Manipulation

Navigation and manipulation, studied in isolation for decades, must be integrated for real-world tasks. A robot that navigates to a kitchen but cannot open the refrigerator is useless. A robot that can grasp objects but cannot navigate to them is equally useless. Mobile manipulation combines these capabilities into the full fetch-and-carry pipeline.

The state space explosion. The combined state space is the Cartesian product of the base configuration ($\mathbb{R}^2 \times SO(2)$ for a differential-drive base, or $\mathbb{R}^3 \times SO(3)$ for a flying base), the

arm configuration ($\mathbb{R}^7$ to $\mathbb{R}^{10}$ for standard arms), and the object state. Jointly optimizing base placement and arm motion is computationally expensive. The base determines reachability: a badly placed base renders even the best arm motion incapable of grasping the target.

Current approaches. The predominant strategy is hierarchical: a navigation policy selects a base goal pose; the base drives there; a manipulation policy executes the grasp. This decouples the problem but loses the opportunity for joint optimization. End-to-end approaches that learn a unified policy $\pi(a_{\text{base}}, a_{\text{arm}} \mid o)$ exist but suffer from the curse of dimensionality. TidyBot (Wu et al., 2023) demonstrated an alternative paradigm: use an LLM to parse a user's preferences ("put clothes on the chair, toys in the drawer"), generate a plan in natural language, and execute it with pre-trained navigation and manipulation skills. The LLM serves as a semantic planner, selecting from a library of learned motor primitives.

12.2.3 Whole-Body Control

Humanoid robots—Tesla Optimus, Figure 02, 1X NEO, Boston Dynamics Atlas—require coordinating legs, arms, torso, and head simultaneously, often exceeding 30 degrees of freedom. The problem is not merely high-dimensional; it is coupled. Kicking a ball involves whole-body momentum transfer; carrying a heavy object requires coordinated leg and arm compensation; climbing stairs involves dynamic balance during arm motion.

Reinforcement learning for humanoids. The standard pipeline mirrors quadrupedal locomotion: train in Isaac Gym with thousands of parallel environments, domain randomize over terrain, friction, and payload parameters, and transfer to physical hardware. This pipeline has produced humanoid policies for walking, running, backflipping, and parkour on uneven terrain. Extending it to whole-body manipulation—locomoting to an object, picking it up while maintaining balance, and carrying it to a destination—is the current frontier.

The embodiment gap. Human demonstration data is abundant (Ego4D, Epic Kitchens, YouTube) but the kinematics of human bodies differ fundamentally from those of humanoids. Human hands have 21 DoF; most robot hands have fewer. Human arms have shoulder-elbow-wrist kinematics; robot arms have revolute joints with different link lengths and joint limits. Transferring human demonstrations to humanoid policies requires retargeting: mapping human motion capture to humanoid joint trajectories. Optimization-based retargeting (minimizing keypoint distances, matching end-effector trajectories) works for structured settings but struggles with contact-rich interactions.

12.2.4 Long-Horizon Autonomy

Current autonomous task execution horizons are measured in minutes. A home robot needs to operate for days without failure. The bottleneck is not task execution but error recovery. When an object slips from the gripper, when a door is unexpectedly closed, when an obstacle blocks the path—most deployed robots either do not notice the anomaly or cannot recover from it.

Anomaly detection and recovery. Learned recovery policies (Lang et al., 2024) monitor visual and proprioceptive observations for deviations from expected trajectories. When the deviation exceeds a threshold, the robot interrupts the current skill, executes a recovery behavior (e.g., re-grasping the object, re-localizing, calling for human assistance), and resumes the task. The recovery skills themselves can be learned from replay buffer data: the failures naturally collected during RL training provide examples of anomalous states from which recovery is needed.

Hierarchical planning with replanning. Task and motion planning (TAMP) frameworks combine symbolic planning over discrete task states with continuous motion planning. When a symbolic action fails (e.g., "pick(x)" fails because x is not where expected), the planner replans at the symbolic level, choosing an alternative sequence of actions. The challenge is that symbolic planning requires a symbolic state space, which must be extracted from raw sensor data through perception—a brittle pipeline in unstructured environments.

12.2.5 Natural Interaction

Robots deployed in human environments must interact with people safely, predictably, and comfortably. This requires three capabilities:

Human intent prediction. Before acting, the robot should anticipate what nearby humans will do. Models of pedestrian trajectory prediction (Social-GAN, Trajectron++) learn from datasets of human motion to forecast future trajectories conditioned on social context. The latent variables in these models capture the unobserved goals and preferences that drive human behavior.

Robot intent communication. The robot must communicate its own intent to humans. A robot arm that moves suddenly without warning is perceived as threatening. Techniques from animation (anticipatory motion, gaze direction, explicit gesturing) can be incorporated into robot motion planning to make intent legible. The optimization formulation adds a legibility term to the motion cost:

$$C(\tau) = C_{\text{task}}(\tau) + \lambda \cdot C_{\text{legibility}}(\tau), \tag{12.1}$$

where $C_{\text{legibility}}$ penalizes trajectories that are ambiguous with respect to the robot's goal.

Shared autonomy. In shared autonomy, the robot and human share control authority. The robot proposes actions based on its policy π_{robot}; the human provides high-level commands or corrections. The arbitration between human and robot control is continuous and adaptive:

$$a = \alpha(s) \cdot \pi_{\text{robot}}(s) + \big(1 - \alpha(s)\big) \cdot \pi_{\text{human}}(\text{command}, s), \tag{12.2}$$

where $\alpha(s) \in [0, 1]$ is a state-dependent arbitration weight. When the robot is confident (low policy entropy, high value), $\alpha(s) \to 1$ and the robot acts autonomously. When the robot is uncertain, $\alpha(s) \to 0$ and it defers to the human. The arbitration function can be learned or designed based on uncertainty estimates.

12.3 Data: The Defining Bottleneck

Internet-scale text corpora contain trillions of tokens. Robot interaction data is measured in hundreds of thousands of trajectories. This six-order-of-magnitude gap is the fundamental limitation of embodied AI. Four strategies address this gap.

Simulation. Better simulators (SAPIEN, Isaac Sim, MuJoCo) provide ever-more-realistic physics and rendering. Procedural generation of environments and tasks (Infinite World, ProcTHOR, Behavior-1K) creates unlimited training scenarios. The limiting factor is the fidelity of simulation: soft-body dynamics, fluid interactions, and fine-grained tactile sensing remain difficult to simulate accurately. Hybrid approaches—training on simulation with final fine-tuning on real data—balance data abundance with physical fidelity.

Human video. Datasets like Ego4D, Epic Kitchens, and Something-Something contain millions of hours of humans performing everyday tasks. Extracting robot policies from human video requires solving the cross-embodiment mapping problem—a major open challenge. Current approaches learn visual representations from human video (R3M, VIP, VC-1) that transfer to robot tasks, but do not extract full motor policies.

Play data. Unstructured, undirected interaction data—humans teleoperating robots to push, pick, and rearrange objects without specific task goals—is easier to collect than task-specific demonstrations. Goal-conditioned learning from play (Lynch et al., 2020) extracts reusable motor skills from such data. The play paradigm aligns with the self-supervised pre-training approach that proved transformative in NLP and vision.

Fleet learning. Deploying a fleet of robots in real environments enables continuous, parallel data collection. RT-1 collected 130,000 trajectories from 13 robots over 17 months. Scaling this

to hundreds or thousands of robots, each collecting data continuously in diverse homes and workplaces, would begin to approach the data scale needed for robust generalization. The engineering challenges—fleet management, data quality control, privacy preservation—are substantial but solvable.

12.4 The Road to General-Purpose Robots

The dominant bet in embodied AI is on foundation models: pre-train a massive VLA on diverse sensory-motor data, then fine-tune for specific embodiments and tasks. The evidence from RT-2, Octo, and OpenVLA supports this direction. But a deeper question remains unresolved: the **modular vs. end-to-end debate**.

The modular approach decomposes the robot into subsystems: perception, state estimation, planning, control. Each module is designed, trained, and tested independently. This is the architecture of industrial automation and self-driving (Waymo, Cruise). It provides interpretability, separate failure modes, and the ability to hard-code safety constraints.

The end-to-end approach trains a single neural network to map from pixels to actions. Foundation models are end-to-end in the sense that they do not have explicit perception, planning, and control modules. But they are *pre-trained on modular objectives*—the language model's pre-training objectives (next-token prediction, visual grounding) are not the robot's ultimate task but provide useful structure.

The probable resolution is a synthesis: a small number of generalist foundation models serve as the "operating system" for robots, providing visual understanding, language grounding, and motor primitives. Task-specific fine-tuning adds structured planning, safety constraints, and embodiment-specific policies atop this foundation. The robot's "mind" is the VLA; its "skills" are fine-tuned policies that the VLA orchestrates.

12.5 Ethical Considerations

The deployment of embodied AI into human spaces raises ethical questions that the research community must address.

Safety. A robot with sharp tools, heavy payloads, or high speeds can cause injury. Current safety mechanisms are external (cages, light curtains, emergency stops) or software-based (torque limits, speed limits, collision detection). As robots operate in unconstrained home environments among children, pets, and elderly people, software safety must become as reliable as physical safety. Formal verification of learned policies, runtime shielding (an external safety monitor that overrides unsafe actions), and compliant hardware design are complementary approaches.

Privacy. Always-on cameras and microphones in homes raise privacy concerns that are qualitatively different from those of smartphones or smart speakers. Robots see everything in a room, including people in vulnerable states. On-device processing, differential privacy, and user-controlled data deletion are technical mechanisms that can mitigate but not eliminate these concerns.

Labor displacement. Robots that perform physical labor will displace human workers, just as automation has in manufacturing, agriculture, and logistics. The pace and scope of displacement are uncertain, but the historical pattern is clear: technology creates new categories of work even as it eliminates old ones. The transition requires social and economic policy responses that are beyond the scope of the technical literature but must be part of the conversation.

Dual use. The same technology that enables a home assistant robot enables autonomous weapons. The embodied AI community has a responsibility to consider the downstream uses of its research and to engage with policy discussions about regulation, export control, and norms of use.

12.6 Recommended Reading Trajectory

For the reader seeking depth in specific subfields of embodied AI, the following reading trajectory provides a structured path through the literature.

Navigation: Habitat simulator (Savva et al., 2019) → DD-PPO (Wijmans et al., 2020) → SemExp (Chaplot et al., 2020) → PONI (Ramakrishnan et al., 2022) → ZER (Al-Halah et al., 2022).

Vision-and-Language Navigation: Room-to-Room (R2R) benchmark (Anderson et al., 2018) → Speaker-Follower (Fried et al., 2018) → EnvDrop (Tan et al., 2019) → PREVALENT (Hao et al., 2020) → HAMT (Chen et al., 2021).

Manipulation: Dex-Net (Mahler et al., 2017) → Transporter Networks (Zeng et al., 2021) → CLIPort (Shridhar et al., 2022) → SayCan (Ahn et al., 2022) → RT-1 (Brohan et al., 2022) → RT-2 (Brohan et al., 2023) → Octo (Ghosh et al., 2024).

Machine Learning: DAgger (Ross et al., 2011) → GAIL (Ho & Ermon, 2016) → HER (Andrychowicz et al., 2017) → Domain Randomization (Tobin et al., 2017) → R3M (Nair et al., 2022).

Foundation Models: PaLM-E (Driess et al., 2023) → RT-2 (Brohan et al., 2023) → OpenVLA (Kim et al., 2024) → π_0 (Physical Intelligence, 2024).

12.7 Summary

Embodied AI in 2025–2026 is a field of extraordinary progress and enduring challenges. Dexterous manipulation, mobile manipulation, whole-body control, long-horizon autonomy, and natural human interaction define the capability frontier. Data—its scarcity, its cost, and its embodiment-specificity—is the dominant bottleneck. Foundation models, trained on internet data and fine-tuned on robot data, are the leading strategy for building general-purpose robot policies. The modular-vs-end-to-end debate is converging toward a synthesis: foundation models as operating systems for robots, with task-specific fine-tuning for safety, reliability, and embodiment-specific skills. The ethical questions—safety, privacy, labor, dual use—are not secondary; they are integral to the deployment of embodied AI into human society. The next decade will determine whether robots remain laboratory curiosities or become part of the fabric of daily life.